

Code-based Scalable Collaborative SNARKs

Christodoulos Pappas
HKUST
cpappas@connect.ust.hk

Dimitrios Papadopoulos
HKUST
dipapado@cse.ust.hk

Charalampos Papamanthou
Yale University & Lagrange Labs
charalampos.papamanthou@yale.edu

April 14, 2026

Abstract

We propose the first *collaborative SNARK* based on error-correcting codes that is *scalable*, i.e., the proof computation overhead is distributed among the N provers. As a starting point, we introduce the notion of (t, l) -*zero-knowledge collaborative codes* that ensure that, when collaboratively computing a codeword over a distributed message, no coalition of up to t corrupted parties learns any additional information about the message, even having queried up to l codeword positions. We show that tensor codes consisting of the composition of two Reed-Solomon codes satisfy our definition, while also being *foldable*. We then propose a *collaborative interactive oracle proof of proximity (coIOPP)* for testing codeword closeness in our code, show how it can be made a zero-knowledge IOPP using randomness logarithmic in the size of the message (as opposed to linear with prior approaches), and we use it to construct a coIOPP for multi-linear polynomial evaluation. To compile our coIOPPs into non-interactive arguments, we prove that a natural extension of the compiler of Ben-Sasson-Chiesa-Spooner (TCC 2016) in the collaborative setting preserves round-by-round (knowledge) soundness against quantum adversaries, which may be of independent interest for future work in collaborative SNARKs. Finally, we use an optimized collaborative version of the Spartan PIOP to build the first *transparent* and *post-quantum secure* scalable collaborative SNARK. Our experimental evaluation demonstrates that our scheme consistently outperforms the best existing (non-post-quantum secure) scalable collaborative SNARKs, both in end-to-end prover time and in total communication among provers, for all tested configurations.

1 Introduction

Collaborative Succinct Non-Interactive Arguments of Knowledge (coSNARKs) are distributed protocols that allow a set of provers to interact in order to collectively produce a succinct proof for the statement $(\mathbf{x}, \mathbf{w} = (\mathbf{w}_1, \dots, \mathbf{w}_N)) \in \mathcal{R}$, where $\mathbf{w}_i$ is a part of the witness privately held by prover $\mathcal{P}_i$. A coSNARK should not only satisfy “standard” zero-knowledge, i.e., the final proof should not reveal anything about $\mathbf{w}$ to the verifier $\mathcal{V}$, but should also guarantee that the interaction among $\mathcal{P}_i$ ’s reveals nothing about the private witnesses of honest provers to a coalition of up to t corrupted provers, referred to as *t-zero knowledge*. This notion was first introduced by Ozdemir and Boneh [1], who adapted the provers of standard zkSNARKs (Groth16 [2], Marlin [3], Plonk [4] and Fractal [5]) in the collaborative setting, by introducing optimized multi-party computation (MPC) protocols for performing multi-scalar multiplication (MSM) over elliptic curves, polynomial division, and FFT.

Subsequent works along the same lines [6, 7, 8] proposed optimized collaborative versions of Spartan and Marlin.

Scalable collaborative SNARKs. In terms of performance, all the coSNARK works mentioned above require each $\mathcal{P}_i$ to spend time asymptotically the same as if running a (centralized) zkSNARK. That is, if the provers are interacting to prove the evaluation of a circuit C that takes as input $(\mathbf{x}, \mathbf{w})$, each party is running in time $\Omega(|C|)$. Simply put, there is no performance benefit from running a collaborative SNARK—just the strong privacy guarantee of not revealing individual witness parts. This, might be suitable for application scenarios where a set of mutually distrustful institutions/companies need to collaborate to perform a joint computation and produce a publicly verifiable proof of the result, e.g., publicly verifiable proof-of-inference over private ML models and inputs [9, 10], or hospitals collaborating to provably compute sensitive healthcare statistics (see [1] for a discussion of other applications).

On the other hand, more recent works in this area proposed the notion of *scalable coSNARKs* [11, 12, 13, 14]. In this paradigm, the overhead of running a centralized zkSNARK is distributed among the N provers, instead of being replicated. For a given computation, each prover’s overhead *decreases* inversely to N , while still achieving t -zero knowledge. Apart from the obvious efficiency benefit for the above applications, this is also significant for a *delegated* setting, where a single party wishes to outsource the heavy computational task of computing a zero-knowledge proof to multiple third parties, without revealing the witness directly. Scalable coSNARKs ensure that the delegator’s cost is (asymptotically) the same as delegating to a single prover, while scalability ensures the task will finish much faster.

A first attempt towards this direction was made by Garg et al. [11] who observed that certain common SNARK operations (such as MSM) exhibit a SIMD (single-instruction-multiple-data) structure. Using *packed secret sharing* [15] and specialized MPC protocols for these operations, the authors were able to achieve sublinear complexity per prover. However, their schemes involved FFT operations for which one of the provers needs to spend time quasi-linear to $|C|$.

More recently, Liu et al. [12, 13] proposed the first fully scalable coSNARKs, where all the provers need time sublinear to $|C|$. Instead of using zkSNARKs based on FFTs as done in prior work [11], they rely on proof systems based on the sumcheck protocol [16] and utilize the SIMD structure of its prover to instantiate a scalable collaborative sumcheck protocol. Then, they combine their sumcheck with a scalable variant of the PST *polynomial commitment scheme* (PCS) [17] to build a collaborative version of Libra [12] and Hyperplonk [13]. Unfortunately, the MPC protocol they use for the MSMs needed for PST is not compatible with the optimized Pippenger’s algorithm [18], and this leads to significant prover slowdown. The work of Fang et al. [14] proposes a collaborative bivariate KZG [19] version for the PCS and the authors set up its parameters in a way that makes it possible to apply Pippenger’s algorithm over packed secret shared values, achieving better concrete prover times. Moreover, they adapt Pianist [20], a distributed Plonk-based zkSNARK, in the collaborative setting, achieving constant proof size and verification time.

Limitations of Existing Works. Observe that all existing scalable coSNARKs, operate over elliptic curve (EC) groups. To some extent, it is not surprising that we can build such schemes using the “classic” polynomial interactive oracle proof + polynomial commitment scheme (PIOP+PCS) framework: If one uses a parallel-friendly PIOP (as in the above works), the main remaining issue is identifying a PCS that is also parallel-friendly. There do exist several examples of EC-based PCS that satisfy this, e.g., PST (in [12, 13]), and univariate (in [11]) or bivariate (in [14]) KZG.

Unfortunately, such schemes face the following crucial limitations. First, their security is based on hardness assumptions that are known to not hold against adversaries with quantum computing capabilities. Next, performance-wise, they tend to have slower provers than alternative constructions,

such as code-based ones [21], as they rely on heavy cryptographic operations like MSMs over elliptic curves (see [22] for a detailed discussion). Finally, the EC-based PC schemes used in these works require a trusted setup to generate their public parameters.

What about code-based scalable coSNARKs? Based on the above, we are motivated to design scalable schemes that overcome these limitations. In the non-collaborative setting, the most widely used class of such schemes is arguably ones based on error-correcting codes (e.g., [23, 22, 21, 24]). The main challenge one needs to address in that case is how to efficiently compute codewords in a collaborative manner. Unfortunately, there is no existing solution in the literature for this task that simultaneously achieves: (i) per-party computation that is sublinear to the codeword size, and (ii) relative distance that does not decrease inversely proportionally to the number of parties N ¹.

For instance, the collaborative variant of Fractal [5] proposed in [1] (based on Reed-Solomon codes [25]), which, to our knowledge, is the only existing transparent and post-quantum coSNARK, does not achieve (i) (nor (ii)), as it requires all parties to spend time proportional to the codeword size. This holds even when using the state-of-the-art protocol for collaboratively computing RS codes as proposed in [11].

In fact, this is a challenging task even if one ignores the privacy requirement. The majority of existing works in code-based *distributed SNARKs* [26, 27, 28, 29] (i.e., scalable but not hiding each prover’s witness) adopt an *interleaved code* [30, 22] approach where each party computes the codeword of its input locally. Unfortunately, this violates (ii) as the resulting interleaved code has a much smaller relative distance than the initial codes, hence it again leads to non-succinct schemes with proof proportional to Nl . One exception is the scheme of Li et al. [29] that uses a distributed RS encoding algorithm, but it is not clear how to make their solution privacy-preserving while maintaining scalability, for the reason discussed above.

1.1 Our Results

In this work, we propose the first transparent and provably post-quantum secure scalable coSNARK.

Collaborative Codes. Our starting point is to introduce the definition of *collaborative codes* (Section 3.1). At a high level, collaborative codes enable multiple parties to compute a codeword over their private inputs such that corrupted parties can learn nothing about the inputs of the honest ones even if they make a bounded number of queries on the code.

One natural approach to constructing such a collaborative code is by running the encoding algorithm of an existing code using an MPC protocol. In fact, this approach has been explored in previous work [1]; however, it faces the following key limitation: lack of scalability. This mainly stems from the fact that the encoding algorithms of the vast majority of codes nowadays deployed in zkSNARKs such as Reed-Solomon, Spielman, Expand-Accumulate codes [25, 31, 32] do not have a strictly SIMD structure²—desirable for scalable MPC protocols [33] (see Section 3.1 for a more detailed discussion). This raises the following intriguing question: “*Is it possible to construct a scalable collaborative code with “large” minimum relative distance?*”.

In this work, we answer this question affirmatively by introducing the first fully scalable collaborative code with per-party encoding time of $\mathcal{O}(|\mathbf{x}| \log |\mathbf{x}|/N)$, where $|\mathbf{x}|$ the “global” message size and N the number of parties, and a *single communication round* between parties of $\mathcal{O}(|\mathbf{x}|)$ total communication complexity³. At a high level, our construction relies on two main observations. Initially, we observe that there is a family of codes, that of *tensor-codes* that has both large minimum

¹We note that (ii) is very important when building a coSNARK as the number of queries l would increase multiplicatively with N making the final proof size to be at least Nl —which is concretely large.

²A strictly SIMD structure corresponds to circuits consisting of multiple identical sub-circuits with no shared wires.

³This communication round can be omitted if parties hold secret shares of the “global” input $\mathbf{x}$, as in our coSNARK.

Table 1: Asymptotic comparison of scalable coSNARKs. For [13, 14], $\mathbb{F}$ is a field compatible with pairing-friendly curves. We denote with N the number of parties, $|C|$ and d the circuit size and depth. The prover overhead of [13] is proportional to $\log |\mathbb{F}|$ because of a naive MSM over a vector of size $|C|/N$. For SIMD circuits [13] has total communication complexity of $N^2 \log |C|$. Finally, with PQS we denote Post-Quantum Security.

Scheme	Prove	Verify	$ \pi $	Total Comm.	Rounds	Transp.	PQS
Liu et al. [13]	$\frac{ C }{N} \log \mathbb{F} $	$\log C $	$\log C $	$ C $	$\log C $	×	×
Fang et al. [14]	$\frac{ C }{N} \log C \log \mathbb{F} $	1	1	$ C $	1	×	×
Ours	$\frac{ C }{N} \log C $	$\log^2 C + N$	$\log^2 C + N$	$ C $	$\log C $	✓	✓
Liu et al. [12] (SIMD)	$\frac{ C }{N} \log \mathbb{F} $	$d \log C $	$d \log C $	$dN^2 \log C $	$d \log C $	×	×
Ours (SIMD)	$\frac{ C }{N} \log C $	$\log^2 C + N$	$\log^2 C + N$	$N^2 \log C $	$\log C $	✓	✓

distance, and an encoding algorithm with “SIMDish” structure. This makes tensor-codes the perfect candidate for constructing collaborative codes. The second and most important observation is that a tensor-code composed of two RS codes can be naturally extended in the collaborative setting, in the sense that one can run its (randomized [34]) encoding algorithm “as is”, without violating privacy and most importantly without the need of any MPC machinery! Last but not least, we also show that this family of tensor-codes is *foldable* [35, 23], with a folding algorithm that requires no communication between parties. Looking ahead, this property is desirable for constructing interactive oracle proofs of proximity and hash-based PCs with poly-logarithmic proof size and verification times.

Building coIOPPs from our Collaborative Code. We then propose a *collaborative interactive oracle proof of proximity (coIOPP)* for checking codeword closeness for our code, which is essentially a collaborative version of FRI [36]. To reduce the size of the randomness needed to make FRI zero-knowledge when following the approaches from the non-collaborative literature [37, 38], we propose a new way to apply the masking polynomials. This reduces the overall randomness size to $\mathcal{O}(Nl \log |\mathbf{x}|)$ (as opposed to $\mathcal{O}(|\mathbf{x}|)$), where N is the number of parties, l is the number of queries per FRI round, and $\mathbf{x}$ is the message. Next, we need to turn this into a fully blown collaborative PCS. For this, we follow the approach of Basefold [23] with appropriate modifications to make it consistent with our modified FRI above (Section 5.2). We also give detailed proofs for t -zero-knowledge and round-by-round (knowledge) soundness [39]—a strengthening of standard soundness ensuring soundness when making our schemes non-interactive with a protocol that we will introduce next.

In the “standard” (non-collaborative) setting, one would use the classic BCS transformation of Ben-Sasson, Chiesa, and Spooner [40] to compile an interactive oracle proof into a non-interactive polynomial commitment scheme, replacing the oracles by a Merkle tree instantiated with a random oracle. As an independent contribution, we show how an extension of the BCS transformation preserves this compilation in the collaborative setting (see Section 4). Namely, we show that if the coIOP has negligible round-by-round (knowledge) soundness error and is honest-verifier t -zero-knowledge, then the resulting collaborative non-interactive version also has negligible round-by-round (knowledge) soundness error (even against quantum) adversaries, and is t -zero-knowledge (even against malicious provers).

Our coSNARK. Finally, we propose a collaborative PIOP for circuit satisfiability. As a starting point, we introduce a novel collaborative sumcheck protocol that maintains the same asymptotics as prior works [12, 13], but can be shown secure solely in the random oracle model (ROM). Using our coSumcheck we then build our coPIOP. For this, first recall that all existing scalable coSNARK works propose a coPIOP for the Plonk constraint system. Due to its gate consistency check, Plonk

necessitates the provers to perform a multiplication of four packed secret shared vectors. Because of this, to be able to finally reconstruct the secret, in [13] the degree of the interpolating polynomial is decreased by a factor of $1/4$, which limits the corruption threshold t to $t \leq (1/4 - \epsilon)N$, where ϵN is the packing factor of the packed secret sharing scheme. The authors of [14] achieve a better threshold of $t \leq (1/2 - \epsilon)N$, at the cost of increasing the multiplicative depth of their coPIOP. In practice, this blows up the total communication as each multiplication level requires an additional round of $\mathcal{O}(|C|)$ interaction. Finally, that scheme relies on an expensive MPC protocol for computing inverses of packed secret shared elements, needed for enforcing wire consistency.

Instead, in this work, we construct a scalable coPIOP for the R1CS constraint system. As we show, this only requires multiplication of at most two packed secret shared vectors, which gives us the best of both approaches. We achieve $t \leq (1/2 - \epsilon)N$ corruption threshold without the need to increase total communication size. We note that (non-scalable) coPIOPs for R1CS already exist [11, 7, 1, 6]. Our starting point is the most recent of these works, DFS [6], which relies on a collaborative version of Spartan [41], but we use our own scalable collaborative sumcheck. While this suffices to make coPIOP scalable, we further propose a novel distributed protocol for proving sparse polynomial evaluation of the R1CS matrixes. At its core lies a new distributed protocol for the grand product argument of [42]. Naively, a distributed variant of [42] requires $\mathcal{O}(\log^2 |C|)$ communication rounds, and thus hardly scales in a distributed setting despite its supreme performance in the standard setting [6]. Here, we show how to reduce that number to $\mathcal{O}(\log |C|)$ by proposing a distributed sumcheck protocol, inspired by Sparrow’s sumcheck [43], that requires only $\mathcal{O}(1)$ rounds of interaction!

Table 1 shows a detailed comparison between our coSNARK and prior works. Besides being the only transparent and post-quantum secure scalable scheme, our coSNARK asymptotically (slightly) outperforms previous works in terms of prover time. In addition, while the total communication complexity is the same in all works, ours is concretely smaller compared to [14, 13]. Finally, when applied for SIMD circuits, our coSNARK achieves significantly reduced communication.

Making our Construction Secure Against Malicious Provers Throughout the paper, we assume that corrupted provers are honest-but-curious. We note, however, that, similar to prior works on scalable coSNARKs [12, 13], our protocols can be made maliciously secure using the compiler of [44] that works over Shamir’s packed secret shares. Performance-wise, this incurs an overhead of at most $2\times$ in prover time and communication size. Moreover, we believe that using the observations of Garg et al. [45], our scheme can be made maliciously secure “for free” since it can be viewed as an MPC protocol evaluating a reactive function composed by affine or degree-two computations. We leave a formal proof of this claim as future work.

Experimental Evaluation. We implemented our coSNARK and benchmarked its performance (Section 7) in both LAN and WAN settings. First, we compare our scheme with the state-of-the-art scalable coSNARK of [14], when proving the correct computation of a circuit of size ranging from $|C| = 2^{21}$ - 2^{25} and $N = 8, 32, 128$ provers. Our coSNARK is $\times 8.6$ - 10.3 faster in LAN and $\times 2$ - 5.9 faster in WAN (with round-trip-delay of 100ms) than that of [14]. For instance, for $N = 128$, $|C| = 2^{24}$, the per-prover time of our scheme is only 22.9sec in LAN (and 47sec in WAN) compared to the 236.5sec (and 279.4sec) of [14]. In addition, the total communication cost and space utilization are roughly $\times 36.2$ and $\times 8.3$ smaller than [14] making it significantly more scalable! That said, our proof size and verification time are larger compared to [14], which is expected due to their use of a EC-based PC scheme. Furthermore, we also compare our scheme with the centralized (non-private) Spartan zkSNARK implemented with the state-of-the-art WHIR PCS [21]. For any number of parties $N > 4$, our scheme becomes faster Spartan+WHIR. E.g, for $N = 128$ provers and $|C| = 2^{23}$, our scheme takes 11.3sec in LAN (and 31.7sec in WAN), whereas the centralized prover of Spartan+WHIR takes 291.5sec. We provide an open source implementation of our coSNARK in [46].

1.2 Other Related Works

Early attempts to distribute a zkSNARK prover (in the delegation scenario) include Trinocchio [47] and its follow-up [48] that proposed secret-sharing-based versions of [49]. However, these schemes are not scalable. Another relevant line of works, that actually predate the notion of collaborative SNARKs, is that of *publicly auditable MPC (PA-MPC)* protocols [50]. These works focus on producing a proof for the correctness of the result of running an MPC protocol that can then be verified by a third-party auditor. While the majority of works in this area produce proofs linear in size to the entire MPC transcript [51, 52], subsequent works produced PA-MPC with succinct proofs, by producing distributed MPC versions of zkSNARK prover operations [53]. To the best of our knowledge, none of these works achieves our scalability requirements, having the same limitations as the non-scalable coSNARKs discussed above.

2 Preliminaries

Notation. We define the set $[n] = \{1, \dots, n\}$ and let $\mathbb{F}$ be a field of prime order p . A vector is represented with $\mathbf{x} = (x_1, \dots, x_n) \in \mathbb{F}^n$ and $\mathbf{x}[i]$ its i -th element. We use bold capital letters to represent matrices; and denote with $\mathbf{A}[i, :]$ and $\mathbf{A}[:, i]$ is i -th row and column respectively. Finally, for an index i, j we use $\mathbf{i}, \mathbf{j}$ to denote its bit-decomposition.

Multi-Linear Extensions. A *multi-linear extension* of a vector $\mathbf{x} : \{0, 1\}^{\log n} \rightarrow \mathbb{F}$ is a multi-linear polynomial $f_{\mathbf{x}} : \mathbb{F}^{\log n} \rightarrow \mathbb{F}$, such that $f_{\mathbf{x}}(\mathbf{z}) = \sum_{\mathbf{i} \in \{0, 1\}^{\log n}} \beta(\mathbf{z}, \mathbf{i}) \mathbf{x}[\mathbf{i}]$, where $\beta(\mathbf{z}, \mathbf{i}) = \prod_{k \in [\log n]} (\mathbf{i}[k] \mathbf{z}[k] + (1 - \mathbf{z}[k])(1 - \mathbf{i}[k]))$.

2.1 Error-Correcting Codes

An error-correcting code is an injective map $\mathcal{C} \in \Sigma^n \rightarrow \Sigma^{n'}$, where Σ is an alphabet, n is the dimension of the domain vector, called *message*, and n' the dimension of the image vector called *codeword*. We denote with $\rho = \frac{n}{n'}$ the rate of the code and Δ the relative minimum distance defined as $\min_{\mathbf{c}_1, \mathbf{c}_2} \{\Delta(\mathbf{c}_1, \mathbf{c}_2)/n'\}$, for all $\mathbf{c}_1, \mathbf{c}_2 \in \mathcal{C}$ such that $\mathbf{c}_1 \neq \mathbf{c}_2$ and $\Delta(\mathbf{c}_1, \mathbf{c}_2)$ their Hamming distance. We will specifically focus on *linear error-correcting codes*, namely codes such that $\mathcal{C}$ is a linear subspace of $\Sigma^{n'}$.

Zero-Knowledge Codes. In this work, we build upon a special type of error-correcting codes called *l-zero-knowledge codes* first proposed in [34]. These codes have the additional property that accessing at most l elements of the codeword reveals nothing about the original message.

Tensor-Codes [54]. A tensor-code over two codes $\mathcal{C}_1 : \Sigma^{m_1} \rightarrow \Sigma^{m'_1}$ and $\mathcal{C}_2 : \Sigma^{m_2} \rightarrow \Sigma^{m'_2}$ is a code $\mathcal{C} : \mathcal{C}_1 \otimes \mathcal{C}_2 : \Sigma^{m_1 m_2} \rightarrow \Sigma^{m'_1 m'_2}$, with an encoding algorithm that works as follows. First it parses $\mathbf{x}$ as a matrix $\mathbf{X} \in \mathbb{F}^{m_1 \times m_2}$ and computes the (intermediate) matrix $\mathbf{X}' \in \mathbb{F}^{m'_1 \times m_2}$ by encoding each column of $\mathbf{X}$ using the encoding algorithm of $\mathcal{C}_1$. Finally, it computes the codeword $\mathbf{C} \in \mathbb{F}^{m'_1 \times m'_2}$ by encoding each row of $\mathbf{X}'$ using $\mathcal{C}_2$.

Remark 1. Let ρ_1, ρ_2 be the rates and Δ_1, Δ_2 the relative minimum distances of $\mathcal{C}_1$ and $\mathcal{C}_2$ respectively. Then, the rate and relative minimum distance of $\mathcal{C}$ will be $\rho_1 \rho_2$ and $\Delta_1 \Delta_2$. Furthermore, if $\mathcal{C}_1, \mathcal{C}_2$ are linear codes, then $\mathcal{C}$ is also linear [54].

2.2 Secret Sharing

In our work, we rely on Shamir's packed secret sharing [15]. Shamir's packed secret sharing enables a dealer to secret share a vector $\mathbf{x} \in \mathbb{F}^k$ to N parties (where $k \leq N$). To achieve this, the dealer picks

a random polynomial $f : \mathbb{F} \rightarrow \mathbb{F}$ of degree d , such that $f(-i) = \mathbf{x}_i$, for all $i \in [k]$ and then sends $f(i)$ to the i -th party for all $i \in [N]$. We use $\llbracket \mathbf{x} \rrbracket^d$ to denote a secret sharing of $\mathbf{x}$ from a d -degree polynomial and $\llbracket \mathbf{x} \rrbracket_i^d$ its i -th share (but we will usually omit d). This packed secret sharing scheme is secure up to $d - k + 1$ corrupt parties [15], that is any shares up to $d - k + 1$ reveal nothing about the original vector $\mathbf{x}$. Finally, we note that this secret sharing is both additively and multiplicatively homomorphic. Namely, $\llbracket \mathbf{x}_1 \rrbracket^d + \llbracket \mathbf{x}_2 \rrbracket^d = \llbracket \mathbf{x}_1 + \mathbf{x}_2 \rrbracket^d$ and $\llbracket \mathbf{x}_1 \rrbracket^d \cdot \llbracket \mathbf{x}_2 \rrbracket^d = \llbracket \mathbf{x}_1 \circ \mathbf{x}_2 \rrbracket^{2d}$ where $\circ$ denotes element-wise multiplication.

Remark 2. Shamir's packed secret shares $\{f(1), \dots, f(N)\}$, constitute a $d - k + 1$ -zero knowledge Reed-Solomon code with $\rho = \frac{d}{N}$ rate and $\Delta = \frac{N-d}{N}$ relative minimum distance [55].

2.3 (Polynomial) Interactive Oracle Proofs

We outline the main primitives our construction is based on.

Interactive Oracle Proofs. *Interactive Oracle Proofs* [40, 37] for a relation $\mathcal{R}$, with language $\mathcal{L}_{\mathcal{R}} = \{\mathbf{x} \mid \exists \mathbf{w}, (\mathbf{x}, \mathbf{w}) \in \mathcal{R}\}$ are k -round information theoretic protocols between a prover and a verifier that can be considered as a hybrid between interactive proofs [56] and probabilistically checkable proofs [57]. In more detail, the prover, provided $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$, sends an oracle of the string $\pi^{(0)}$ to the verifier that only holds $\mathbf{x}$. Then, in every round $i \in [k]$ the verifier sends a challenge r_i and the prover replies with an oracle of the string $\pi^{(i)}$. The verifier makes some queries on the oracles of $(\pi^{(0)}, \dots, \pi^{(k)})$ and decides whether to accept. A IOP is complete if the verifier always accepts a valid $\mathbf{x}$ and ϵ -sound if it accepts an $\mathbf{x} \notin \mathcal{L}_{\mathcal{R}}$ with probability at most ϵ .

IOPs of Proximity. If in the IOP the prover sends the oracle of $\mathbf{w}$ and the verifier is allowed to make a sublinear number of queries over that oracle, then we say that the IOP is IOP of proximity. Completeness is identical to that of IOP. Soundness states that if $\mathbf{w}$ is δ -far from a valid witness with respect to the relative Hamming distance, then the verifier accepts with probability at most $\epsilon(\delta)$.

Polynomial IOPs. A polynomial IOP (PIOP) is a k -round protocol between a prover and a verifier that is almost identical to an IOP with the exception that the prover can also send an oracle of a polynomial f_i of total degree d_i at the i -th round. Furthermore, the verifier can ask that oracle for evaluations of f_i at random points. Completeness and soundness can be derived from the definition for IOPs.

Polynomial Commitment Schemes. A polynomial commitment scheme (PCS) enables a prover to commit to a polynomial $f : \mathbb{F}^n \rightarrow \mathbb{F}$, and later convince a verifier f has been correctly evaluated at a point $\mathbf{t} \in \mathbb{F}^n$, by generating a small and fast to verify evaluation proof π . A PCS is *knowledge sound* if for any probabilistic polynomial time adversary producing an accepting proof, there exists an extractor $\mathcal{E}_{PC}$ that extracts the committed polynomial f , such that the probability that $f(\mathbf{r}) \neq y$ is negligible. A PCS is *complete* if the verifier always accepts the proof for a correctly evaluated point and *zero-knowledge* if it learns nothing more besides the evaluation.

From IOPP to PCS. To build our collaborative PCS, we will follow the framework formalized by [24] that builds a PCS for multi-linear polynomials from IOPPs. In more detail, the authors of [24] build an IOPP that checks if the oracle of a string $\mathbf{C}'$ is δ -close to a codeword $\mathbf{C}$ and that the multi-linear extension of the underlying message of $\mathbf{C}$ correctly evaluates at y for the evaluation point $\mathbf{t}$. Then, they transform this IOPP into a PCS using Fiat-Shamir.

From PIOPs to SNARKs. Using a PCS, any PIOP can be compiled into an argument of knowledge by replacing the polynomial oracles and queries with the PCS commitment and evaluation algorithm [58, 3].

2.4 Collaborative SNARKs and collaborative PC Schemes

Collaborative SNARKs [1, 7, 11]. They are non-interactive arguments allowing N mutually distrustful provers to produce a proof showing that they collectively know a witness $\mathbf{w} = (\mathbf{w}_1, \dots, \mathbf{w}_N)$ for an NP relationship $\mathcal{R}$, where $\mathbf{w}_i$ is the private witness of the i -th party. Besides completeness and knowledge soundness, coSNARKs satisfy t -zero knowledge: up to t corrupted parties learn nothing about the private witness of honest ones. More formally:

Definition 1. *The set of PPT algorithms $\{\mathcal{G}, \text{Prove}, \text{Verify}\}$ is a coSNARK for an NP relationship $\mathcal{R}$ and secure up to $t \leq N - 1$ corruptions, if the following properties hold:*

1. *Completeness: For any security parameter λ , relationship $\mathcal{R}$ and $(\mathbf{x}, \mathbf{w} = (\mathbf{w}_1, \dots, \mathbf{w}_N)) \in \mathcal{R}$, it holds that:*

$$\Pr \left(\begin{array}{l} pk, vk \leftarrow \mathcal{G}(1^\lambda, \mathcal{R}) \\ \pi \leftarrow \text{Prove}^{\{\mathcal{P}_1, \dots, \mathcal{P}_N\}}(pk, \mathbf{x}, (\mathbf{w}_1, \dots, \mathbf{w}_N)) \\ 1 = \text{Verify}(vk, \mathbf{x}, \pi) \end{array} \right) = 1$$

Where $\langle \cdot, \cdot \rangle$ denotes an interactive protocol between parties, t is the proof transcript between all provers and the verifier and 1 indicates that $\mathcal{V}$ accepted the proof, namely, t .

2. *Knowledge Soundness: For any λ , NP relationship $\mathcal{R}$, instance $\mathbf{x}$ and $pk, vk \leftarrow \mathcal{G}(1^\lambda, \mathcal{R})$ there exists a PPT extractor $\mathcal{E}$ that has (rewinding) access to all provers $\vec{\mathcal{P}}$ such that the following probability is negligible:*

$$\Pr \left(\begin{array}{l} 1 = \text{Verify}(vk, \mathbf{x}, \pi) \wedge (\mathbf{x}, \mathbf{w}) \notin \mathcal{R} : \\ \pi \leftarrow \text{Prove}^{\{\mathcal{P}_1, \dots, \mathcal{P}_N\}}(pk, \mathbf{x}, (\mathbf{w}_1, \dots, \mathbf{w}_N)), \\ \mathbf{w} \leftarrow \mathcal{E}^{\{\mathcal{P}_1, \dots, \mathcal{P}_N\}}(pk) \end{array} \right)$$

3. *t -zero knowledge: A collaborative argument of knowledge is t -zero knowledge if for any PPT adversary $\mathcal{A}$ that corrupts at most t provers as well as the verifier, then, there exists a PPT simulator $\mathcal{S}$ such that the following views are computationally indistinguishable:*

- *The view $\text{View}_{\mathcal{A}}^R(\mathbf{x}, \mathbf{w}, pk, vk)$ of the adversary, that is the list of messages all corrupted provers and the verifier receive when interacting with the honest provers.*
- *The view $\text{View}_{\mathcal{A}}^S(\mathbf{x}, pk, vk)$ of the adversary when interacts with simulator $\mathcal{S}$.*

4. *Succinctness: The proof size and verification times are poly-logarithmic to $|\mathbf{w}|$.*

Collaborative PC Schemes. Likewise, we propose coPC schemes for multi-linear polynomials but one can trivially extend the definition for multi-variate polynomials.

Definition 2 (Collaborative Polynomial Commitment Scheme). *A collaborative PCS is a tuple of algorithms $(\text{Gen}, \text{Commit}, \text{Eval}, \text{Verify})$ defined as follows:*

- $pk, vk \leftarrow \text{Gen}(1^\lambda, M)$: *Given the security parameter λ and the maximum size (i.e., number of coefficients) of the polynomial to be committed, generate a proving and verification key.*
- $C_f \leftarrow \text{Commit}^{\{\mathcal{P}_1, \dots, \mathcal{P}_N\}}(pk, \mathbf{x}_1, \dots, \mathbf{x}_N)$: *An MPC protocol between N provers $\mathcal{P} = \{\mathcal{P}_1, \dots, \mathcal{P}_N\}$. Each $\mathcal{P}_i$ holds its private input $\mathbf{x}_i$ and all provers collaborate to output a polynomial commitment of $f \in \mathbb{F}^{\log M} \rightarrow \mathbb{F}$, the multi-linear extension of $\mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_N) \in \mathbb{F}^M$.*
- $\pi, y \leftarrow \text{Eval}^{\{\mathcal{P}_1, \dots, \mathcal{P}_N\}}(pk, \mathbf{x}_1, \dots, \mathbf{x}_N, \mathbf{t})$: *An MPC protocol between the provers for generating a proof π showing that $f(\mathbf{t}) = y$.*

- $0, 1 \leftarrow \text{Verify}(\mathbf{vk}, C_f, \pi, \mathbf{t}, y)$: Given the PC commitment C_f , an evaluation proof and the evaluation pair $(\mathbf{t}, y)$, the algorithm returns 1 if and only if $f(\mathbf{t}) = y$

A collaborative PCS must satisfy the following:

- **Completeness**: For all $(\mathbf{x}_1, \dots, \mathbf{x}_N)$, $\mathbf{t} \in \mathbb{F}^{\log M}$ and y such that $f(\mathbf{t}) = y$, it holds that:

$$\Pr \left(\begin{array}{l} \mathbf{pk}, \mathbf{vk} \leftarrow \text{Gen}(1^\lambda, M) \wedge \\ C_f \leftarrow \text{Commit}^{\{\mathcal{P}_1, \dots, \mathcal{P}_N\}}(\mathbf{pk}, \mathbf{x}_1, \dots, \mathbf{x}_N) \wedge \\ \pi \leftarrow \text{Eval}^{\{\mathcal{P}_1, \dots, \mathcal{P}_N\}}(\mathbf{pk}, \mathbf{x}_1, \dots, \mathbf{x}_N, \mathbf{t}) \wedge \\ 1 \leftarrow \text{Verify}(\mathbf{vk}, C_f, \pi, \mathbf{t}, y) \end{array} \right) = 1$$

- **Knowledge Soundness**: For all $(\mathbf{x}_1, \dots, \mathbf{x}_2)$ and for any set of PPT adversaries $\mathcal{P}^* = \{\mathcal{P}_1^*, \dots, \mathcal{P}_N^*\}$, there exists a PPT extractor $\mathcal{E}$ with the ability to rewind $\mathcal{P}^*$, such that:

$$\Pr \left(\begin{array}{l} \mathbf{pk}, \mathbf{vk} \leftarrow \text{Gen}(1^\lambda, M) \wedge \\ C_f, \mathbf{t}, y, \pi \leftarrow \mathcal{P}^*(\mathbf{pk}, \mathbf{vk}) \wedge \\ \mathbf{x} \leftarrow \mathcal{E}^{\mathcal{P}^*}(\mathbf{pk}, \mathbf{vk}) : \\ 1 \leftarrow \text{Verify}(\mathbf{vk}, C_f, \pi, \mathbf{t}, y) \wedge \\ f(\mathbf{t}) \neq y \vee C_f \neq \text{Commit}(\mathbf{pk}, \mathbf{x}) \end{array} \right) < \text{negl}(\lambda)$$

- **Honest Verifier t -Zero-Knowledge**: For any PPT adversary controlling at most t adversaries $\{\mathcal{P}_{i_1}, \dots, \mathcal{P}_{i_t}\}$ as well as the verifier $\mathcal{V}$, there exists a PPT simulator $\mathcal{S}$ (with oracle access to the multi-linear extension of $\mathbf{x}$) such that:

$$\Pr(\mathbf{Real}_{A,f}(1^\lambda, \mathbf{pk}, \mathbf{vk})) \stackrel{c}{=} \Pr(\mathbf{Ideal}_{A,S}(1^\lambda, \mathbf{pk}, \mathbf{vk})) \quad (1)$$

Where the **Ideal** and **Real** games are described bellow. We denote as $\mathbf{tr}$ the interaction transcript and $\text{View}_{\mathcal{P}^*, \mathcal{V}}^{\Pi_{\text{Eval}}}(\mathbf{x})$ is the accumulated view of all parties controlled by $\mathcal{A}$ when interacting via the protocol Π_{Eval} over the input $\mathbf{x}$.

$\mathbf{Real}_{A,f}(1^\lambda, \mathbf{pk}, \mathbf{vk})$: 1. $C_f \leftarrow \Pi_{\text{Commit}}^{\mathcal{P}}(\mathbf{pk}, \mathbf{x}_1, \dots, \mathbf{x}_N)$ 2. $\mathbf{t} \leftarrow \mathcal{A}(\mathbf{pk}, \mathbf{vk}, C_f)$ 3. $\pi \leftarrow \text{View}_{\mathcal{P}^*, \mathcal{V}}^{\Pi_{\text{Eval}}}(\mathbf{x})$ 4. Output $b \leftarrow \mathcal{A}(1^\lambda, C_f, y, \pi, \mathbf{pk}, \mathbf{vk})$.	$\mathbf{Ideal}_{A,S}(1^\lambda, \mathbf{pk}, \mathbf{vk})$: 1. $C_f \leftarrow \mathcal{S}_1(1^\lambda, M, \mathbf{x}_{i_1}, \dots, \mathbf{x}_{i_t})$ 2. $\mathbf{t} \leftarrow \mathcal{A}(\mathbf{pk}, \mathbf{vk}, C_f)$ 3. $\pi \leftarrow \mathcal{S}_2(\mathbf{pk}, \mathbf{vk}, \mathbf{x}_{i_1}, \dots, \mathbf{x}_{i_t}, \mathbf{r}, f(\mathbf{r}))$ 4. Output $b \leftarrow \mathcal{A}(1^\lambda, C_f, y, \pi, \mathbf{pk}, \mathbf{vk})$.
--	---

2.5 Round-by-Round (Knowledge) Soundness

Definition 3 (Round-by-Round Soundness). An IOP for the relation $\mathcal{R}$ has round-by-round soundness error ϵ_{rbr} if there exists a doomed set $\mathcal{D}$ of partial and complete transcripts such that:

1. If $\mathbf{x} \notin \mathcal{L}_{\mathcal{R}}$, then $(\mathbf{x}, \emptyset) \in \mathcal{D}$.
2. For a complete proof transcript $\mathbf{t}$, if $(\mathbf{x}, \mathbf{t}) \in \mathcal{D}$ then the verifier rejects.

3. For every $i \in [k]$, and any partial transcript $\mathbf{t}_i$, such that $(\mathbf{x}, \mathbf{t}_i) \in \mathcal{D}$, then for any possible reply $\pi^{(i+1)}$ it holds that:

$$\Pr_{r_{i+1} \xleftarrow{\$} \mathbb{F}} [(\mathbf{t}_i, \pi^{(i+1)}, r_{i+1}) \notin \mathcal{D}] \leq \epsilon_{rbr}$$

Definition 4 (Round-by-Round Knowledge Soundness). *An IOP for the relation $\mathcal{R}$ has round-by-round knowledge soundness error κ_{rbr} if there exists a PPT extractor $\mathcal{E}$ and a doomed set $\mathcal{D}$ of partial and complete transcripts such that:*

1. $(\mathbf{x}, \emptyset) \in \mathcal{D}$ for all possible input.
2. For any input $\mathbf{x}$ and complete proof transcript $\mathbf{t}$, if $(\mathbf{x}, \mathbf{t}) \in \mathcal{D}$ then the verifier rejects.
3. For every $i \in [k]$, and any partial transcript $\mathbf{t}_i$, such that $(\mathbf{x}, \mathbf{t}_i) \in \mathcal{D}$, then for any possible reply $\pi^{(i+1)}$ it holds that if:

$$\Pr_{r_{i+1} \xleftarrow{\$} \mathbb{F}} [(\mathbf{t}_i, \pi^{(i+1)}, r_{i+1}) \notin \mathcal{D}] > \kappa_{rbr}$$

Then, the extractor outputs a witness $\mathbf{w}$ such that $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$.

In this work, we will rely on the BCS transformation that compiles any IOP to a non-interactive protocol so that the following hold:

Theorem of [40, 59, 5] For any IOP with $l(\mathbf{x}), q(\mathbf{x}), \epsilon_{rbr}, \kappa_{rbr}$, there exists a polynomial-time transformation BCS such that for every relation $\mathcal{R}$, random oracle $H : \{0, 1\}^* \rightarrow \{0, 1\}^k$ and query bound $Q \in \mathbb{F}$ it produces a non-interactive proof system for $\mathcal{R}$ with:

- Adaptive soundness error $\epsilon_{fs} \leq Q\epsilon_{rbr} + 3(Q^2 + 1)/2^k$ and adaptive knowledge soundness error of $\kappa_{fs} \leq Q\kappa_{rbr} + 3(Q^2 + 1)/2^k$.
- Adaptive soundness error $\epsilon_{qfs} = \Theta(Q\epsilon_{fs})$ and adaptive knowledge soundness error of $\kappa_{qfs} = \Theta(Q\kappa_{fs})$ for $Q\text{-}\mathcal{O}(q(\mathbf{x}) \log l(\mathbf{x}))$ -query adversaries.

3 Collaborative Codes

In this section, we propose the notion of collaborative codes. At a high level, collaborative codes enable multiple parties to jointly compute a codeword $\mathbf{c} = (\mathbf{c}_1, \dots, \mathbf{c}_N) \in \mathbb{F}^{n_1 \times \dots \times n_N}$ over a “global” message $\mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_N) \in \mathbb{F}^{k_1 \times \dots \times k_N}$ where $\mathbf{x}_i$ and $\mathbf{c}_i$ is the private input and codeword segment of the i -th party. Crucially, collaborative codes must satisfy (t, l) -zero knowledge, namely, up to t corrupted parties can learn nothing about the individual inputs of the honest parties even after querying up to l positions of $\mathbf{c}$. In more detail:

Definition 5 (Collaborative Code). *Let $\mathcal{C} : \mathbb{F}^k \rightarrow \mathbb{F}^n$ be a code ensemble and $\Pi^{\text{Enc}} : (\mathbb{F}^{k_1}, \dots, \mathbb{F}^{k_N}) \rightarrow (\mathbb{F}^{n_1}, \dots, \mathbb{F}^{n_N})$ be a protocol that maps any message $\mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_N)$ into $\mathbf{c} = (\mathbf{c}_1, \dots, \mathbf{c}_N) = \text{Image}(\mathcal{C})$, where $(\mathbf{x}_i, \mathbf{c}_i)$ is the individual input and output of each party $\mathcal{P}_i$. We say that the collaborative code has (t, l) -zero knowledge over Π^{Enc} if for any adversary $\mathcal{A}$ controlling t parties $(\mathcal{P}_{j_1}, \dots, \mathcal{P}_{j_t})$, there exists a PPT simulator $\mathcal{S}$ s.t. the following views are indistinguishable:*

1. $\text{View}_{\mathcal{J}}^{\Pi^{\text{Enc}}}(\mathbf{x})$: Corresponds to $\{(V_{j_1}, \dots, V_{j_t}), (\mathbf{c}[i_1], \dots, \mathbf{c}[i_l])\}$, where $V_{j_k} = \text{View}_{j_k}^{\Pi^{\text{Enc}}}(\mathbf{x})$ the view of $\mathcal{P}_{j_k}$, $\mathcal{I} = \{i_1, \dots, i_l\}$ the set of index queries selected by $\mathcal{A}$ and $\mathbf{c}[i_1], \dots, \mathbf{c}[i_l]$ the replies.
2. $\mathcal{S}(\mathbf{x}_{j_1}, \dots, \mathbf{x}_{j_t}, \mathcal{I})$: The view of $\mathcal{A}$ obtained by $\mathcal{S}$.

A trivial way to construct a collaborative code $\mathcal{C}$ is by using *l-zero knowledge codes* (see Section 2.1). If we let each party $\mathcal{P}_i$ use an *l-zero knowledge code* $\mathcal{C}_i : \mathbb{F}^{k_i} \rightarrow \mathbb{F}^{n_i}$ then, each $\mathcal{P}_i$ can simply encode its own private input $\mathbf{x}_i$ locally. This results in code $\mathcal{C}$ that is an interleaved code of $\mathcal{C}_1 \times \dots \times \mathcal{C}_N$. This approach requires no communication among the parties; however, as an interleaved code, $\mathcal{C}$ has relative minimum distance *inversely proportional* to the number of parties N . This makes the above code not well-suited for constructing a scalable coSNARK as it would lead to proof sizes and verification times $\Omega(Nl)$.

Building collaborative codes with “large” relative minimum distance. To avoid this issue, one can build a (t, l) -zero-knowledge collaborative code $\mathcal{C}$ by composing a zero-knowledge code $\mathcal{C}'$ with a multi-party computation (MPC) protocol. In particular, all parties first secret share their inputs (with threshold t) and then interact using an MPC protocol, such as [60, 55, 61], for evaluating a circuit that emulates the encoding algorithm of an *l-zero knowledge code* $\mathcal{C}'$. Importantly, we want the protocol to terminate before the step of reconstructing the final circuit output $\mathbf{c}' \leftarrow \text{Enc}_{\mathcal{C}'}(\mathbf{x})$. Instead, we assume that parties stop when each of them receives its corresponding secret shares $\llbracket \mathbf{c}' \rrbracket_i$ (e.g., in the protocol of [60] as presented in [61], parties terminate right before step 3 of the protocol **Protocol 4.1**—each party i ends up holding a share of each output wire).

Therefore, the collaborative code $\mathcal{C}$ maps $\mathbf{x}$ to the secret shares of $\mathbf{c}'$, namely, $\mathbf{c} = (\mathbf{c}_1, \dots, \mathbf{c}_N) = (\llbracket \mathbf{c}' \rrbracket_1, \dots, \llbracket \mathbf{c}' \rrbracket_N)$. Interestingly, the only prior work [1] on code-based coSNARKs adopts this approach to collaboratively compute RS codewords. Unfortunately, it uses the “standard” Shamir’s secret sharing, hence each party must run in time $\mathcal{O}(|\mathbf{x}| \log |\mathbf{x}|)$. The same problem remains even if we used the best known approach for collaboratively computing RS codes via the MPC protocol of [11]. In fact, to build scalable collaborative codes that will then be used to build a scalable coSNARK, we need to address the following challenges:

Challenge 1 (Scalability): Most MPC protocols have a per-party computational complexity that scales with the size of the circuit being computed. While there do exist such scalable MPC protocols (e.g., [62, 33]), they are only truly scalable if the underlying circuit itself has low depth and consequently minimal interaction between the parties and is strictly SIMD (e.g., it consists of multiple identical sub-circuits without shared wires). (One exception is the recent work of Garg et al. [63] that works for circuits with arbitrary structure; since it relies on Chinese-remainder secret sharing, it is not very beneficial for building collaborative codes as the latter can be defined over relatively small fields, and, moreover, it is unclear how to make it efficiently work over field extensions which are commonly used in hash-based SNARKs.) Unfortunately, the encoding algorithms of error-correcting codes commonly used to construct zkSNARKs, such as Reed-Solomon, Spielman, Expand-Accumulate codes [25, 31, 32], do not exhibit this structure. In particular, although their underlying arithmetic circuits possibly consist of multiple identical sub-circuits, these sub-circuits share wires between them. This necessitates the reshuffling of the packed shares, which not only adds computation and communication overhead but also increases the number of interactions between parties. For instance, as observed in [11], it is unclear how to construct an MPC protocol for computing Reed-Solomon codes with a sub-linear time per-party without incurring logarithmic rounds of interaction and a quasi-linear total communication cost.

Challenge 2 (SNARK-compatibility): The second challenge is that we do not have any guarantee about how “compatible” the produced code $\mathcal{C}$ is with building a SNARK. In principle, one can build a coSNARK relying on the underlying code $\mathcal{C}'$ which can be freely chosen to be compatible with a SNARK that achieves poly-logarithmic proof size and verification time in the non-collaborative setting. However, adapting existing schemes that work over $\mathcal{C}'$ to the collaborative setting (e.g., as the collaborative Fractal [1] did) would significantly increase the proof size and verification time by a factor of at least $t + 1$. This is because querying the i -th position of $\mathbf{c}'$ requires the verifier to obtain at least $t + 1$ elements of $\mathbf{c}$ just to reconstruct $\mathbf{c}'[i]$. To avoid this blowup, we would like our

collaborative code to be *SNARK-compatible*, in that the underlying proximity tests should directly work over $\mathbf{c}$ and not $\mathbf{c}'$.

3.1 Our Scalable Collaborative Code

We build our collaborative code that addresses the above challenges by utilizing tensor codes. First, note that the circuit emulating the encoding algorithm of a tensor code exhibits a SIMD structure [64]. Specifically, it consists of three “levels”. The first level takes as input a matrix $\mathbf{X}$ and computes $\mathbf{X}'$ by applying the encoding circuit for $\mathcal{C}_1$ once for each column of $\mathbf{X}$. The second level transposes $\mathbf{X}'$ and the third level computes the final codeword $\mathbf{C}$ by applying the encoding circuit for $\mathcal{C}_2$ once for each column of $\mathbf{X}'^T$.

Second, we observe that if we choose $\mathcal{C}_1, \mathcal{C}_2$ to be two Reed-Solomon (RS) codes (randomized as in [34]), then running the encoding algorithm of that code over the packed secret shared values of $\mathbf{X}$ produced using Shamir’s scheme, is *entirely redundant*. That is because there is an inherent connection between Shamir’s packed secret shares and RS codes. In particular, they are equivalent since each share is itself an element of a (randomized) RS codeword! This connection leads to the following crucial observation: When parties compute the secret shares of their inputs, they are, in fact, computing “for free” the columns of the matrix $\mathbf{X}'$. In addition, after the input distribution phase, each party $\mathcal{P}_i$ already has the i -th row of $\mathbf{X}'$ (alternatively i -th column of $\mathbf{X}'^T$) which is used to compute the i -th row of $\mathbf{C}$.

Note that the above solution does not require any MPC machinery. In fact, and as we show next, this family of tensor-codes is inherently collaborative in the sense that parties can run a distributed version of its encoding algorithm “as is”, without violating (t, l) -zero knowledge. Also, it is not hard to see that because of this, our collaborative code is also by default “SNARK-friendly”, because at the end of the encoding protocol each party ends up with a segment of the codeword $\mathbf{C}$ and not its packed secret shares.

Our Collaborative-Friendly Tensor-Code. Based on the above discussion, we first give a formal description of the tensor-code we will use (which we denote with $\text{RS}^{\otimes 2}$) and later show how to extend its encoding algorithm in the collaborative setting. In more detail, let $\mathcal{L}'_1 \times \mathcal{L}'_2 \subset \mathcal{L}_1 \times \mathcal{L}_2$ be evaluation domains of size $N \cdot M$ and $(N + k') \cdot (M + m')$ respectively. In addition, let $\mathbf{X} \in \mathbb{F}^{k \times m}$ be the original message, $\tilde{\mathbf{X}} \in \mathbb{F}^{k' \times m'}$ the message padded with random rows and columns and $p : \mathbb{F}^2 \rightarrow \mathbb{F}$ the bivariate polynomial of individual degrees of at most (k', m') that interpolates a matrix $\tilde{\mathbf{X}}$ on $(\mathcal{L}_1 \times \mathcal{L}_2) \setminus (\mathcal{L}'_1 \times \mathcal{L}'_2)$. $\text{RS}^{\otimes 2}$ is defined as:

$$\text{RS}^{\otimes 2}[\mathbb{F}, (\mathcal{L}'_1 \times \mathcal{L}'_2), (k, k', N), (m, m', M)] = \{f : (\mathcal{L}'_1 \times \mathcal{L}'_2) \rightarrow \mathbb{F} : f(x, y) = p(x, y), \forall (x, y) \in \mathcal{L}'_1 \times \mathcal{L}'_2\}$$

Note that the rate of the code is $\rho = \frac{m'k'}{MN}$ and its relative minimum distance is $\Delta = (1 - \frac{m'}{M})(1 - \frac{k'}{N})$ (see remark 1).

Next, we provide an algorithm for computing a codeword $\mathbf{C} \in \text{RS}^{\otimes 2}$ from a message $\mathbf{X}^{k \times m}$. Compared to the standard one of first interpolating p and then evaluating it on $\mathcal{L}'_1 \times \mathcal{L}'_2$ our algorithm is “collaborative-friendly”, that is, it can be extended in the collaborative setting without any significant modifications. At a high level, instead of interpolating p “at once” our algorithm first interpolates the columns of $\tilde{\mathbf{X}}$ and computes $\mathbf{X}'$ consisting of the evaluations of these polynomials on $\mathcal{L}'_1$. Then it interpolates the rows of $\mathbf{X}'$ and uses these polynomials to compute $\mathbf{C}$. In more detail, it works as follows: (1) expands $\mathbf{X}$ to $\tilde{\mathbf{X}} \in \mathbb{F}^{k' \times m'}$ by padding it with $(m' - m)$ columns and $(k' - k)$ rows selected uniformly at random. (2) Computes the k' -degree polynomial $c^{(j)}(X) = p(X, z_2^j)$, $z_2^j \in \mathcal{L}_2 \setminus \mathcal{L}'_2$ that interpolates each column j of $\tilde{\mathbf{X}}$ and evaluates $\mathbf{X}' \in \mathbb{F}^{N \times m'}$ s.t. $\mathbf{X}'[i, j] = c^{(j)}(\omega_1^i)$ where $\omega_1^i \in \mathcal{L}_1$. (3) Finally, for each row j of $\mathbf{X}'$, it interpolates the polynomial $r^{(j)}(Y) = p(\omega_1^j, Y)$ evaluates $r^{(j)}$ at $\mathcal{L}_2$ thus computing the j -th row of $\mathbf{C}$.

Our Collaborative Code Construction. Now we have everything we need to construct our scalable collaborative code. **Construction 2** gives a detailed description of the encoding protocol of the $RS^{\otimes 2}$ code in the collaborative setting. Initially, each party $\mathcal{P}_i$, holds its private input $\mathbf{X}_i \in \mathbb{F}^{(k \times \frac{m}{N})}$ and interacts with all other N parties to obtain the i -th row of the codeword matrix $\mathbf{C}$ over the joint input $\mathbf{X} = (\mathbf{X}_1, \dots, \mathbf{X}_N)$. Initially, all parties follow an input independent pre-processing phase to jointly compute the submatrix $\mathbf{R}'$ consisting of the last $m' - m$ columns of $\mathbf{X}'$. At the end of this phase each $\mathcal{P}_i$ ends-up with the i -th row of $\mathbf{R}'$. This is done by having each party call the $\mathcal{F}_{rand}$ which can be instantiated using the protocol of [62] that generates packed secret shares of random vectors.

Then, to evaluate $\mathbf{C} = \text{Enc}(\mathbf{X})$, parties invoke the Π^{Enc} protocol. First, each party $\mathcal{P}_i$ pads $\mathbf{X}_i$ with $k' - k$ random rows, thus forming $\tilde{\mathbf{X}}_i$ and then it computes the polynomial $c^{(i,j)}(X)$ that interpolates each j -th column of $\tilde{\mathbf{X}}_i$, and $\mathbf{X}'_i$ by setting $\mathbf{X}'_i[j, h] = c^{(i,j)}(\omega_1^h)$ (step 1). Next, it sends the h -th row of $\mathbf{X}'_i$ to the $\mathcal{P}_h$ party and waits until it receives the i -th row of the matrixes of all other parties (step 2). At the end of this step, $\mathcal{P}_i$ holds the row $[\mathbf{X}'_1[i, :], \dots, \mathbf{X}'_N[i, :], \mathbf{R}'[i, :]]$ which equals to the i -th row of $\mathbf{X}'$ as defined in the step 2 of the encoding algorithm! Hence $\mathcal{P}_i$ can directly obtain $r^{(i)}(Y)$ and consequently $\mathbf{C}[i, :]$ (step 3).

Construction 2: Protocol for evaluating our collaborative code. Let N be the number of parties, and $\mathbf{X} \in \mathbb{F}^{k \times m}$ the “global” input, defined to be the concatenation of the private inputs of each party, that is, $\mathbf{X} = (\mathbf{X}_1, \dots, \mathbf{X}_N)$, where $\mathbf{X}_i \in \mathbb{F}^{k \times \frac{m}{N}}$. Without loss of generality, all inputs are equal-sized.

Pre-Processing. All parties, use the $\mathcal{F}_{rand}$ functionality to generate $l = m' - m$ Shamir’s packed secret shares of columns of the random matrix $\mathbf{R} \in \mathbb{F}^{k' \times (m' - m)}$. Finally, each party $\mathcal{P}_i$, obtains $\mathbf{R}'[i, :] \in \mathbb{F}^{m' - m}$.

Protocol Π^{Enc} : Protocol for computing $\mathbf{C} = \text{Enc}(\mathbf{X}) \in \mathbb{F}^{N \times M}$ where Enc is the encoding algorithm of the code described above. At the end of the protocol, $\mathcal{P}_i$ ends up with the i -th row of $\mathbf{C}$.

1. $\mathcal{P}_i$: Expand the matrix $\mathbf{X}_i$ to $\tilde{\mathbf{X}}_i \in \mathbb{F}^{k' \times \frac{m}{N}}$ by padding it with $k' - k$ random rows sampled uniformly at random. For each column $j \in [\frac{m}{N}]$, interpolate the k' -degree polynomial $c^{(i,j)} : \mathbb{F} \rightarrow \mathbb{F}$ such that $c^{(i,j)}(z_1^h) = \tilde{\mathbf{X}}_i[j][h]$, for all $h \in [k']$. Finally, evaluate the matrix $\mathbf{X}'_i \in \mathbb{F}^{N \times \frac{m}{N}}$ such that $\mathbf{X}'_i[h][j] = c^{(i,j)}(\omega_1^h)$, for all $j \in [\frac{m}{N}]$ and $h \in [N]$.
2. $\mathcal{P}_i$: Send the j -th row of $\mathbf{X}'_i$ to $\mathcal{P}_j$, for all $j \in [N] \setminus i$. Wait for the i -th row of $\mathbf{X}'_j$ for all $j \in [N]$.
3. $\mathcal{P}_i$: Form $\mathbf{X}'[i, :]$ by concatenating all rows received and also the row $\mathbf{R}[i, :]$ computed in the pre-processing phase. Interpolate the m' -degree polynomial $r^{(i)} : \mathbb{F} \rightarrow \mathbb{F}$ such that $r^{(i)}(z_2^j) = \mathbf{X}'[i, j]$ and compute $\mathbf{C}[i, :] \in \mathbb{F}^M$, such that $\mathbf{C}[i, j] = r^{(i)}(\omega_2^j)$, for all $j \in [M]$.

Finally, we state the following theorem which we prove in Appendix A.

Theorem 1. *Construction 2 is a (t, l) -zero knowledge collaborative code for $RS^{\otimes 2}$, with $t \leq k' - k + 1$ and $l = m' - m$. Furthermore, assuming that $k' = \Theta(N)$, the above protocol has per-party time complexity of $\mathcal{O}(\frac{k'm'}{N} \log k'm')$, one (online) round with a total communication complexity of $\approx Nm' \log |\mathbb{F}|$ bits. In addition, our protocol is complete, that is, it produces the same result as if we run the encoding algorithm locally using the same randomness.*

Proof Intuition. Observe that the only communication that happens between the parties is at Step 2 of **Construction 2**. Thus, we have to prove that (i) Step 2 reveals nothing about the individual inputs of each party and that (ii) the same holds even if the adversary obtains l additional elements of $\mathbf{C}$. For (i), recall that steps 1,2, can be considered as pack secret sharing the columns $\mathbf{X}_i$ using Shamir’s packed secret sharing scheme hence privacy is guaranteed by that scheme. As for (ii), zero-knowledge is guaranteed by the randomness used to mask the rows of $\mathbf{X}$, namely the matrix $\mathbf{R}$.

On the Foldability of $\text{RS}^{\otimes}$. We would like our tensor-code to be foldable, namely, to have a folding function that given a codeword $\mathbf{C} \in \text{RS}^{\otimes 2}$ and a random element a “hashes” $\mathbf{C}$ into a codeword in $\text{RS}^{\otimes 2}$ but of smaller size. Surprisingly, such a function exists [65] but, unfortunately, it does not fit our setting as it needs to fold both rows and columns of $\mathbf{C}$. The last one, not only would require additional interaction between parties but also can violate (t, l) -zero knowledge since it enables the adversary to obtain secret shares of honest parties. Instead, in this work we propose a folding algorithm that only folds the rows of $\mathbf{C}$ (using the folding algorithm of RS codes) thus overcoming all aforementioned issues. Naturally, if $\mathbf{C}$ is far from $\text{RS}^{\otimes 2}$, then, its folding would also be far. To prove this, we follow the same argument as in [35]. For completeness, we provide a detailed proof in B.

4 Collaborative IOPs

A *collaborative Interactive Oracle Proof* (coIOP) for a relation $\mathcal{R}$ is an interactive protocol between multiple provers $\mathcal{P}_1, \dots, \mathcal{P}_N$ and a single verifier $\mathcal{V}$, that works in the following way. Initially, each prover $\mathcal{P}_i$ has the instance $\mathbf{x}$, its private witness $\mathbf{w}_i$ and a state $\mathbf{st}_i$ denoting its view, that is, the list of messages it received from all other parties (initially set to $\emptyset$) and collaborates following an MPC protocol to compute the string $\pi^{(0)}$, the oracle of which is sent to the verifier. Note that at the end of that protocol each prover might only hold a sub-string of $\pi^{(0)}$. Subsequently, in every round $j \in [k]$, the verifier sends a challenge $r_j \in \mathbb{F}$ to the provers that collaborate to respond with the oracle of the string $\pi^{(i)}$. Finally, when all rounds are completed, the verifier makes some queries to the oracles $\pi^{(0)}, \dots, \pi^{(k)}$ and decides whether to accept or reject. More formally, by denoting the interaction process between the provers and the verifier:

$$\begin{aligned} \pi^{(0)} &\leftarrow \langle \mathcal{P}_1(\mathbf{w}_1, \emptyset), \dots, \mathcal{P}_N(\mathbf{w}_N, \emptyset) \rangle(\mathbf{x}) \\ \pi^{(1)} &\leftarrow \langle \mathcal{P}_1(\mathbf{w}_1, \mathbf{st}_1), \dots, \mathcal{P}_N(\mathbf{w}_N, \mathbf{st}_N) \rangle(\mathbf{x}, r_1) \\ &\quad \dots \\ \pi^{(k)} &\leftarrow \langle \mathcal{P}_1(\mathbf{w}_1, \mathbf{st}_1), \dots, \mathcal{P}_N(\mathbf{w}_N, \mathbf{st}_N) \rangle(\mathbf{x}, r_1, \dots, r_k) \end{aligned}$$

with $\pi^{(0)}, \dots, \pi^{(k)} \leftarrow \langle \mathcal{P}_1(\mathbf{w}_1), \dots, \mathcal{P}_N(\mathbf{w}_N) \rangle(\mathbf{x})$, we define a coIOP as follows:

Definition 6 (Collaborative Interactive Oracle Proofs). *Let $\text{coIOP} = (\mathcal{P}_1, \dots, \mathcal{P}_N, \mathcal{V})$ be a collaborative IOP for the relation $\mathcal{R}$. We say that the coIOP is complete if for any $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$ it holds that:*

$$\Pr \left(\begin{array}{l} \mathcal{V}^{(\pi^{(0)}, \dots, \pi^{(k)})}(r_1, \dots, r_k) = 1, \\ \pi^{(0)}, \dots, \pi^{(k)} \leftarrow \langle \mathcal{P}_1(\mathbf{w}_1), \dots, \mathcal{P}_N(\mathbf{w}_N) \rangle(\mathbf{x}) \end{array} \right) = 1$$

A coIOP is sound with soundness error of ϵ_{coIOP} if for any $\mathbf{x} \notin \mathcal{R}$ the following holds:

$$\Pr \left(\begin{array}{l} \mathcal{V}^{(\pi^{(0)}, \dots, \pi^{(k)})}(r_1, \dots, r_k) = 1, \\ \pi^{(0)}, \dots, \pi^{(k)} \leftarrow \langle \mathcal{P}_1(\mathbf{w}_1), \dots, \mathcal{P}_N(\mathbf{w}_N) \rangle(\mathbf{x}) \end{array} \right) \leq \epsilon_{\text{coIOP}}$$

An *coIOP* is knowledge sound with knowledge soundness error κ_{coIOP} and extraction time $t_{\mathcal{E}}$, if there exists a PPT extractor $\mathcal{E}$ such that for any $\{\mathcal{P}_1, \dots, \mathcal{P}_N\}$ and $\mathbf{x}$ we get:

$$\Pr \left(\begin{array}{l} \mathcal{V}^{(\pi^{(0)}, \dots, \pi^{(k)})}(r_1, \dots, r_k) = 1 \wedge (\mathbf{x}, \mathbf{w}) \notin \mathcal{R}, \\ \pi^{(0)}, \dots, \pi^{(k)} \leftarrow \left\langle \mathcal{P}_1(\mathbf{w}_1), \dots, \mathcal{P}_N(\mathbf{w}_N) \right\rangle(\mathbf{x}) \\ \mathbf{w} \leftarrow \mathcal{E}^{\{\mathcal{P}_1, \dots, \mathcal{P}_N\}}(\mathbf{x}, \pi^{(0)}, \dots, \pi^{(k)}, r_1, \dots, r_k) \end{array} \right) \leq \kappa_{\text{coIOP}}$$

Finally, a *coIOP* is honest verifier t -zero knowledge if for any PPT adversary that controls up to t provers $\{\mathcal{P}_{j_1}, \dots, \mathcal{P}_{j_t}\}$ and the honest verifier there exists a PPT simulator $\mathcal{S}$ such that the following distributions are statistically indistinguishable (up to an error z_{coIOP}):

- The view $(\mathbf{st}_{j_1}, \dots, \mathbf{st}_{j_t}, \pi) \leftarrow \text{View}_{\mathcal{A}}^{\mathcal{R}}(\mathbf{x}, \mathbf{w})$ produced when the adversary interacts with the honest provers.
- The view $(\mathbf{st}_{j_1}, \dots, \mathbf{st}_{j_t}, \pi) \leftarrow \text{View}_{\mathcal{S}}(\mathbf{x}, \mathbf{w}_{j_1}, \dots, \mathbf{w}_{j_t}, \{j_1, \dots, j_t\})$ produced when the adversary interacts with the simulator.

We note that π indicates the proof transcript, that is, the oracles and the replies from the queries made by the verifier. If the above holds against malicious provers then we say that the *coIOP* is **maliciously secure** t -zero knowledge.

We note that we can naturally extend the round-by-round (knowledge) soundness definitions for *coIOPs*. Additionally, one can naturally adapt the notions of completeness, (round-by-round) soundness and knowledge from *IOPs* to *coIOPs*. In addition, we say that a *coIOP* satisfies honest-verifier t -zero knowledge if up to t corrupted provers (possibly colluding with the verifier) can learn nothing about the witness of honest provers. Furthermore, if the above holds against malicious provers then we say that the *coIOP* is honest-verifier maliciously secure t -zero knowledge.

Performance Characteristics. The performance of a *coIOP* is measured by the following: (1) $k_{\mathcal{V}}(|\mathbf{x}|, N) = k$, the number of communication rounds between the provers and the verifier, (2) $k_{\mathcal{P}}(|\mathbf{x}|, N)$ the number of communication rounds between the provers, (3) $l_i(|\mathbf{x}|)$ the total size of the substrings of $(\pi^{(0)}, \dots, \pi^{(k)})$ held by $\mathcal{P}_i$ (4) $q(|\mathbf{x}|)$ the number of verifier queries over the oracles, (5) $vt(|\mathbf{x}|, N)$ the verification time (6) $pt_i(|\mathbf{x}|, N)$ the time complexity of $\mathcal{P}_i$, (7) $pc(|\mathbf{x}|, N)$ the total communication complexity between the provers and finally (8) the security threshold t .

Making a *coIOP* Non-Interactive (wrt the Verifier). We note that it is possible to remove any interaction between the provers and the verifier for any *coIOP* by adapting the BCS [40] transformation in the collaborative setting. In particular, let $\mathcal{H} : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ be a random oracle, and $MT^{\mathcal{H}} = (\text{Commit}, \text{Open})$ a (hiding) Merkle tree commitment scheme. For each round $j \in [k]$ the provers work as follows (with **blue** we denote the additional steps required to safeguard against malicious provers):

1. $\mathcal{P}_i$: Compute $rt_i^{(j)} = MT^{\mathcal{H}}.Commit(\pi_i^{(j)})$, where $\pi_i^{(j)}$ the segment of $\pi^{(j)}$ held by $\mathcal{P}_i$. Sample a salt $\tau_i^{(j)} \in \{0, 1\}^{\frac{\lambda}{N-t}}$ and broadcast $(rt_i^{(j)}, \tau_i^{(j)})$ to provers.
2. $\mathcal{P}_i$ Receive all “partial” roots and salts, and set $rt^{(j)} \leftarrow MT^{\mathcal{H}}.Commit(rt_1^{(j)}, \dots, rt_N^{(j)})$ and $\tau^{(j)} = \tau_1^{(j)} | \dots | \tau_N^{(j)} \in \{0, 1\}^{\frac{\lambda N}{N-t}}$. Derive the next challenge $r_{j+1} = \mathcal{H}(r_j | rt^{(j)} | \tau^{(j)})$.
3. $\mathcal{P}_i$: **Broadcast rt_{j+1} to all other provers and wait until receiving the challenges of every other prover. If there exists $r'_{j+1} \neq r_{j+1}$ abort.**

4. $\mathcal{P}_i$: If $j = k$, compute the query indexes of the verifier $\mathcal{I}$. Compute $\mathcal{I}^{(i)}$ containing only the indexes that belong to $\{\pi_i^{(j)}\}_{j \in [k]}$ and send to $\mathcal{P}_1$ the list $\mathbf{pr}_i$ containing the values on these indexes and their opening proofs created by calling $MT^{\mathcal{H}}.Open$.
5. $\mathcal{P}_1$: Collects the messages from all provers, concatenates them, and sends the final proof $\pi = (\{rt_i\}_{i \in [k]}, \{\mathbf{pr}_i\}_{i \in [N]})$ to the verifier.

The verifier validates the proof π in the same way as done in the verification algorithm of the BCS transformation. Observe that step 3 is necessary when dealing with malicious since otherwise the protocol is prone to the following attack: In every round the malicious provers send inconsistent messages to the honest provers. In turn, this makes each honest prover produce replies for different query indexes which gives the opportunity to the adversary to learn more values from the oracle sub-strings it does not hold.

Performance-wise, $k_{\mathcal{V}}^{FS}$ becomes 1 while $k_{\mathcal{P}}^{FS}$ increases to $k_{\mathcal{V}}$ (or $2k_{\mathcal{V}}$ in malicious setting). vt and pt_i change as in the standard BCS transformation, the security threshold and number of queries remain the same, while the total communication cost increases by at most $\mathcal{O}(N^2 k_{\mathcal{V}})$. Last but not least, our transformation guarantees the following: (1) If the underlying coIOP has negligible (knowledge) round-by-round soundness error then the non-interactive protocol has negligible adaptive (knowledge) soundness even against quantum adversaries. Furthermore, if the coIOP is honest-verifier (maliciously secure) t -zero knowledge then the non-interactive protocol would be (maliciously secure) t -zero knowledge.

Finally, we can claim the following (which we prove in Appendix C):

Theorem 2. *For any coIOP with $l(\mathbf{x}), q(\mathbf{x})$, and round-by-round (knowledge) soundness errors $\epsilon_{rbr}, \kappa_{rbr}$ and for any $\mathcal{R}$, random oracle $H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ and query bound $Q \in \mathbb{F}$ our transformation produces a non-interactive (with respect to the verifier) proof system for $\mathcal{R}$ with:*

- Adaptive soundness error $\epsilon_{fs} \leq Q\epsilon_{rbr} + 3(Q^2 + 1)/2^\lambda$ and adaptive knowledge soundness error of $\kappa_{fs} \leq Q\kappa_{rbr} + 3(Q^2 + 1)/2^\lambda$.
- Adaptive soundness error $\epsilon_{qfs} = \Theta(Q\epsilon_{fs})$ and adaptive knowledge soundness error of $\kappa_{qfs} = \Theta(Q\kappa_{fs})$ for $Q\text{-}\mathcal{O}(q(\mathbf{x}) \log l(\mathbf{x}))$ -query quantum adversaries.
- If the IOP is honest-verifier and prover statistical t -zero knowledge, then the non-interactive protocol is statistical t -zero knowledge with error $z_{fs}^H \leq z_{coIOP} + z_{MT}(\lambda, l(|\mathbf{x}|), q(|\mathbf{x}|), Q) + \frac{Q}{2^{N\lambda/(N-t)}}$ where $z_{MT}(\lambda, l(|\mathbf{x}|), q(|\mathbf{x}|), Q)$ the blinding error of the Merkle Tree commitment scheme.
- If the IOP is honest-verifier and **maliciously secure** t -zero knowledge, then the non-interactive protocol is statistical t -zero knowledge with error $z_{fs}^M \leq z_{coIOP} + z_{MT}(\lambda, l(|\mathbf{x}|), q(|\mathbf{x}|), Q) + \frac{Q}{2^\lambda}$.
- The protocol obtains the following performance characteristics: (1) $k_{\mathcal{V}}^{fs}(|\mathbf{x}|, N) = 1$, (2) $k_{\mathcal{P}}^{fs}(|\mathbf{x}|, N) = k_{\mathcal{V}}(|\mathbf{x}|, N) + k_{\mathcal{P}}(|\mathbf{x}|, N)$, (3) $l^{fs}(|\mathbf{x}|) = l(|\mathbf{x}|)$, (4) $q^{fs}(|\mathbf{x}|) = q(|\mathbf{x}|)$, (5) $vt^{fs}(|\mathbf{x}|, N) = vt(|\mathbf{x}|, N) + \mathcal{O}(q(|\mathbf{x}|) \log l(|\mathbf{x}|))$, (6) $pt_i^{fs}(|\mathbf{x}|, N) = pt_i(|\mathbf{x}|, N) + \mathcal{O}(l_i(|\mathbf{x}|, N)) + \mathcal{O}(k_{\mathcal{V}}(|\mathbf{x}|, N) \cdot N)$, (7) $pc(|\mathbf{x}|, N) = \mathcal{O}(k_{\mathcal{V}}(|\mathbf{x}|, N) \cdot N^2) + pc(|\mathbf{x}|, N)$ and (8) $t^{fs} = t$.

5 Our Collaborative PCS

In this section, we show how to use our collaborative code to instantiate a collaborative hash-based PCS via a coIOPP. First, we present a coIOPP for a codeword in $RS^{\otimes 2}$ and then, show how to use it in order to build our coPCS.

5.1 A coIOPP for our Collaborative Code

Here, we present our coIOPP protocol for showing that a string $\mathbf{C}$ is close to a codeword in our code $\text{RS}^{\otimes 2}[\mathbb{F}, \mathcal{L}_1 \times \mathcal{L}_2^{(0)}, (k, k', N), (m, m', M)]$, where m' is the next power of two of $m + l$, and l the number of queries over the oracles. Our protocol can be considered as a generalization of FRI-like IOPPs [36, 35] in the collaborative setting. In particular, the provers collaborate to sequentially reduce the task of checking that a function $\mathbf{C}^{(i)}$ is close to a codeword in $\text{RS}_i^{\otimes 2}$ to checking that $\mathbf{C}^{(i+1)}$ is close to a codeword in $\text{RS}_{i+1}^{\otimes 2}$, where $\text{RS}_i^{\otimes 2} = \text{RS}^{\otimes 2}[\mathbb{F}, \mathcal{L}_1 \times \mathcal{L}_2^{(i-1)}, (k, k'N), (\frac{m}{2^{i-1}}, \frac{m'}{2^{i-1}}, \frac{M}{2^{i-1}})]$ —which is possible due to the folding function of $\text{RS}^{\otimes 2}$.

Next, to build a t -zero knowledge coIOPP we need to use a zero-knowledge variant of FRI since the queries made by the verifier over the original and the folded functions might leak information about message $\mathbf{X}$. Existing techniques [37, 38, 66] for making FRI-like protocols zero-knowledge proceed as follows. (1) Encode the message using a randomized code (which is already achieved by our collaborative code), and (2) mask the original string $\mathbf{C}$, with a random string $\mathbf{C}_r$ by computing their linear combination $\mathbf{C} + b\mathbf{C}_r$ over a random point $b \in \mathbb{F}$, and then run the FRI-like protocol. While this approach is fine in the standard setting, applying step (2) “as is”, presents a significant challenge in the collaborative setting: All provers must jointly generate $\Omega(|\mathbf{X}|)$ secret random elements. This may significantly increase the concrete costs of deploying this protocol, even if moved to offline pre-processing. In contrast, our scheme reduces the total size of needed randomness to only $\mathcal{O}(Nl \log |\mathbf{X}|)$, where N is the number of provers. At a high level, instead of performing the masking at the beginning of the protocol, we delay it until the final round, where the size of the reduced $\mathbf{C}^{(i)}$ is only proportional to N . However, this modification by itself does not preserve t -zero knowledge, as all the queries of the previous rounds are left exposed. To overcome this, we mask the codeword of each round with a randomized codeword from a message that contains only a small number of random elements—just enough to mask the query replies for that specific round.

Protocol Description. Our protocol consists of three phases. The *PreProcessing* phase where the provers generate all masking codewords, the *Commit* phase, where the provers compute the oracles of the reduced codewords and the *Query* phase, where the verifier does the necessary queries and checks to validate that $\mathbf{C} = \mathbf{C}^{(1)}$ is close to $\text{RS}_1^{\otimes 2}$.

PreProcessing Phase. At the beginning of this phase, all provers run the PreProcessing protocol of the Π^{Enc} , in order to establish the necessary (secret) randomness to encode the joint message $\mathbf{X}$ using a (t, l) -zero knowledge collaborative code. Furthermore, the provers invoke the $\mathcal{F}_{\text{rand}}^{\text{Enc}}(x, y)$ functionality to compute the masked codewords. At a very high level, this functionality generates a (x, y) -collaborative codeword $\mathbf{C}_r$ of a (secret) random message $\mathbf{R}$, and provides the i -th row $\mathbf{C}_r$ to $\mathcal{P}_i$. Using $\mathcal{F}_{\text{rand}}^{\text{Enc}}$ with inputs $(t, 3l)$, the provers jointly compute the codewords $\mathbf{C}_r^{(1)}, \dots, \mathbf{C}_r^{(\log m' - c - 1)}$ such that $\mathbf{C}_r^{(i)} \in \text{RS}_i^{\otimes 2}$. We note that to compute all codewords except the last one $\mathbf{C}_r^{(\log m' - c - 1)}$, $\mathbf{R}$ does not have to be secret or random. In fact, the provers set it to $\mathbf{0}$, and they only have to generate $3l$ random columns used to make $\mathbf{C}_r^{(i)}$ $(t, 3l)$ -zero-knowledge. Observe that $c \in \mathbb{N}$ is a constant parameter selected by the prover. Note that c must be larger $c \geq 1 + \lceil \log 3l \rceil$ since a $(t, 3l)$ -zero knowledge code $\mathbf{C}_r^{(i)}$ with $i < c$, has $m < \lceil \log 3l \rceil$ and thus does not belong to any $\text{RS}_j^{\otimes 2}$, for $j \in [\log m']$.

Commit Phase. Initially, all provers invoke the Π^{Enc} protocol to jointly compute the codeword of $\mathbf{X}$. Finally, each prover $\mathcal{P}_i$ ends up with the i -th row of $\mathbf{C}^{(1)}$, denoted with $\mathbf{C}_i^{(1)}$ (step 1). Then, parties send the oracles of the original codeword $\mathbf{C}^{(1)}$ and the masks $\mathbf{C}_r^{(1)}, \dots, \mathbf{C}_r^{(\log m' - c - 1)}$ to the verifier (step 2). Next, for every of the $j = \{1, \dots, \log m' - c - 1\}$ rounds, the provers and the verifier reduce the “closeness” claim of $\mathbf{C}^{(j)}$ into a “closeness” claim of the smaller $\mathbf{C}^{(j+1)}$ as follows. First, the verifier samples two random points $a_j, b_j \in \mathbb{F}$ and sends them to the provers. Each prover $\mathcal{P}_i$, uses

b_j to compute the i -th row of the masked codeword $\mathbf{C}^{(j)} + b_j \mathbf{C}_r^{(j)}$ and a_j to fold it into $\mathbf{C}_i^{(j+1)}$. The last one can be computed without any interaction between the provers since the folding algorithm introduced in Section 3.1 corresponds to running the RS folding algorithm [36, 21] for each row of the codeword $\mathbf{C}^{(j)}$, which can be done locally by every prover $\mathcal{P}_i$ (step 3.b). Finally, $\mathcal{P}_1$ sends the oracle of $\mathbf{C}^{(j+1)}$ to the verifier using the same protocol as in step 2. Eventually, $\mathcal{P}_i$ ends up with $\mathbf{C}_i^{(\log m' - c)}$ and further reduces it to $\mathbf{C}_i^{(\log m')}$ following the above steps but without using any masking and sends it to the verifier (step 4).

Query Phase. Given the oracles $\mathbf{C}^{(1)}, \dots, \mathbf{C}^{(\log m' - 1)}$ and the codeword $\mathbf{C}^{(\log m')}$, the verifier applies the same consistency checks as in FRI-like protocols. With $\text{INTERPOLATE}(a, \mathbf{C}[x, y], \mathbf{C}[x, -y])$ we denote the function computing $\frac{\mathbf{C}[x, y] + \mathbf{C}[x, -y]}{2} + \frac{a(\mathbf{C}[x, y] - \mathbf{C}[x, -y])}{2y}$ where $-y = x + \frac{|\mathbf{C}[x, :]|}{2}$.

Construction 3: Our coIOPP. N provers collaborate to show that $\mathbf{C}^{(1)} \in \mathbb{F}^{N \times M}$ is close to $\text{RS}^{\otimes 2}$.

Pre-Processing. All parties run the pre-processing protocol of Π^{Enc} for an (t, l) -zero knowledge code where l is set to be the number of queries of our IOPP in one round. They also invoke the $\mathcal{F}_{\text{rand}}^{\text{Enc}}$ to jointly compute the $(t, 3l)$ -zero knowledge tensor-codes $\mathbf{C}_r^{(1)}, \dots, \mathbf{C}_r^{(\log m' - c - 1)}$ in $\text{RS}_1^{\otimes 2}, \dots, \text{RS}_{\log m' - c - 1}^{\otimes 2}$ respectively over secret random messages. The messages of $\log m' - c - 2$ are set to be $\mathbf{0}$. Each prover $\mathcal{P}_i$ only obtains the i -th row of these codes.

Protocol $\Pi^{\text{coIOPP.Commit}}$. Each prover $\mathcal{P}_i$ initially has its own private input $\mathbf{X}_i \in \mathbb{F}^{k \times \frac{m}{N}}$, and interacts with all other provers over the following protocol:

1. $\mathcal{P}_i$: Follow the Π^{Enc} protocol to obtain the i -th row of $\mathbf{C}^{(1)}$, denoted with $\mathbf{C}_i^{(1)}$.
2. $\mathcal{P}_1$: Send the oracles of $\mathbf{C}^{(1)}$ and $\mathbf{C}_r^{(1)}, \dots, \mathbf{C}_r^{(\log m' - c - 1)}$ to $\mathcal{V}$.
3. For $j = \{1, \dots, \log m' - c - 1\}$ work as follows:
 - (a) $\mathcal{V}$: Samples random points a_j, b_j and broadcasts it to all provers.
 - (b) $\mathcal{P}_i$: Set $\mathbf{C}_i^{(j+1)} = \text{Fold}_{\text{RS}}(a_j, (\mathbf{C}_i^{(j)} + b_j \mathbf{C}_{r,i}^{(j)}))$.
 - (c) $\mathcal{P}_1$: Send the oracle of $\mathbf{C}^{(j+1)}$ to $\mathcal{V}$.
4. $\mathcal{P}_i$: Repeat step 3 for the rest of the c rounds but without using any masking. Finally send $\mathbf{C}_i^{(\log m')} \in \mathbb{F}^{\rho^{-1}}$ to $\mathcal{V}$.

coIOPP.Query. Takes as input all oracles $\mathbf{C}^{(1)}, \dots, \mathbf{C}^{(\log m' - 1)}$ and the string $\mathbf{C}^{(\log m')} \in \mathbb{F}^{N \times \rho^{-1}}$ and accepts if the following checks are satisfied:

1. The verifier repeats the following l times:
 - (a) Samples $(x, y) \leftarrow [N] \times [\frac{M}{2}]$
 - (b) for $j = \{1, \dots, \log m'\}$ (excluding any blue in the last c rounds) check if:
 - i. $\mathbf{C}^{(j+1)}[(x, y)] = \text{INTERPOLATE}(a_j, \mathbf{C}^{(j)}[(x, y)] + b_j \mathbf{C}_r^{(j)}[(x, y)], \mathbf{C}^{(j)}[(x, -y)] + b_j \mathbf{C}_r^{(j)}[(x, -y)])$
 - ii. If $y > \frac{M}{2^{j+1}}$ set $y \leftarrow y - \frac{M}{2^{j+1}}$
2. The verifier checks if $\mathbf{C}^{(\log m')}$ is a valid codeword, namely, it corresponds to a univariate polynomial g' of individual degree at most k' .

Performance-wise, our protocol has $k_V = k_P = \mathcal{O}(\log(L/N))$, $l_i = \mathcal{O}(L/N)$, $vt = q = N \cdot \rho_{RS^{\otimes 2}} + \mathcal{O}(l \log(L/N))$, $pt_i = \mathcal{O}(L \log L/N)$ and that $pc = \mathcal{O}(L)$, where $L = |\mathbf{X}|$ (or $pc = \mathcal{O}(\log(L))$ if we assume that the parties already hold the packed secret shares of $\mathbf{X}$). Finally, we state the following which we prove in Appendix D.

Theorem 3. *Let $\delta = \min(\Delta/3, \max(\{\Delta(\mathbf{C}_r^{(i)}, RS_i^{\otimes 2})\}_i, \Delta(\mathbf{C}^{(1)}, RS_1^{\otimes 2}))$, where Δ the relative minimum distance of $RS^{\otimes 2}$. **Construction 3** is a coIOPP for showing that $\Delta(\mathbf{C}^{(1)}, RS_1^{\otimes 2}) \leq \delta$ with perfect completeness, soundness error of $\epsilon = (4(\log M - c)|\mathbf{C}^{(1)}|/|\mathbb{F}| + (1 - \delta)^l)$, round-by-round (knowledge) soundness error of $\max(2|\mathbf{C}^{(1)}|/|\mathbb{F}|, (1 - \rho)^l)$, and honest-verifier and prover perfect t -zero knowledge supporting up to $t \leq k' - k + 1$ corruptions.*

Proof Intuition. Regarding (round-by-round) soundness, our protocol is similar to FRI but in each round it also combines the original code with a randomized one (that has been non-adaptively selected). Hence, we follow similar proofs to that of [35, 67], but adapt it to encompass these modifications. Regarding t -zero knowledge, observe that each prover only holds a single row of all codewords $\mathbf{C}^{(1)}, \dots, \mathbf{C}^{(\log m' - 1)}$ and $\mathbf{C}_r^{(1)}, \dots, \mathbf{C}_r^{(\log m' - 1)}$ and l additional elements derived from the Query algorithm. If all these codes were (t, l) -zero knowledge then by Theorem 1 our construction would be t -zero knowledge. However, we only know that $\mathbf{C}^{(1)}$ is (t, l) -zero knowledge and that all masking codewords are $(t, 3l)$ -zero knowledge. In our proof we show that this is enough to make the folded codes (t, l) -zero knowledge.

5.2 Collaborative PCS from our coIOPP

In this section, we show how to construct a collaborative polynomial commitment scheme. In more detail, our scheme enables multiple provers $\{\mathcal{P}_1, \dots, \mathcal{P}_N\}$ to jointly compute the commitment of the polynomial $f^* : \mathbb{F}^{\log km} \rightarrow \mathbb{F}$ that is the multi-linear extension of the concatenation of the provers' inputs $\mathbf{X} = (\mathbf{X}_1, \dots, \mathbf{X}_N) \in \mathbb{F}^{k \times m}$ and subsequently generate a proof showing that $y = f^*(\mathbf{t})$ for any point $\mathbf{t} \in \mathbb{F}^{\log km}$. For this, we first construct a coIOPP for the relation $\mathcal{R}'_{MLE}$ defined as:

$$\mathcal{R}'_{MLE} = \{\mathbf{x} = (i, N, \delta, y, \mathbf{t}); \{\mathbf{w}_i = (\mathbf{C}_i^{(1)}, \mathbf{X}_i)\}_{i \in [N]} : \\$$

$$f^*(\mathbf{t}) = y \wedge \Delta(\mathbf{C}^{(1)}, \text{Enc}_{\mathcal{C}_i}(\mathbf{X})) \leq \delta\}$$

Then, show that our coIOPP has negligible round-by-round (knowledge) soundness error and perfect t -zero knowledge which enable us to instantiate a coPCS using our BCS transformation of Section 4. For the rest of this section we focus on giving a high level idea of our coIOPP protocol.

We partition our coIOPP into three sub-protocols: A pre-processing, a commit and an evaluation protocol where the commit and evaluation protocols will correspond to the commitment and evaluation protocols of the coPCS. With that in mind, the Pre-Processing protocol is similar to that of our coIOPP while the commitment protocol is the same with the first two steps of **Construction 3**. Our evaluation protocol applies the ideas of Basefold [23] that combines the sumcheck protocol with an IOPP. In more detail, the Basefold's prover runs an FRI-like IOPP over a codeword that encodes the coefficients of the polynomial f^* . In each round of that IOPP the prover also completes one round of the sumcheck protocol over the instance $y = \sum_{\mathbf{x} \in \{0,1\}^{\log km}} f^*(\mathbf{x})\beta(\mathbf{t}, \mathbf{x})$. At the final round of the IOPP, the verifier decodes the "reduced" polynomial of f^* and continues with rest of the sumcheck rounds locally. Note however, that the sumcheck replies might reveal some information about the committed polynomial even though our collaborative IOPP is t -zero knowledge. To overcome this challenge, someone could use the zero-knowledge variant of the sumcheck protocol by linearly combining the above instance with a random polynomial. In fact, this approach has also

been used to make Basefold zero-knowledge [68, 66]. This, however, would require the provers to generate some additional randomness just to instantiate the random polynomial. Instead, we prove the above sumcheck in zero-knowledge directly by using the masking codewords of the previous section.

We denote with $p^*, p : \mathbb{F}^2 \rightarrow \mathbb{F}$ the bivariate polynomials that interpolate $\mathbf{X}$ and the “masked” input $\tilde{\mathbf{X}}$. In addition, we let $g^* : \mathbb{F}^{\log km} \rightarrow \mathbb{F}$ and $g : \mathbb{F}^{\log k'm'} \rightarrow \mathbb{F}$ be the multi-linear extensions of the coefficients of p^* and p .

Preprocessing Step. In this step, provers run a protocol similar to the preprocessing step of the coIOPP protocol of Section 5.1. The only difference now is that for the first $\log m' - c - 2$ codewords, they do not set the underlying matrixes $\mathbf{R}_i$ to be equal to $\mathbf{0}$ but, instead, they set the first two columns of the matrix $\mathbf{R}_i$ be random. Looking ahead, this minor modification will enable us to mask the sumcheck messages in the evaluation protocol. At the end of this step, each prover holds the i -th row of all codewords $\mathbf{C}_r^{(j)}$. Furthermore, it interpolates this row as a m' -degree polynomial $p_{i,r}^{(j)}$ and computes the multi-linear extension of its coefficients denoted as $g_{i,r}^{(j)}$.

Commitment Protocol. Initially, the provers interact following the Π^{Enc} protocol so that each party $\mathcal{P}_i$ eventually receives the i -th row of the codeword of $\mathbf{X}$ denoted with $\mathbf{C}^{(1)}$. In addition, each prover $\mathcal{P}_i$, also stores the m' -degree univariate polynomial $r^{(i)}(Y)$, produced by interpolating $\mathbf{C}_i^{(1)}$, as well as $g_i : \mathbb{F}^{\log m'} \rightarrow \mathbb{F}$ that is the multi-linear extension of the coefficients of $r^{(i)}$ (step 1). Finally, the provers send the oracle of $\mathbf{C}^{(1)}$ to the verifier. This process repeats for all masking codewords $\mathbf{C}_r^{(1)}, \dots, \mathbf{C}_r^{(\log m' - c)}$.

Protocol Description. First, observe that the authors of Basefold directly encode the coefficients of f^* . This subsequently enables them to link the reduced polynomial obtained by the final codeword with the reduced sumcheck instance. Unfortunately, it is unclear how to achieve something similar in the collaborative setting. That is because our encoding scheme, directly encodes the coefficients of g —the polynomial that interpolates $\tilde{\mathbf{X}}$ —and not f^* . To overcome this, we transform the Basefold’s sumcheck instance so that it involves the polynomial $g^* : \mathbb{F}^{\log km} \rightarrow \mathbb{F}$ and not f^* . To achieve this, we observe that:

$$\begin{aligned}
y &= \sum_{i,j \in [k] \times [m]} p(z_1^i, z_2^j) \beta(\mathbf{t}_1, \mathbf{i}) \beta(\mathbf{t}_2, \mathbf{j}) \\
&= \sum_{i \in [k]} \sum_{j \in [m]} p(i, j) \beta(\mathbf{t}_1, \mathbf{i}) \beta(\mathbf{t}_2, \mathbf{j}) \\
&= \sum_{i \in [k]} \sum_{j \in [m]} \left(\sum_{a,b \in [k] \times [m]} (z_1^i)^a (z_2^j)^b c_{a,b} \right) \beta(\mathbf{t}_1, \mathbf{i}) \beta(\mathbf{t}_2, \mathbf{j}) \\
&= \sum_{a,b \in [k] \times [m]} c_{a,b} \sum_{i,j \in [k] \times [m]} z_1^{ia} z_2^{jb} \beta(\mathbf{t}_1, \mathbf{i}) \beta(\mathbf{t}_2, \mathbf{j}) \\
&= \sum_{a,b \in [k] \times [m]} c_{a,b} \left(\sum_{i \in [k]} (z_1^a)^i \beta(\mathbf{t}_1, \mathbf{i}) \right) \left(\sum_{j \in [m]} (z_2^b)^j \beta(\mathbf{t}_2, \mathbf{j}) \right) \\
&= \sum_{a,b \in [k] \times [m]} c_{a,b} \cdot (p_{\mathbf{t}_1}(z_1^a) p_{\mathbf{t}_2}(z_2^b))
\end{aligned}$$

By setting the multi-linear polynomials $f_{\mathbf{t}_1} : \mathbb{F}^{\log k} \rightarrow \mathbb{F}$, $f_{\mathbf{t}_2} : \mathbb{F}^{\log m} \rightarrow \mathbb{F}$ such that $f_{\mathbf{t}_i}(\mathbf{j}) = p_{\mathbf{t}_i}(z_i^j)$, then we can replace f^* from the original Basefold sumcheck instance with g^* as follows:

$$y = \sum_{\mathbf{x}_1, \mathbf{x}_2 \in \{0,1\}^{\log k + \log m}} g^*(\mathbf{x}_1, \mathbf{x}_2) f_{\mathbf{t}_1}(\mathbf{x}_1) f_{\mathbf{t}_2}(\mathbf{x}_2) \quad (2)$$

We note, however, that there is still one minor issue. The above sumcheck is defined over g^* , the multi-linear extension of the coefficients of p^* , but $\mathbf{C}^{(1)}$ encodes the coefficients of p —the polynomial that interpolates the masked message. Consequently, we have to replace g^* with g . This is done by padding $\mathbf{t}_1$ and $\mathbf{t}_2$ with $\log k' - \log k$ and $\log m' - \log m$ zeros respectively. We denote the padded evaluation points with $\mathbf{t}'_1, \mathbf{t}'_2$.

Construction 4 gives a detailed description of our evaluation protocol. Our protocol makes use of the $\mathcal{F}_{ip}$ functionality, that takes as input the packed secret shares of the columns of a matrix $\mathbf{R} \in \mathbb{F}^{k \times m}$ and a public vector $\mathbf{y} = (\mathbf{y}_1 \otimes \mathbf{y}_2) \in \mathbb{F}^{km}$, for $\mathbf{y}_1 \in \mathbb{F}^k$ and $\mathbf{y}_2 \in \mathbb{F}^m$ and outputs $y = \sum_{i,j} \mathbf{R}[i,j] \mathbf{y}_1[i] \mathbf{y}_2[j]$.

With that in mind, all parties interact over a modified variant of our coIOPP commit protocol of our coIOPP (for the closeness of our code) in that the provers not only aggregate and then fold the codeword $\mathbf{C}^{(i)}$ with the masked one $\mathbf{C}_r^{(i)}$, but also their sumcheck instances—since otherwise the verifier would be able to validate the last step of the sumcheck. In more detail, for every round $j \in [\log m' - c - 1]$ this is done as follows. First, all provers setup the vector containing all the folding challenges of the verifier up until that round (i.e., $\mathbf{r} = (r_1, \dots, r_j)$) and then, invoke the functionality $\mathcal{F}_{ip}$ to compute $y_r^{(j)} = \sum_{\mathbf{x}, \mathbf{y}} f_{\mathbf{t}'_1}(\mathbf{x}) f_{\mathbf{t}'_2}(\mathbf{r}, \mathbf{y}) g_r^{(j)}(\mathbf{x}, \mathbf{y})$ (step 1.a). Finally, $\mathcal{P}_1$ sends $y_r^{(j)}$ to the verifier which replies with the aggregation challenge b_j (step 1.b). Given b_j provers aggregate the sumcheck instances of $\mathbf{C}^{(j)}$ and $\mathbf{C}_r^{(j)}$ by linearly combining the polynomials $g(\mathbf{r}, \mathbf{x})$ and $g_r^{(j)}(\mathbf{x})$, and running one sumcheck round over $y + b_j y_r^{(j)} = \sum_{\mathbf{x}, \mathbf{y}} f_{\mathbf{t}'_1}(\mathbf{x}) f_{\mathbf{t}'_2}(\mathbf{r}, \mathbf{y}) (g(\mathbf{r}, \mathbf{x}) + b_j g_r^{(j)}(\mathbf{x}))$ to compute the reply polynomial $h^{(j)}(X)$.

In the collaborative setting this is done as follows. First each prover $\mathcal{P}_i$ sets $g_i(\mathbf{r}, \mathbf{x}) \leftarrow g_i(\mathbf{r}, \mathbf{x}) + b_j g_{i,r}^{(j)}(\mathbf{x})$. Then, it runs the first round of the sumcheck instance $\sum_{\mathbf{y}} g_i(\mathbf{r}, \mathbf{y}) f_{\mathbf{t}'_2}(\mathbf{r}, \mathbf{y})$ and sends the reply polynomial $\hat{h}^{(j)}(\omega_1^i, X)$ to $\mathcal{P}_1$ (step 1.c). The latter, interpolates all such polynomials into $h(Y, X)$ that has individual degree of k' and 3. Finally, it computes sumcheck reply which is set to $h^{(j)}(X) \leftarrow \sum_{\mathbf{y} \in \{0,1\}^{\log k}} \beta(\mathbf{t}_1, \mathbf{y}) h(\mathbf{y}, X)$ (step 1.d). Upon receiving $h^{(j)}(X)$, the verifier sends the folding challenge r_j to all provers which use it to fold $\mathbf{C}^{(j)} + b_j \mathbf{C}_r^{(j)}$ into $\mathbf{C}^{(j+1)}$, the oracle of which is sent to the verifier.

After completing all iterations of step 1, each prover $\mathcal{P}_i$ sends to the verifier the i -th row of $\mathbf{C}^{(\log m' - c)}$. Finally, the verifier runs if the Query algorithm of our coIOPP and the verification algorithm of the sumcheck protocol. After completing the last one, it will end up with an evaluation claim $y^{(\log m' - c)} = h^{(\log m' - c)}(r_{\log m' - c})$ which, similarly to Basefold, is validated locally by the verifier using the polynomial that interpolates the codeword $\mathbf{C}^{(\log m' - c)}$.

Optimization 2. Observe in order to perform the third check of step 3, the verifier has to compute $f_{\mathbf{t}_2}(\mathbf{r}, \mathbf{y})$, which requires $\mathcal{O}(m)$ field operations. In order to reduce verification time to $\mathcal{O}(\log m)$, the verifier delegates that step to $\mathcal{P}_1$ which uses a protocol similar to the one presented in Figure 1 of [69]. In this way, the verification time is reduced to $\mathcal{O}(\log^2 m)$ and adds an overhead of $\mathcal{O}(m)$ to $\mathcal{P}_1$ (when using the specialized sumcheck for iFFTs/FFTs presented in [70]).

Construction 4: Our coIOPP for $\mathcal{R}'_{MLE}$. Assume N provers $\{\mathcal{P}_1, \dots, \mathcal{P}_N\}$ and a verifier $\mathcal{V}$. Each prover holds its own private input $\mathbf{X}_i$, and collaborates with all other provers to instantiate a IOPP for $\mathcal{R}'_{MLE}$ over the multi-linear extension $f^* : \mathbb{F}^{\log km} \rightarrow \mathbb{F}$ of the concatenation of all inputs $\mathbf{X} = (\mathbf{X}_1, \dots, \mathbf{X}_N)$.

Pre-Processing. All parties run the $\mathcal{F}_{rand}^{Enc}$ functionality to initialize and encode $[\log m' - c - 1]$ (secret) random matrixes $\mathbf{R}_i \in \mathbb{F}^{k \times \frac{m}{2^{i-1}}}$ defined as follows: For all matrixes except from $\mathbf{R}_i^{\log m' - c - 1}$

all columns are zero except from the first two. As for $\mathbf{R}_i^{\log m' - c - 1}$ all columns are random. At the end of this protocol, each prover $\mathcal{P}_i$ holds the i -th row $(t, 3l)$ -zero knowledge codes $\mathbf{C}_i^{(j)}$. It also holds $g_{i,r}^{(j)}$, the multi-linear extension of the coefficients of the polynomial $p_r^{(j)}(\omega_1^{i+k'}, x)$ (which are directly obtained by $\mathbf{C}_i^{(j)}$).

Protocol Π^{Commit} . Each prover $\mathcal{P}_i$ initially has its own private input $\mathbf{X}_i \in \mathbb{F}^{k \times \frac{m}{N}}$. All provers collaborate to output a commitment of the multi-linear polynomial $f^* : \mathbb{F}^{\log km} \rightarrow \mathbb{F}$ using the following protocol:

1. $\mathcal{P}_i$: Invoke the Π^{Enc} protocol to obtain $\mathbf{C}_i^{(1)}$, the i -th row of $\mathbf{C}^{(1)} \in \text{RS}_1^{\otimes 2}$. In addition, form the polynomial $g_i : \mathbb{F}^{\log m'} \rightarrow \mathbb{F}$ that is the multi-linear extension of the coefficients of the m' -degree polynomial $r^{(i)}(Y)$ (obtained at step 3 of **Construction 2**).
2. $\mathcal{P}_1$: Send the oracle of $\mathbf{C}^{(1)}$ and those of the masking codewords $\mathbf{C}_r^{(1)}, \dots, \mathbf{C}_r^{(\log m' - c - 1)}$.

Protocol Π^{Eval} . In this protocol, all parties are given an oracle of $\mathbf{C}$, an evaluation point $\mathbf{t} = (\mathbf{t}_1, \mathbf{t}_2) \in \mathbb{F}^{\log k} \times \mathbb{F}^{\log m}$ as well as a point $y \in \mathbb{F}$ claimed to be $y = f^*(\mathbf{t})$. Furthermore, let $\mathbf{t}' = (\mathbf{t}'_1, \mathbf{t}'_2)$ be the padded evaluation points. Each prover $\mathcal{P}_i$ also holds the i -th row of $\mathbf{C}^{(1)}$, the multi-linear polynomial $g_i : \mathbb{F}^{\log m'} \rightarrow \mathbb{F}$ as well as the i -th row of the mask codewords $\mathbf{C}_r^{(j)}$ and the multi-linear extensions $g_{i,r}^{(j)} : \mathbb{F}^{\log m' - i} \rightarrow \mathbb{F}$ as defined in the main body. All parties interact as follows:

1. For $j = \{1, \dots, \log m' - c - 1\}$ work as follows:
 - (a) $\mathcal{P}_1, \dots, \mathcal{P}_N$: If $j = 1$ set $\mathbf{r} \leftarrow \{\}$ otherwise $\mathbf{r} \leftarrow \{r_1, \dots, r_{j-1}\}$. Call the $\mathcal{F}_{ip}$ functionality to compute the inner product $y_r^{(j)} = \sum_{\mathbf{x}, \mathbf{y}} f_{\mathbf{t}'_1}(\mathbf{x}) f_{\mathbf{t}'_2}(\mathbf{r}, \mathbf{y}) g_r^{(j)}(\mathbf{x}, \mathbf{y})$.
 - (b) $\mathcal{V}$: Samples a random point $b_j \in \mathbb{F}$ and sends it to the provers.
 - (c) $\mathcal{P}_i$: Set $g_i(\mathbf{r}, \mathbf{x}) \leftarrow g_i(\mathbf{r}, \mathbf{x}) + b_j g_{i,r}^{(j)}(\mathbf{x})$. Then, compute and send to $\mathcal{P}_1$ the polynomial $\hat{h}^{(j)}(\omega_1^i, X) = \sum_{\mathbf{x} \in \{0,1\}^{\log m' - j}} g_i(\mathbf{r}, X, \mathbf{x}) f_{\mathbf{t}'_2}(\mathbf{r}, X, \mathbf{x})$.
 - (d) $\mathcal{P}_1$: Collect all $\hat{h}^{(j)}(\omega_1^1, X), \dots, \hat{h}^{(j)}(\omega_1^N, X)$, interpolate the polynomial $h(Y, X)$ of individual degree k' and 3 respectively. Finally, send $h^{(j)}(X) \leftarrow \sum_{\mathbf{y} \in \{0,1\}^k} \beta(\mathbf{t}_1, \mathbf{y}) h(\mathbf{y}, X)$ to $\mathcal{V}$.
 - (e) $\mathcal{V}$: Pick a random point r_j and send it to all the provers.
 - (f) $\mathcal{P}_i$: Set $\mathbf{C}_i^{(j+1)} \leftarrow \text{Fold}_{RS}(r_j, \mathbf{C}_i^{(j)} + b_j \mathbf{C}_{r,i}^{(j)})$. $\mathcal{P}_1$ sends the accumulated oracle $\mathbf{C}^{(j+1)}$ to $\mathcal{V}$.
2. $\mathcal{P}_i$: Send $\mathbf{C}_i^{(\log m' - c)}$ directly to $\mathcal{V}$.
3. $\mathcal{V}$: Check if: (1) *coIOPP.Query* outputs accept, (2) $h^{(i-1)}(r_{i-1}) + b_i y_r^{(i)} = h^{(i)}(0) + h^{(i)}(1)$ with $h^{(0)}(r_0) = y$ and (3) $h^{(\log m' - c - 1)}(r_{\log m' - c}) = \sum_{\mathbf{x}, \mathbf{y} \in \{0,1\}^{\log k' + \log c}} f_{\mathbf{t}'_1}(\mathbf{x}) f_{\mathbf{t}'_2}(\mathbf{r}, \mathbf{y}) g'(\mathbf{x}, \mathbf{y})$, where g' the multi-linear polynomial obtained by decoding $\mathbf{C}^{(\log m' - c)}$ and $\mathbf{r} = (r_1, \dots, r_{\log m' - c - 1})$.

Lemma 1 (Performance). *Let $L = km$ be the total size of the message and N the number of parties. Then, the coIOPP of **Construction 4** has the following performance characteristics:*

- $k_V(L, N) \in \mathcal{O}(\log L)$
- $k_P(L, N) \in \mathcal{O}(\log L)$
- $l(L) \in \mathcal{O}(L)$ and $l_i(L, N) \in \mathcal{O}(\frac{L}{N})$
- $q(L, N) = vt(L, N) \in \mathcal{O}(l \log L + N)$
- $pt_i(L, N) \in \mathcal{O}(\frac{L}{N} \log N)$
- $cm(L, N) = Nm \log |\mathbb{F}|$ bits.

Theorem 4. Assuming $\delta < \frac{\Delta}{3}$ and $t \leq k' - k + 1$ corrupted provers, then our construction is a coIOPP for the relation $\mathcal{R}'_{MLE}$ with perfect completeness, $\text{negl}(\lambda)$ soundness and knowledge soundness error with an expected polynomial-time extractor, round-by-round (knowledge) soundness errors of $\max(\frac{|C^{(1)}|}{|\mathbb{F}|}, (1 - \delta)^l)$ and perfect honest-verifier and prover t -zero knowledge. Finally, our BCS we get a coPCS that is transparent and post-quantum with $\mathcal{O}(\frac{L}{N} \log L)$ per-prover time complexity, $\mathcal{O}(l \log^2 L)$ proof size and verification time and total communication cost of $\mathcal{O}(L)$, where $L = mn$ the size of the polynomial.

Proof Intuition. The proof is similar to that of Theorem 3 but adapted to encompass the additional sumcheck. We provide more a detailed proof in Appendix E.

6 Our Collaborative PIOP and SNARK

In this section, we construct a coPIOP for the NP relation $\mathcal{R}_C = \{(\mathbf{x}; \mathbf{w}) : C(\mathbf{x}, \mathbf{w}) = 1\}$ where C is an arithmetic circuit. We start with our scalable, transparent and post-quantum coSumcheck and then move to our coPIOP.

6.1 Our Collaborative Sumcheck

As a first step, we propose a scalable collaborative version of sumcheck. We note that a scalable coSumcheck already exists [12, 13]. However, for t -zero knowledge, the authors rely on the masking techniques of Libra [71] and explicitly work with the EC-based multilinear PCS of [17]—making their final construction neither transparent nor post-quantum secure. Here, we propose a new coSumcheck argument that is transparent and post-quantum secure.

To simplify discussion, we focus on proving the sumcheck instance $K = \sum_{\mathbf{x} \in \{0,1\}^{\log n}} f(\mathbf{x})$, and assume all provers hold K and packed secret shares of f . Similar to Libra [71], we mask the original sumcheck instance with a random multi-linear polynomial f_r of the same degree as f but only a logarithmic number of non-zero coefficients. The challenge arises in how to instantiate the polynomial oracle of f_r in the collaborative setting. Naively, someone could commit the entirety of f_r using our coPCS. Concretely, however, this would result in a $\times 2$ overhead in both the prover and verifier, despite the fact that f_r has a small number of coefficients. Instead, we construct a specialized coIOPP for $\mathcal{R}'_{MLE}$ specifically for sparse multi-linear polynomials with up to a poly-logarithmic number of non-zero coefficients. At a high level, the provers of our coIOPP in a pre-processing phase collaborate to encode the (secret) matrix $\mathbf{R}_1$ consisting of the non-zero coefficients of f_r , the oracle of which is sent to the verifier. Furthermore, provers repeat the same process for a different masking (secret) matrix $\mathbf{R}_2$ of the same dimension as $\mathbf{R}_1$. Next, to generate an evaluation proof, the provers first compute the evaluation claims of f'_r (the sparse polynomial of $\mathbf{R}_2$) at the same points as f_r (using

the $\mathcal{F}_{ip}$ functionality), and then accumulate these claims into a single masked polynomial f_r'' the matrix of which $\mathbf{R}'$ is given to the verifier who can now check these claims locally.

Detailed Description of the Protocol. **Construction 5** provides a detailed description of our coSumcheck. Initially, all provers invoke the $\mathcal{F}_{rand}^{Enc}(t, l)$ functionality to (secretly) sample the matrixes $\mathbf{R}_1, \mathbf{R}_2 \in \mathbb{F}^{k \times \log \frac{n}{k}}$ and compute their codewords $\mathbf{C}^{(1)}$ and $\mathbf{C}^{(2)}$. Each prover $\mathcal{P}_i$ obtains the i -th row of $\mathbf{C}^{(1)}, \mathbf{C}^{(2)}$ and uses it to compute the i -th packed secret share of every column of $\mathbf{R}_1$ and $\mathbf{R}_2$. The verifier, obtains the oracles of $\mathbf{C}^{(1)}, \mathbf{C}^{(2)}$.

After that phase, all parties have everything they need to run the coSumcheck protocol. First, using $\mathbf{R}_1$, each prover forms the i -th secret share of the random polynomial $f_r : \mathbb{F}^{\log n} \rightarrow \mathbb{F}$. This is done by producing a vector of size $\frac{n}{k}$ for which all elements are zero except from the indexes in $\{1, 2, \dots, 2^{\log \frac{n}{k}}\}$ where $\mathcal{P}_i$ assigns the i -th secret shares of $\mathbf{R}_1$. Next, all parties interact using the $\mathcal{F}_{ip}$ functionality to compute $K' = \sum_{i,j} \mathbf{R}_1[i, j] = \sum_{\mathbf{x}} f_r'(\mathbf{x})$ and send K' to the verifier. The later replies with a random challenge b and provers run the collaborative sumcheck (step 3) over the sumcheck instance: $K + bK' = \sum_{\mathbf{x}} (f(\mathbf{x}) + bf_r(\mathbf{x}))$. This is done by calling the $\mathcal{F}_{sumcheck}$ functionality which takes as input the secret shared polynomials and outputs an accepting sumcheck proof. In practice, we instantiate this functionality using the protocol of Figure 19 of [13].

In the last round of the sumcheck, the verifier ends up with evaluation claims of f and f_r at a random point $\mathbf{r}$ the correctness of which is validated by the polynomial oracles of f and f_r . In practice, the oracle of f can be instantiated using the coPCS scheme of Section 5.2. So we will focus on efficiently instantiating the oracle of f_r . For this, provers first jointly compute $y_r' \leftarrow f_r'(\mathbf{r})$, using $\mathbf{R}_2$, where $f_r' : \mathbb{F}^{\log n} \rightarrow \mathbb{F}$ is the sparse multi-linear polynomial of $\mathbf{R}_2$. This is done by invoking the $\mathcal{F}_{ip}$ functionality on $\mathbf{R}_2$ (which is a specialized MPC protocol computing the inner product of a secret vector with a public tensor-like vector [12, 11]). Next, $\mathcal{P}_1$ sends y_r' to the verifier which replies with a challenge $c \in \mathbb{F}$. Finally, each prover $\mathcal{P}_i$ sends aggregated tensor-code row $\mathbf{C}'_i \leftarrow \mathbf{C}_i^{(1)} + c\mathbf{C}_i^{(2)}$ to the verifier. Upon collecting all aggregated rows, the verifier forms $\mathbf{C}'$ and performs the following checks: (1) First, it checks if $\mathbf{C}'$ is a valid codeword and decodes $\mathbf{R}' = \mathbf{R}_1 + c\mathbf{R}_2$. (2) Then, it checks if $\mathbf{C}'$ is very “close” to the linear combination of $\mathbf{C}^{(1)}$ and $\mathbf{C}^{(2)}$. This is done by checking that $\mathbf{C}'[i] = \mathbf{C}^{(1)}[i] + c\mathbf{C}^{(2)}[i]$ for l arbitrary indexes. (3) Finally, the verifier checks if the sparse polynomial f_r'' formed by $\mathbf{R}'$ evaluates on $y_r + cy_r'$ at the point $\mathbf{r}$.

Optimization. Observe that the final step has query and verification complexity of $\mathcal{O}(N(l + \log n))$. It is possible to reduce the query complexity to $\mathcal{O}(N + l \log(l + \log n))$, by having the verifier delegate steps (1) and (3) to the provers. This can be done by running a (non-zero knowledge) coIOPP similar to that of 5.2. This only concretely increases the number of rounds between the verifier and provers. However, it concretely reduces the verification time and, asymptotically, the query complexity.

Construction 5: Our coSumcheck. Each prover $\mathcal{P}_i$ is given a packed secret share of the coefficients of the multi-linear polynomial $f : \mathbb{F}^{\log n} \rightarrow \mathbb{F}$, denoted with $\llbracket f \rrbracket_i$. We assume that the verifier has a polynomial oracle of f .

Pre-Processing. All provers invoke the functionality $\mathcal{F}_{rand}^{Enc}$ for the random matrixes $\mathbf{R}_1, \mathbf{R}_2 \in \mathbb{F}^{k \times (\log \frac{n}{k})}$. Each prover $\mathcal{P}_i$ holds the i -th rows of the codewords $\mathbf{C}^{(1)}, \mathbf{C}^{(2)}$. In addition, each prover also uses that row to retrieve the i -th packed secret share of each column of $\mathbf{R}_1, \mathbf{R}_2$. Finally, the provers send the oracles of $\mathbf{C}^{(1)}, \mathbf{C}^{(2)}$ to $\mathcal{V}$.

Protocol $\Pi^{coSumcheck}$. Each prover has a claimed sum $K \in \mathbb{F}$, and wants to convince a verifier $\mathcal{V}$ that $K = \sum_{\mathbf{x} \in \{0,1\}^{\log n}} f(\mathbf{x})$.

1. $\mathcal{P}_i$: Using the packed secret shares of $\mathbf{R}_1$, each prover instantiates the packed secret shares of

the multi-linear polynomial $f_r : \mathbb{F}^{\log n} \rightarrow \mathbb{F}$, that is the multi-linear extension of the matrix $\mathbf{R}'_1 \in \mathbb{F}^{k \times \frac{n}{k}}$ where all columns are zero except from those with indexes $i \in \{1, 2, \dots, 2^{\log \frac{n}{k}}\}$, which equal to the $\log i$ -th row of $\mathbf{R}_1$.

2. $\mathcal{P}_1, \dots, \mathcal{P}_N$: Invoke the functionality $\mathcal{F}_{ip}$, with input $\mathbf{R}_1$ and public vectors filled with ones to compute $K' = \sum_{\mathbf{x} \in \{0,1\}^{\log n}} f_r(\mathbf{x})$. Finally, $\mathcal{P}_1$ sends K' .
3. $\mathcal{V}$: Selects a random point b and sends it to the provers.
4. All parties invoke the $\mathcal{F}_{sumcheck}$ functionality (which can be instantiated using the coSum-check protocol of [13]) for the instance $K + bK' = \sum_{\mathbf{x} \in \{0,1\}^{\log n}} (f(\mathbf{x}) + bf_r(\mathbf{x}))$. All parties get a sumcheck proof as well as the evaluation claims y and y_r of f, f_r at the point $\mathbf{r}$.
5. $\mathcal{V}$: Validates the last round of the sumcheck using y, y_r and checks the validity of y by querying the polynomial oracle of f . For y_r it works as follows:
 - (a) $\mathcal{P}_1, \dots, \mathcal{P}_N$: Invoke the $\mathcal{F}_{ip}$ functionality (using $\mathbf{R}_2$) to compute $y'_r \leftarrow f'_r(\mathbf{r})$ where $f'_r : \mathbb{F}^{\log n} \rightarrow \mathbb{F}$ be a polynomial defined similarly as f_r but using $\mathbf{R}_2$. $\mathcal{P}_1$ sends y'_r to $\mathcal{V}$.
 - (b) $\mathcal{V}$: Sends $c \in \mathbb{F}$ to all provers.
 - (c) $\mathcal{P}_i$: Sends $\mathbf{C}'_i \leftarrow \mathbf{C}_i^{(1)} + c\mathbf{C}_i^{(2)}$ to $\mathcal{V}$.
 - (d) $\mathcal{V}$: Forms $\mathbf{C}' = (\mathbf{C}'_1, \dots, \mathbf{C}'_N)$ and performs the following checks: (1) It ensures that $\mathbf{C}'$ is a valid codeword and decodes $\mathbf{R}' = \mathbf{R}_1 + c\mathbf{R}_2$, (2) makes l queries over the oracles of $\mathbf{C}^{(1)}, \mathbf{C}^{(2)}$ and checks if $\mathbf{C}'[i] = \mathbf{C}^{(1)}[i] + c\mathbf{C}^{(2)}[i]$. (3) Finally, it forms the polynomial $f''_r \leftarrow f + cf'_r$ and checks if $f''_r(\mathbf{r}) = y_r + cy'_r$.

Finally, we claim the following which we prove in Appendix F.

Theorem 5. Fix a parameter $\delta < \frac{\Delta}{3}$. **Construction 5** is a coPIOP for proving the sumcheck instance $K = \sum_{\mathbf{x} \in \{0,1\}^{\log n}} f(\mathbf{x})$ that is perfectly complete and honest verifier and prover t -zero knowledge in the $\mathcal{F}_{rand}^{Enc}, \mathcal{F}_{sumcheck}, \mathcal{F}_{ip}$ -hybrid model. Furthermore, it has soundness error of $\epsilon = \frac{1+\log n}{|\mathbb{F}|} + (1-\delta)^l$ and round-by-round (knowledge) soundness errors of $\epsilon_{rbr} = \kappa_{rbr} = \max(\frac{2\log n}{|\mathbb{F}|}, (1-\delta)^l)$.

6.2 Our coPIOP

High Level Overview of the Spartan PIOP. Before delving into the details of our construction, we first provide an overview of the Spartan protocol [41]. Given the R1CS matrixes $(\mathbf{A}, \mathbf{B}, \mathbf{C}) \in (\mathbb{F}^{m \times n})^3$ and the proving data $(\mathbf{w}, (\mathbf{u}_L, \mathbf{u}_R, \mathbf{u}_O))$, the prover shows that: (1) $\mathbf{u}_L \circ \mathbf{u}_R = \mathbf{u}_O$ and that (2) $\mathbf{u}_L = \mathbf{A}\mathbf{w}, \mathbf{u}_R = \mathbf{B}\mathbf{w}, \mathbf{u}_O = \mathbf{C}\mathbf{w}$. The verifier is only given oracle access to the multi-linear polynomials of $\mathbf{A}, \mathbf{B}, \mathbf{C}$ and $\mathbf{w}$. To prove (1)-(2) both parties interact over the following steps: (a) They first run a sumcheck protocol over the instance $0 = \sum_{\mathbf{x} \in \{0,1\}^{\log m}} \beta(\mathbf{r}, \mathbf{x})(f_L(\mathbf{x})f_R(\mathbf{x}) - f_O(\mathbf{x}))$ where $\mathbf{r} \in \mathbb{F}^{\log m}$ a random challenge from the verifier and f_L, f_R, f_O the MLEs of $\mathbf{u}_L, \mathbf{u}_R, \mathbf{u}_O$. Let y_L, y_R, y_O be the evaluation claims of f_L, f_R, f_O at $\mathbf{r}'$ obtained at the last round of the sumcheck protocol. (b) Next, the prover receives three challenge points $a, b, c \in \mathbb{F}$ from the verifier and computes $f_p(\mathbf{x}) = af_A(\mathbf{r}', \mathbf{x}) + bf_B(\mathbf{r}', \mathbf{x}) + cf_C(\mathbf{r}', \mathbf{x})$, where f_A, f_B, f_C the MLEs of $\mathbf{A}, \mathbf{B}, \mathbf{C}$. (c) Both parties interact following the sumcheck protocol to prove (2) over the instance: $ay_L + by_R + cy_O =$

$\sum_{\mathbf{x} \in \{0,1\}^{\log n}} f_p(\mathbf{x})f_w(\mathbf{x})$, where f_w is the MLE of $\mathbf{w}$. (d) Finally, the verifier uses the oracles to validate the evaluation claims y_A, y_B, y_C and y_w of f_A, f_B, f_C and f_w obtained at the final sumcheck round.

Construction 6: Our coPIOP. $\mathcal{P}_i$'s **inputs**. Each prover holds the R1CS matrixes $(\mathbf{A}, \mathbf{B}, \mathbf{C})$, and the packed secret shares of (1) the witness $\mathbf{w}^* = (\mathbf{w}, \mathbf{w}_r = (rnd, \mathbf{0})) \in \mathbb{F}^{2n}$, where $rnd \in \mathbb{F}$ and (2) $\mathbf{u}_L = (\mathbf{A}\mathbf{w}^T, (\mathbf{r}_1, \mathbf{0}))$, $\mathbf{u}_R = (\mathbf{B}\mathbf{w}^T, (\mathbf{r}_2, \mathbf{0}))$, $\mathbf{u}_O = (\mathbf{C}\mathbf{w}^T, (\mathbf{r}_3, \mathbf{0})) \in \mathbb{F}^{2m}$, where $\mathbf{r}_1, \mathbf{r}_2 \in \mathbb{F}^2$ and $\mathbf{r}_3 \in \mathbb{F}^3$. Note that the secret shares of the random vectors have been produced in a pre-processing phase.

$\mathcal{V}$'s **inputs**. The verifier holds the polynomial oracles of the multi-linear extensions of the R1CS matrixes $f_A, f_B, f_C : \mathbb{F}^{\log m + \log n} \rightarrow \mathbb{F}$.

Protocol Π^{coPIOP} . Let $f_L, f_R, f_O : \mathbb{F}^{\log m + 1} \rightarrow \mathbb{F}$ be the multi-linear extensions of $\mathbf{u}_L, \mathbf{u}_R, \mathbf{u}_O$. Each prover works as follows:

1. $\mathcal{P}_1, \dots, \mathcal{P}_N$: Send the oracles of $f_{\mathbf{w}^*} : \mathbb{F}^{1+\log n}$, $f_1, f_2, f_3 : \mathbb{F}^{\log m} \rightarrow \mathbb{F}$ the MLEs of $\mathbf{w}^*$, $(\mathbf{r}_1, \mathbf{0})$, $(\mathbf{r}_2, \mathbf{0})$ and $(\mathbf{r}_3, \mathbf{0})$ respectively.
2. $\mathcal{V}$: Sample a random point $\mathbf{r} \in \mathbb{F}^{\log m}$ and send it to the provers.
3. $\mathcal{P}_1, \dots, \mathcal{P}_N$: All provers invoke the $\mathcal{F}_{ip}$ functionality to compute $K' = \sum_i \beta(\mathbf{i}, \mathbf{r})(\mathbf{r}_1[i]\mathbf{r}_2[i] - \mathbf{r}_3[i])$. Finally, $\mathcal{P}_1$ sends K' to $\mathcal{V}$ which replies with another random point $\mathbf{r}'$.
4. $\mathcal{P}_1, \dots, \mathcal{P}_N, \mathcal{V}$: Set $\mathbf{r} \leftarrow (\mathbf{r}, \mathbf{r}')$ and interact following the coSumcheck protocol of **Construction 5** for proving the instance:

$$r'K' = \sum_{\mathbf{x} \in \{0,1\}^{1+\log m}} \beta(\mathbf{r}, \mathbf{x}) (f_L(\mathbf{x})f_R(\mathbf{x}) - f_O(\mathbf{x}))$$

At the end of the protocol, parties end up with the evaluation claims y_L, y_R, y_O of f_L, f_R, f_O at $\mathbf{r}'$.

5. $\mathcal{V}$: Sends the random points $a, b, c \in \mathbb{F}$.
6. $\mathcal{P}_1, \dots, \mathcal{P}_N$: All provers collaborate to compute the packed secret shares of the vector $\mathbf{p}$, the MLE of which is the polynomial $f_p(\mathbf{x}) = af_A(\mathbf{r}', \mathbf{x}) + bf_B(\mathbf{r}', \mathbf{x}) + cf_C(\mathbf{r}', \mathbf{x})$. Then set $\mathbf{p}' = (\mathbf{p}, \mathbf{0}) \in \mathbb{F}^{2n}$. In addition, compute the secret shares of the vector $\mathbf{b} = [\beta((0, \mathbf{0}), \mathbf{r}'), \dots, \beta((0, \mathbf{n}-1), \mathbf{r}')] \in \mathbb{F}^n$. Set $\mathbf{b}' = (\mathbf{b}, \mathbf{0}) \in \mathbb{F}^{2n}$.
7. $\mathcal{P}_1, \dots, \mathcal{P}_N, \mathcal{V}$: Interact following the coSumcheck protocol of **Construction 5** for proving the instance:

$$ay_L + by_R + cy_O = \sum_{\mathbf{x} \in \{0,1\}^{1+\log n}} (f_{\mathbf{p}'}(\mathbf{x})f_w(\mathbf{x}) + f_{\mathbf{b}'}(\mathbf{x})(af_1(\mathbf{x}) + bf_2(\mathbf{x}) + cf_3(\mathbf{x})))$$

At the end of the protocol, the verifier ends up with the evaluation claims $y_w, y_A, y_B, y_C, y_{b'}$ and y_1, y_2, y_3 of the polynomials $f_w, f_A, f_B, f_C, f_{b'}, f_1, f_2, f_3$ at $\mathbf{r}''$.

8. $\mathcal{V}$: Validate $y_{b'}$ locally and for the rest of the polynomials use their respective oracles.

A Scalable Spartan-based coPIOP for General Circuits. We assume that all provers hold packed secret shares of the proving data: $(\mathbf{w}, (\mathbf{u}_L, \mathbf{u}_R, \mathbf{u}_O))$. That said, observe that step (a) contains a sumcheck instance with a product of two secret shared vectors and a public vector which can be proven using our coSumcheck. Furthermore, because the public vector has tensor-like structure, (e.g., it can be decomposed to a tensor product of at least two smaller vectors) we can prove the above instance without having to convert the public vector into a secret shared one [12]. In step (b), the provers need to jointly compute the packed secret shares of the vector $\mathbf{p} \in \mathbb{F}^n$. This is done, by following the protocol proposed in Section 3.3 of [13] for computing the secret shares of the reduced permutation matrix. Using the secret shared f_p and f_w , provers run our coSumcheck to complete step (c). Finally, the verifier follows step (d) of standard Spartan. To make our construction t -zero knowledge we use techniques similar to those of Section 4.2 of [71] to mask y_L, y_R, y_O and y_w .

In more detail, assume that all provers have Shamir’s packed secret shares of $\mathbf{w}, \mathbf{u}_L, \mathbf{u}_R, \mathbf{u}_O$. In addition, we also assume that the provers have packed secret shares of the vectors $\mathbf{w}_r = (rnd, 0) \in \mathbb{F}^n$, $\mathbf{r}_1, \mathbf{r}_2 \in \mathbb{F}^2$ and $\mathbf{r}_3 \in \mathbb{F}^3$. The last vectors can be computed in a pre-processing step and they are used to mask the intermediate evaluation claims produced by the Spartan PIOP. Finally, we assume that the verifier holds as the instance the original instance $\mathbf{x}$ of $\mathcal{R}_C$, and the polynomial oracles of $f_A, f_B, f_O, f_{\mathbf{w}^*} : \mathbb{F}^{\log m+1} \rightarrow \mathbb{F}$, where $\mathbf{w}^* = (\mathbf{w}, \mathbf{w}_r)$. **Construction 6** gives a detailed description of the coPIOP.

Note that the original vectors $\mathbf{w}, \mathbf{u}_L, \mathbf{u}_R, \mathbf{u}_O$ and thus their multi-linear extensions are masked by appending the random vectors $\mathbf{w}_r, \mathbf{r}_1, \mathbf{r}_2$ and $\mathbf{r}_3$. Based on this modification, our protocol works as follows. First the provers send the oracles of $f_1, f_2, f_3 : \mathbb{F}^{\log m} \rightarrow \mathbb{F}$, which are the MLEs of $(\mathbf{r}_1, \mathbf{0}), (\mathbf{r}_2, \mathbf{0}), (\mathbf{r}_3, \mathbf{0}) \in \mathbb{F}^m$. Next, the verifier samples a random point $\mathbf{r} \in \mathbb{F}^{\log m}$ and sends it to the provers (step 2), which invoke the $\mathcal{F}_{ip}$ functionality to compute $K' = \sum_{i \in [3]} \beta(\mathbf{i}, \mathbf{r})(\mathbf{r}_1[i]\mathbf{r}_2[i] - \mathbf{r}_3[i])$. This step must be necessary since $\mathbf{r}_1 \circ \mathbf{r}_2 - \mathbf{r}_3$ is usually non-zero. Finally, the verifier selects another random point $r' \in \mathbb{F}$ (step 3) and all parties interact over the sumcheck instance with input the masked polynomials f_L, f_R, f_O .

Likewise, we adapt the second sumcheck. For this, observe that $y_L = f_L(\mathbf{r}'_l, r_r) = (1 - r_r)f'_L(\mathbf{r}'_l) + r_r f_1(\mathbf{r}'_l)$, where f'_L is the MLE of $\mathbf{A}\mathbf{w}$. Using these “partial” evaluations, we expand them into sumcheck instances to form the final instance that is proven in step 7.

This is done by “expanding” the masked evaluation $y_L = f_L(\mathbf{r}'_l, r_r)$ (and y_R, y_O respectively) can be re-written as:

$$f_L(\mathbf{r}'_l, r_r) = \sum_{\mathbf{x} \in \{0,1\}^{\log n}} f_A(\mathbf{r}_l, \mathbf{x}) f_{\mathbf{w}}(\mathbf{x}) + r_r \sum_{x \in [2]} \beta(\mathbf{x}, \mathbf{r}_l) f_1(\mathbf{x}) = \sum_{\mathbf{x} \in \{0,1\}^{\log n+1}} f_A(\mathbf{r}_l)$$

6.2.1 Scalable coPIOPs for Sparse Polynomial Evaluations

So far, we assumed that the parties hold polynomial oracles of f_A, f_B, f_C . While the number of non-zero coefficients of these polynomials is at most $|C|$ their degree is prohibitively large, that is, quadratic to $|C|$. As a result, directly using a coPCS to instantiate these oracles would be prohibitively expensive. Instead, we propose a coPIOP for committing to and evaluating these polynomials in the collaborative setting (note this coPIOP does not need to be zero-knowledge). For this, we directly adapt the PIOP of [41] as it minimizes the number of committed elements, which not only leads to reduced prover time but also to smaller total communication.

To achieve this, we need to construct an efficient coPIOP for the grand product argument of [42]. Because [42] is a specialized GKR protocol for proving the correctness of a circuit with a binary tree-like structure one could use the coPIOP based on GKR as proposed in [72]. However, the number of communication rounds would be $\mathcal{O}(\log^2 |C|)$ since, the only known distributed sumcheck [72]

requires $\mathcal{O}(\log |C|)$ communication rounds and the provers must invoke it sequentially $\log |C|$ times. This is unacceptable especially in networks with large delays such as WANs.

Instead, we propose a distributed sumcheck protocol that *incurs only a constant number of communication rounds* by relying on the ideas of the sumcheck protocol of Sparrow [43]. At a very high level, we replace the first $\log N$ variables of the MLEs involved in the sumcheck with only two of individual degrees of $\log N$ and $N/\log N$. Next, the provers run only the first two rounds of the sumcheck protocol over the modified polynomials. After the second round, provers send the “reduced” polynomials to only one who runs the rest of the M/N -sized sumcheck instance locally, where M is the original instance size. Finally, following a similar protocol with that of [43], provers reduce the evaluations of the modified polynomials to the original multi-linear ones.

Detailed Description of Our Distributed Sumcheck. To ease explanation, we focus on the sumcheck instance of the form $K = \sum_{\mathbf{x} \in \{0,1\}^{\log M}} f(\mathbf{x})g(\mathbf{x})$ for f, g the MLEs of the vectors $\mathbf{x}_1, \mathbf{x}_2 : \mathbb{F}^M$. For now, we assume that $N^2 \leq \frac{M}{N}$, where N is the number of provers. Later, we will show how to modify the protocol to support any M and N .

Having established that, **Construction 7** gives a detailed description of our protocol. Initially, each prover $\mathcal{P}^{(i)}$, holds the i -th, M/N -sized segments of $\mathbf{x}_1$ and $\mathbf{x}_2$ and forms the multi-variate polynomials $\hat{f}^{(i)}, \hat{g}^{(i)}$, such that

$$\hat{f}^{(i)}(X_1, X_2, \mathbf{b}) = \sum_{i,j,\mathbf{z}} L_i^{(1)}(X_1) L_j^{(2)}(X_2) \beta(\mathbf{z}, \mathbf{b}) \mathbf{x}_1^{(i)}[i, j, \mathbf{z}]$$

Where $i, j, \mathbf{z} \in [\log N] \times [\frac{N}{\log N}] \times \{0, 1\}^{\log(M/N^2)}$ (and likewise it forms $\hat{g}^{(i)}$ using $\mathbf{x}_2^{(i)}$). Note that this is the same way Sparrow’s sumcheck [43] “modifies” the original multi-linear polynomials to multi-variate ones.

Using these polynomials, the provers follow a sumcheck protocol similar to that of [43] but in the distributed setting (and not the streaming one). First, they collaborate to compute $h_1(X_1) = \sum_{\mathbf{b}} \hat{f}(X_1, \mathbf{b}) \hat{g}(X_1, \mathbf{b})$ (steps 1-2). Upon sending h_1 and receiving the challenge r_1 from the verifier, provers collaborate to compute the second reply $h_2(X_2) = \sum_{\mathbf{b}'} \hat{f}(X_1, \mathbf{b}) \hat{g}(X_1, \mathbf{b})$ (steps 3-4). Since $h_2(X_2)$ has size proportional to N one can use the protocol of [43] to delegate the verifier’s checks to the prover $\mathcal{P}_1$.

After receiving the second challenge r_2 , each $\mathcal{P}_i$ computes the “reduced” polynomials $\hat{f}^{(i)}(r_1, r_2, \mathbf{z})$ and $\hat{g}^{(i)}(r_1, r_2, \mathbf{z})$ and sends them to $\mathcal{P}_1$ who forms the multi-linear polynomials $\hat{f}(r_1, r_2, \mathbf{z}), \hat{g}(r_1, r_2, \mathbf{z})$ (by concatenating the coefficients) and runs the rest of the sumcheck locally (steps 5-6). Finally, provers run a protocol similar to **Phase 2** of [43] to reduce the evaluation claims of the multi-variate polynomials $\hat{f}, \hat{g}$ to those of f, g .

Performance Analysis. Regarding per-prover time, step 1 requires $\mathcal{O}(M(\log \log N)/N)$ field operations since each $\mathcal{P}_i$ must compute $\frac{M}{N \log N}$ polynomial multiplications of size $\log N$ each. Step 2 requires $N \log N$ field operations by $\mathcal{P}_1$. Step 3 requires $\mathcal{O}(\frac{M}{N})$ field operations since each prover $\mathcal{P}_i$ has to do $\frac{M}{N^2 \log N}$ polynomial multiplications of size N each. Step 4 is dominated by $\mathcal{O}((N/\log N)^2)$ field operations (for computing h_2) since the delegation protocol of $\mathcal{P}_1$ requires no more than $\mathcal{O}(N)$ field and hash operations [43]. Steps 5,7 require $\mathcal{O}(M/N)$ field operations by each $\mathcal{P}_i$ while in step 6 $\mathcal{P}_1$ has to perform additionally $\mathcal{O}(M/N)$ field operations. Finally, step 8 requires $\mathcal{P}_1$ to perform $\mathcal{O}(N^2)$. Overall, the per-prover time of each $\mathcal{P}_i$ is $\mathcal{O}(M \log \log N/N)$. $\mathcal{P}_1$ has an additional $\mathcal{O}(N^2)$ overhead which (by our initial assumption), is smaller than $\frac{M}{N}$.

Regarding the number of communication rounds, observe that the only communication happening between provers is at steps 1, 3, 5 and 7. Hence the protocol has 4 communication rounds. Furthermore, it is not hard to see that these steps, have $\mathcal{O}(\log^2 N)$, $\mathcal{O}((N/\log N)^2)$, $\mathcal{O}(M/N)$ and $\mathcal{O}(N^2)$ total

Construction 7: Our Constant Round Distributed Sumcheck. Let $f, g : \mathbb{F}^{\log M} \rightarrow \mathbb{F}$ be the MLEs of $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{F}^M$. Provers interact with the verifier to show that $K = \sum_{\mathbf{x} \in \{0,1\}^{\log M}} f(\mathbf{x})g(\mathbf{x})$.

$\mathcal{P}_i$'s inputs. Each prover holds the i -th segment of $\mathbf{x}_1$ and $\mathbf{x}_2$ of size M/N each. It also initializes the multi-variate polynomials $\hat{f}^{(i)}, \hat{g}^{(i)} : \mathbb{F}^{2+\log(M/N)} \rightarrow \mathbb{F}$ in the same way as in [43], where the first two variables have degrees of $\log N$ and $N/\log N$ while the rest variables have degree 1.

$\mathcal{V}$'s inputs. The verifier holds the polynomial oracles of the multi-linear extensions of f, g .

Protocol Π^{Sumcheck} Each prover works as follows:

1. $\mathcal{P}_i$: Compute $h_1^{(i)}(X_1) = \sum_{\mathbf{b}} \hat{f}^{(i)}(X_1, \mathbf{b})\hat{g}^{(i)}(X_1, \mathbf{b})$ and send it to $\mathcal{P}_1$.
2. $\mathcal{P}_1$ - $\mathcal{V}$: $\mathcal{P}_1$ computes $h_1(X_1) = \sum_{i \in [N]} h_1^{(i)}(X_1)$ and sends it to $\mathcal{V}$ which checks if $\sum_{i \in [\log N]} h_1(\omega_1^i) = K$, samples a random point $r_1 \in \mathbb{F}$, computes $K_1 = h_1(r_1)$ sends r_1 to all provers.
3. $\mathcal{P}_i$: Compute $h_2^{(i)}(X_2) = \sum_{\mathbf{b}'} \hat{f}^{(i)}(r_1, X_2, \mathbf{b}')\hat{g}^{(i)}(r_1, X_2, \mathbf{b}')$ and send it to $\mathcal{P}_1$.
4. $\mathcal{P}_1$ - $\mathcal{V}$: $\mathcal{P}_1$ computes $h_2(X_2) = \sum_{i \in [N]} h_2^{(i)}(X_2)$. sends a PC commitment of $h_2(X_2)$ to the verifier which replies with a random point $r_2 \in \mathbb{F}$. Finally, it uses the delegation protocol of [43] (see **Protocol 1**, Second Round, step 2) to show to the verifier that $\sum_{i \in [N/\log N]} h_2(\omega_2^i) = K_1$ and that $K_2 = h_2(r_2)$.
5. $\mathcal{P}_i$: Computes and sends the multi-linear polynomials $\hat{f}(r_1, r_2, \mathbf{x}), \hat{g}(r_1, r_2, \mathbf{x}) : \mathbb{F}^{\log(M/(N^2))} \rightarrow \mathbb{F}$, to $\mathcal{P}_1$.
6. $\mathcal{P}_1$ - $\mathcal{V}$: $\mathcal{P}_1$ interacts with $\mathcal{V}$ using the standard sumcheck protocol to show $K_2 = \sum_{\mathbf{b}''} \hat{f}(r_1, r_2, \mathbf{b}'')\hat{g}(r_1, r_2, \mathbf{b}'')$.
7. $\mathcal{P}_i$: Let $\mathbf{r}' = (\mathbf{r}'_l, \mathbf{r}'_g) \in \mathbb{F}^{\log(M/N^2) \times \log N}$. $\mathcal{P}_i$ computes $f^{(i)}(\mathbf{z}, \mathbf{r}'_l), g^{(i)}(\mathbf{z}, \mathbf{r}'_l) : \mathbb{F}^{\log N} \rightarrow \mathbb{F}$ and sends it to $\mathcal{P}_\infty$.
8. $\mathcal{P}_1$ - $\mathcal{V}$: Computes $f(\mathbf{z}, \mathbf{r}') = \sum_{i \in [N]} \beta(\mathbf{i}, \mathbf{r}'_g)f^{(i)}(\mathbf{z}, \mathbf{r}'_l)$ (and likewise for g) and interacts with the verifier using the batched sumcheck protocol as described in (Step 2 of **Phase 2** of [43]). Finally, the verifier ends up with evaluation claims at f, g which are validated through the polynomial oracles.

communication complexity, which is $\mathcal{O}(N^2 + M/N)$. Finally, proof size and verification time remain the same as in [43].

Handling the Case Where $N^2 > \frac{M}{N}$. From our asymptotic analysis, observe that when $N^2 > M/N$, our distributed sumcheck stops being scalable since the complexity of $\mathcal{P}_1$ would be dominated by $\mathcal{O}(N^2)$. Note that this complexity appears at steps 4 and 8, where $\mathcal{P}_1$ computes h_2 and $f(\mathbf{z}, \mathbf{r}')$ respectively. To overcome that bottleneck, first observe that in these steps, $\mathcal{P}_1$ does nothing more than aggregating N vectors of size N . Our solution is to further parallelize this task by having each prover $\mathcal{P}_i$ send the j -th element of its vector to $\mathcal{P}_j$, for all $j \in [N]$ and $\mathcal{P}_j$ to aggregate the N elements together which are later sent back to the prover (or verifier). Hence this part is reduced from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$ and adds only two addition rounds between the provers (that is, to have $\mathcal{P}_1$ collect the aggregated vectors).

Theorem 6. *Our coPIOP for the relation $\mathcal{R}_C$ has perfect completeness, honest-verifier and prover t -*

zero knowledge with security threshold of $t \leq \frac{N}{2} - k + 1$, soundness error of $\epsilon = (3 \log m + 2 \log n + 10)/|\mathbb{F}| + 4(1 - \delta)^l$ and round-by-round (knowledge) soundness error of $\max(2 \log n/|\mathbb{F}|, 2 \log m/|\mathbb{F}|, (1 - \delta)^l)$. The non-interactive protocol obtained by the replacing the polynomial oracles with our coIOPP for $\mathcal{R}'_{MLE}$ and applying the collaborative BCS transformation is a coSNARK with $\mathcal{O}(\frac{|C|}{N} \log |C|)$ per-prover time, $\mathcal{O}(\log^2 |C|)$ proof size and verification time, $\mathcal{O}(|C|)$ total communication and $\mathcal{O}(\log |C|)$ prover rounds.

We provide a detailed proof of the above theorem in Appendix G.

Our coSNARK for SIMD Circuits. We can optimize our coSNARK when $|C|$ has SIMD structure following techniques similar to those of [13] resulting in a coSNARK with only $\mathcal{O}(N^2 \log |C|)$ total communication cost!

7 Experimental Evaluation

Here, we report our results from implementing and experimentally evaluating our scheme.

Software. We implemented our coSNARK in C++ using ≈ 8000 lines of code. For field operations we used the 61-bit Mersenne prime extension field library of [69] and for hash operations we used the open-source implementation [73] of the Blake3 hash function. To coordinate communication among provers, we used the MPI library [74]. We set the rate of the second code of our collaborative tensor-code to $\frac{1}{4}$. This results in a relative minimum distance of $\Delta = \frac{3}{8}$. We set the number of queries of each FRI round at $l = 520$, leading to ≈ 100 bits of security (as per Theorem 4). We used the same threshold configuration as in [14, 11], e.g., set the packing factor to $k = N/4$ and $t \leq N/4$.

Hardware. To run our benchmarks, we used 128 machines of type CX43 from the HETZNER Cloud Provider, with 8 vCPUs, 16GB of RAM, and running Linux Ubuntu 24.04. In all our experiments, each prover utilizes a single vCPU.

Network. We tested our protocol in three different network settings: (1) a LAN network with 10Gbps Bandwidth and round-trip-delay of $RTT=2ms$, a WAN network which we emulated using the Linux `tc` command with 200Mbps and (2) $RTT=50ms$ and (3) $RTT=100ms$.

7.1 Benchmarking Our Construction

Scalability w.r.t Variable Circuit Size. We first test the performance of our scheme when proving the correct computation of arbitrary circuits ranging from $|C| = 2^{21}$ - 2^{25} and $N = 8, 32$ and 128 provers over the three network settings. We measure the *end-to-end* prover time, including local computation as well as communication delays incurred by the network. Furthermore, we also report total communication, space utilization, proof size and verification time.

Figure 1 reports the end-to-end time needed to generate a proof, when running our coSNARK in a LAN (left), WAN with $RTT=50ms$ (middle) and WAN with $RTT=100ms$ (right) for variable circuit sizes. Overall, our prover time ranges from 3.6-49.8sec in LAN, 11.1-66sec in WAN with $RTT=50ms$ and 18.1-78.9sec in WAN with $RTT=100ms$. Observe that in WAN, the prover time increases sublinearly due to the inherent overhead added due to the network delay dominating the prover time for smaller instance sizes. This effect, however, diminishes as instances become larger.

Table 7.1 shows the total communication complexity, space utilization, proof size and verification time of our coSNARK for $N = 128$. Our total communication and space utilization per prover increase almost linearly with respect to the circuit size. Concretely, for $|C| = 2^{25}$, the total communication is 6.2GB, or roughly 48MB per prover, while its space usage is only 2.7GB. Finally, our proof size and

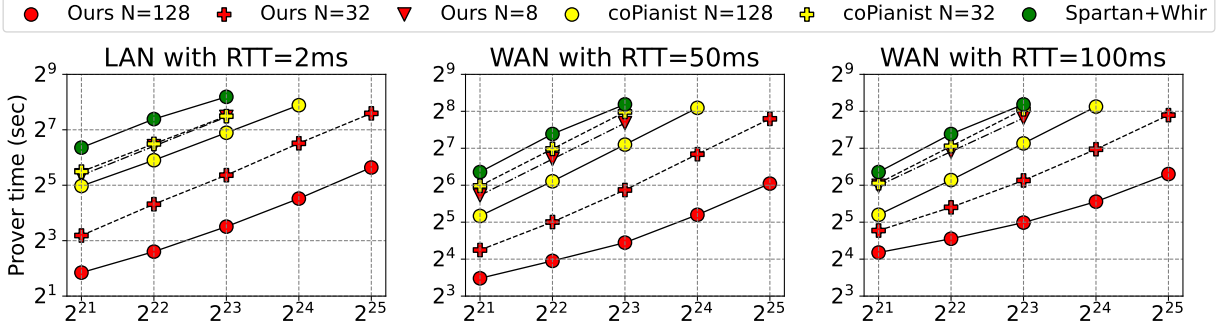


Figure 1: Time required for proof generation in seconds in LAN (left) WAN with RTT=50ms (middle) and WAN with RTT=100ms (right) of our coSNARK and coPianist for $N = 8, 32$ and 128 provers and the Spartan zkSNARK complied with the WHIR PCS and increasing circuit size.

$ C $	2^{21}	2^{22}	2^{23}	2^{24}	2^{25}
Total Commun. (GB)	0.4	0.8	1.6	3.1	6.2
Space (GB)	0.17	0.36	0.71	1.2	2.7
$ \pi $ (MB)	4.4	5	5.6	6.5	7.3
$\mathcal{V}$ (ms)	101	121	138	161	186

Table 2: From top to bottom: total communication cost, space utilization per prover, proof size and verification time of our coSNARK for $N = 128$, and variable circuit size.

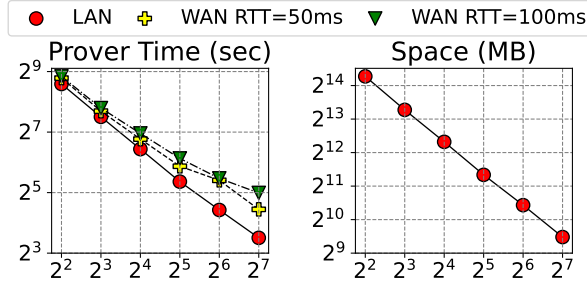


Figure 2: Prover time in seconds (left) and space utilization in MBs (right), when proving a circuit with $|C| = 2^{23}$ and provers between $N=4-128$.

communication costs increase sub-linearly. For $|C| = 2^{25}$, our proof is 7.3MB and our verifier takes 186ms.

Scalability w.r.t Number of Provers. Here, we test the scalability of our coSNARK by measuring end-to-end prover time, proof size, verification time, total communication cost and space utilization for $|C| = 2^{23}$ and a number of provers ranging between $N = 4-128$.

Figure 2 shows the prover time (left) and space utilization (right) for a fixed circuit size and variable number of provers. As expected, prover time reduces from 385.1sec to 11.3sec in a LAN, 440.7sec to 21.8sec in a WAN (with RTT=50ms) and 458.3sec to 31.77sec in a WAN (with RTT=100ms). For the LAN, this is an $\times 34$ improvement, indicating excellent scalability as the number of provers increases! In WAN, we get a slightly smaller improvement of $\times 20.1$ and $\times 14.4$ —due to the inherent overhead incurred by the communication between the provers. As mentioned above, this improvement becomes better for larger instances. Next, note that the communication cost remains almost identical e.g., it only increases from 1.56GB to 1.6GB. This is expected from our analysis, since the majority of communication comes from computing and committing to witness information for the sparse evaluation PIOP.

Similarly to prover time, the memory utilization of each prover also decreases from 19.8GB to only

714MB—a $\times 29$ improvement! In other words, the more provers we have, the larger the circuits we can prove, with fewer limitations due to memory exhaustion. Finally, the proof size and verification times remain almost identical, i.e., almost independent from the number of provers—which again agrees with our asymptotic analysis. In particular, they both range within 5.5-5.6MB and 116-125ms, respectively, for $N=4$ -128.

Performance of our scheme on SIMD Circuits. Finally, we also tested the performance of our scheme when proving a SIMD circuit consisting of $|C|/\log |C|$ identical copies of sub-circuits of $\log |C|$ gates each (again we range $|C|$ from 2^{21} - 2^{25}). We observed that our prover time significantly decreases, ranging from 1.5-27.1sec in LAN, 5.8-33.7sec in WAN with RTT=50ms and 9.7-39.1 in WAN with RTT=100ms for $N=128$. This is roughly a $\times 2$ improvement over general circuits. We note that a similar trend also appears in memory utilization. This decrease stems from the fact that our prover does not have to invoke the PIOP for sparse polynomial evaluation. In addition, the total communication decreases significantly, ranging from 0.40-0.49MB, as per the asymptotic complexity analysis in Section 5.1. Finally, the proof size and verification times decrease to 2.1-3.7MB and 34-56ms, respectively.

7.2 Comparison with Prior Works

Next, we compare our coSNARK with prior works. Recall that there exist no prior post-quantum secure, scalable coSNARKs. Hence, for benchmarking purposes, we compare with a “standard” centralized (non-private) SNARK, the state-of-the-art non-post-quantum secure scalable coSNARKs [14, 13], and a non-scalable coSNARK [6].

Comparison with a non-private SNARK. As a baseline, we investigate the scalability of our coSNARK by comparing it with a “monolithic” non-private SNARK based on the Spartan protocol, but we replaced its elliptic-curve-based PCS with the state-of-the-art WHIR [21] that not only offers faster prover and verification times but also has the smallest proof size compared to other hash-based schemes. As Figure 1 suggests, in the LAN setting, even for $N = 8$ provers, the prover time of our scheme $\times 1.9$ times faster than generating the proof locally. This indicates an almost perfect speedup, as N grows. Furthermore, observe that for more provers, our coSNARK scales for larger instances due to the smaller space requirements per prover.

Comparison with Scalable coSNARKs. Next, we compare our scheme with the scalable coSNARKs of [14, 13].

Comparison with [14]. Figure 1 compares our coSNARK with the coSNARK of [14], which we call coPianist (we omit benchmarks for $N = 8$ since their prover time is slower than computing a proof locally). Our coSNARK is $\times 8.6$ -10.3 faster in LAN and $\times 3.2$ -7.4 and $\times 2$ -5.9 in both WANs. E.g., for 128 provers and $|C| = 2^{24}$, our prover time is 22.91, 36.7 and 47.04sec in LAN and both WANs respectively, while that of coPianist is 236.5, 273 and 279.5sec. Finally, our total communication complexity and space utilization is $\times 36$ and $\times 3$ -8.3 smaller! E.g., for $|C| = 2^{24}$, $N = 128$, provers communicate in total 116GB and use 10.9GB of memory vs our 3.1GB and 1.2GB. As discussed in the introduction, this huge improvement, especially in communication, is expected as coPianist requires $\mathcal{O}(|C|)$ communication for each multiplication level.

Comparison with [13]. As shown in [14], the prover time of [13] is significantly larger than coPianist, making it much slower than our coSNARK. Moreover, we outperform [13] in communication complexity and memory utilization. For $N=128$ and $|C| = 2^{21}$, their total communication cost is 5.5GB compared to our 400MB, while space utilization of their prover is 492.5MB compared to our 174MB.

Both schemes have significantly smaller proofs and verification times, which is typical for EC-based schemes.

Comparison with Non-Scalable coSNARKs. Recall the per-prover time and space utilization of non-scalable coSNARKs remains constant regardless of the number of parties N , as opposed to in scalable coSNARKs where they decrease proportionally to N . Hence, for large enough N , our scheme always outperforms non-scalable coSNARKs.

That said, comparison with non-scalable coSNARKs is still meaningful in the context of specific scenarios, such as private delegation. For our comparison, we assume a delegator with a “budget” of 128 computing nodes and compare our coSNARK with the state-of-the-art non-scalable coSNARK, DFS [6], which allows a delegated prover to internally parallelize its workload (non-privately) across multiple computing nodes. When using our coSNARK, each computing node corresponds to a different proving party. Using the AddSS secret sharing version of DFS, we assume 2 proving parties (which is the best possible scenario), each controlling 64 machines. With this setup, our prover is $\times 2.1$ faster in LAN and becomes up to $\times 1.6$ faster in WAN with $\text{RTT}=100\text{ms}$. Specifically, for $|C| = 2^{24}$, DFS takes 59.2sec compared to the 22.9 and 47sec of our scheme in LAN and WAN. Furthermore, our communication cost and space complexity are at least $\times 5$ and $\times 1.9$ smaller than DFS. As with previous comparisons, proof size and verification times of DFS are smaller, but note that it is not post-quantum nor transparent. Finally, [6] introduces a second construction based on RSS [75] but it necessitates at least three parties (and not two as AddSS). This would lead to concretely even slower prover times than AddSS, since each party can now parallelize only across 43 machines.

8 Acknowledgements

Portions of this work were performed by Dimitrios Papadopoulos as part of consulting services to Lagrange Labs. Charalampos Papamanthou was partially supported by the National Science Foundation.

References

- [1] A. Ozdemir and D. Boneh, “Experimenting with collaborative zk-snarks: Zero-knowledge proofs for distributed secrets,” in *USENIX Security*, 2022, pp. 4291–4308.
- [2] J. Groth, “On the size of pairing-based non-interactive arguments,” in *Eurocrypt*. Springer, 2016, pp. 305–326.
- [3] A. Chiesa, Y. Hu, M. Maller, P. Mishra, P. Vesely, and N. P. Ward, “Marlin: Preprocessing zk-snarks with universal and updatable SRS,” in *EUROCRYPT*, 2020, pp. 738–768.
- [4] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, “PLONK: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge,” Cryptology ePrint Archive, Paper 2019/953, 2019.
- [5] A. Chiesa, D. Ojha, and N. Spooner, “Fractal: Post-quantum and transparent recursive proofs from holography,” in *EUROCRYPT*, 2020, pp. 769–793.
- [6] Y. Hu, P. Mishra, X. Wang, J. Xie, K. Yang, Y. Yu, and Y. Zhang, “DFS: delegation-friendly zk-snark and private delegation of provers,” in *USENIX Security*, 2025, pp. 2065–2084.
- [7] A. Chiesa, R. Lehmkuhl, P. Mishra, and Y. Zhang, “Eos: Efficient private delegation of zk-snark provers,” in *USENIX Security*, 2023.

- [8] Y. Yang, Y. Cheng, K. Wang, X. Li, J. Sun, J. Shen, X. Dong, Z. Cao, G. Yang, and R. H. Deng, “Siniel: Distributed privacy-preserving zkSNARK,” in *NDSS*. The Internet Society, 2025.
- [9] “Collaborative SNARKs with EZKL.” [Online]. Available: <https://blog.ezkl.xyz/post/cosnarks/>
- [10] W. Wu, S. Homs, and Y. Zhang, “Confidential and verifiable machine learning delegations on the cloud,” in *European Symposium on Research in Computer Security*. Springer, 2024, pp. 182–201.
- [11] S. Garg, A. Goel, A. Jain, G. Policharla, and S. Sekar, “zkSaaS: Zero-knowledge snarks as a service,” in *USENIX Sec*, 2023, pp. 4427–4444.
- [12] X. Liu, Z. Zhou, Y. Wang, B. Zhang, and X. Yang, “Scalable collaborative zk-SNARK: fully distributed proof generation and malicious security,” *Cryptology ePrint Archive*, 2024.
- [13] X. Liu, Z. Zhou, Y. Wang, Y. Pang, J. He, B. Zhang, X. Yang, and J. Zhang, “Scalable collaborative {zk-SNARK} and its application to fully distributed proof delegation,” in *(USENIX Security)*, 2025.
- [14] Z. Fang, S. Garg, B. Roberts, W. Wu, and Y. Zhang, “Collaborative zkSNARKs with sublinear prover time and constant proof size,” *IACR Cryptol. ePrint Arch.*
- [15] M. K. Franklin and M. Yung, “Communication complexity of secure computation (extended abstract),” in *ACM STOC*, 1992, pp. 699–710.
- [16] C. Lund, L. Fortnow, H. J. Karloff, and N. Nisan, “Algebraic methods for interactive proof systems,” *J. ACM*, pp. 859–868, 1992.
- [17] C. Papamanthou, E. Shi, and R. Tamassia, “Signatures of correct computation,” in *TCC*, 2013, pp. 222–242.
- [18] N. Pippenger, “On the evaluation of powers and monomials,” *SIAM J. Comput.*, pp. 230–250, 1980.
- [19] A. Kate, G. M. Zaverucha, and I. Goldberg, “Constant-size commitments to polynomials and their applications,” in *ASIACRYPT*, 2010, pp. 177–194.
- [20] T. Liu, T. Xie, J. Zhang, D. Song, and Y. Zhang, “Pianist: Scalable zkrollups via fully distributed zero-knowledge proofs,” in *IEEE SP*, 2024, pp. 1777–1793.
- [21] G. Arnon, A. Chiesa, G. Fenzi, and E. Yogev, “WHIR: reed-solomon proximity testing with super-fast verification,” in *EUROCRYPT*, 2025, pp. 214–243.
- [22] A. Golovnev, J. Lee, S. T. V. Setty, J. Thaler, and R. S. Wahby, “Brakedown: Linear-time and field-agnostic snarks for R1CS,” in *CRYPTO*, 2023, pp. 193–226.
- [23] H. Zeilberger, B. Chen, and B. Fisch, “Basefold: Efficient field-agnostic polynomial commitment schemes from foldable codes,” in *CRYPTO*, 2024, pp. 138–169.
- [24] M. Brehm, B. Chen, B. Fisch, N. Resch, R. D. Rothblum, and H. Zeilberger, “Blaze: Fast snarks from interleaved RAA codes,” in *EUROCRYPT*, 2025, pp. 123–152.
- [25] I. S. Reed and G. Solomon, “Polynomial codes over certain finite fields,” *Journal of the society for industrial and applied mathematics*, pp. 300–304, 1960.

- [26] T. Xie, J. Zhang, Z. Cheng, F. Zhang, Y. Zhang, Y. Jia, D. Boneh, and D. Song, “zkbridge: Trustless cross-chain bridges made practical,” in *ACM CCS*, 2022, pp. 3003–3017.
- [27] H. Xu, M. Gama, E. H. Beni, and J. Kang, “Frittata: Distributed proof generation of fri-based snarks,” *IACR Cryptol. ePrint Arch.*, 2025.
- [28] Y. Yu, M. Liu, Y. Zhang, S.-F. Sun, T. Ma, M. H. Au, and D. Gu, “HyperFond: A Transparent and Post-Quantum Distributed SNARK with Polylogarithmic Communication,” *Cryptology ePrint*, 2025.
- [29] W. Li, Z. Zhang, S. S. M. Chow, Y. Guo, B. Gao, X. Song, Y. Deng, and J. Liu, “Shred-to-shine metamorphosis in polynomial commitment evolution,” *IACR Cryptol. ePrint Arch.*, 2025.
- [30] S. Ames, C. Hazay, Y. Ishai, and M. Venkatasubramanian, “Ligero: Lightweight sublinear arguments without a trusted setup,” in *ACM CCS*, 2017, pp. 2087–2104.
- [31] D. A. Spielman, “Linear-time encodable and decodable error-correcting codes,” *IEEE Trans. Inf. Theory*, pp. 1723–1731, 1996.
- [32] A. R. Block, Z. Fang, J. Katz, J. Thaler, H. Waldner, and Y. Zhang, “Field-Agnostic SNARKs from Expand-Accumulate Codes,” in *CRYPTO*, 2024, pp. 276–307.
- [33] I. Damgård, Y. Ishai, M. Krøigaard, J. B. Nielsen, and A. D. Smith, “Scalable multiparty computation with nearly optimal work and resilience,” in *CRYPTO*, 2008, pp. 241–261.
- [34] N. Ron-Zewi and M. Weiss, “Zero-knowledge iops approaching witness length,” in *CRYPTO*, 2024, pp. 105–137.
- [35] E. Ben-Sasson, S. Kopparty, and S. Saraf, “Worst-case to average case reductions for the distance to a code,” in *CCC*, 2018, pp. 24:1–24:23.
- [36] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, “Fast Reed-Solomon Interactive Oracle Proofs of Proximity,” in *ICALP*, 2018, pp. 14:1–14:17.
- [37] E. Ben-Sasson, A. Chiesa, M. Riabzev, N. Spooner, M. Virza, and N. P. Ward, “Aurora: Transparent succinct arguments for R1CS,” in *EUROCRYPT*, 2019, pp. 103–128.
- [38] U. Haböck and A. Kindi, “A note on adding zero-knowledge to starks,” *IACR Cryptol. ePrint Arch.*, 2024.
- [39] R. Canetti, Y. Chen, J. Holmgren, A. Lombardi, G. N. Rothblum, and R. D. Rothblum, “Fiat-shamir from simpler assumptions,” *IACR Cryptol. ePrint Arch.*, 2018.
- [40] E. Ben-Sasson, A. Chiesa, and N. Spooner, “Interactive oracle proofs,” in *TCC*, 2016, pp. 31–60.
- [41] S. T. V. Setty, “Spartan: Efficient and general-purpose zksnarks without trusted setup,” in *CRYPTO*, 2020, pp. 704–737.
- [42] J. Thaler, “Time-optimal interactive proofs for circuit evaluation,” in *CRYPTO*, 2013, pp. 71–89.
- [43] C. Pappas and D. Papadopoulos, “Sparrow: Space-efficient zksnark for data-parallel circuits and applications to zero-knowledge decision trees,” in *ACM CCS*, 2024, pp. 3110–3124.
- [44] V. Goyal, A. Polychroniadou, and Y. Song, “Sharing transformation and dishonest majority MPC with packed secret sharing,” in *CRYPTO*, 2022, pp. 3–32.

- [45] S. Garg, A. Goel, A. Jain, B. Roberts, and S. Sekar, “Malicious security in collaborative zk-snarks: More than meets the eye,” in *CRYPTO*, 2025, pp. 396–428.
- [46] “Code and detailed proofs,” <https://github.com/ChristodoulosPappas/Code-Based-Scalable-coSNARKs>.
- [47] B. Schoenmakers, M. Veeningen, and N. de Vreede, “Trinocchio: Privacy-preserving outsourcing by distributed verifiable computation,” in *ACNS*, 2016, pp. 346–366.
- [48] M. Veeningen, “Pinocchio-based adaptive zk-snarks and secure/correct adaptive function evaluation,” in *AFRICACRYPT*, 2017, pp. 21–39.
- [49] B. Parno, J. Howell, C. Gentry, and M. Raykova, “Pinocchio: Nearly practical verifiable computation,” in *IEEE SP*, 2013, pp. 238–252.
- [50] C. Baum, I. Damgård, and C. Orlandi, “Publicly auditable secure multi-party computation,” in *SCN*, 2014, pp. 175–196.
- [51] S. Ling, C. N. Ngo, K. H. Tang, and H. Wang, “Publicly auditable garbled circuit,” *ePrint Arch.*, 2025.
- [52] Y. Li, J. Zhou, X. Chen, J. Zhang, and R. Li, “Pampc: Publicly auditable multi-party computation with cheater detection,” in *Frontiers in Cyber Security*, 2024.
- [53] S. Kanjalkar, Y. Zhang, S. Gandlur, and A. Miller, “Publicly auditable mpc-as-a-service with succinct verification and universal setup,” in *IEEE EuroS&P*, 2021, pp. 386–411.
- [54] “Robust local testability of tensor products of ldpc codes,” in *International Workshop on Approximation Algorithms for Combinatorial Optimization*, 2006, pp. 304–315.
- [55] I. Damgård and J. B. Nielsen, “Scalable and unconditionally secure multiparty computation,” in *CRYPTO*, 2007, pp. 572–590.
- [56] S. Goldwasser, S. Micali, and C. Rackoff, “The knowledge complexity of interactive proof-systems (extended abstract),” in *ACM STOC*, 1985, pp. 291–304.
- [57] L. Babai, L. Fortnow, L. A. Levin, and M. Szegedy, “Checking computations in polylogarithmic time,” in *ACM STOC 91*, pp. 21–32.
- [58] B. Bünz, B. Fisch, and A. Szepieniec, “Transparent snarks from DARK compilers,” in *EUROCRYPT*, 2020, pp. 677–706.
- [59] A. Chiesa, P. Manohar, and N. Spooner, “Succinct arguments in the quantum random oracle model,” in *TCC*, 2019, pp. 1–29.
- [60] M. Ben-Or, S. Goldwasser, and A. Wigderson, “Completeness theorems for non-cryptographic fault-tolerant distributed computation (extended abstract),” in *ACM STOC*, 1988, pp. 1–10.
- [61] G. Asharov and Y. Lindell, “A full proof of the BGW protocol for perfectly secure multiparty computation,” *J. Cryptol.*, pp. 58–151, 2017.
- [62] I. Damgård, Y. Ishai, and M. Krøigaard, “Perfectly secure multiparty computation and the computational overhead of cryptography,” in *EUROCRYPT*, 2010, pp. 445–465.

- [63] S. Garg, A. Jain, P. Mukherjee, and M. Wang, “Scalable multiparty computation from non-linear secret sharing,” *IACR Cryptol. ePrint Arch.*, p. 1007, 2025.
- [64] C. Pappas and D. Papadopoulos, “Hobbit:Space-EfficientzkSNARK with optimal prover time,” in *USENIX Security*, 2025, pp. 4917–4936.
- [65] D. Augot, S. Bordage, and J. Nardi, “Efficient multivariate low-degree tests via interactive oracle proofs of proximity for polynomial codes,” *Designs, Codes and Cryptography*, pp. 1111–1151, 2023.
- [66] Y. Guo, X. Liu, K. Huang, W. Qu, T. Tao, and J. Zhang, “DeepFold: Efficient Multilinear Polynomial Commitment from Reed-Solomon Code and Its Application to Zero-knowledge Proofs,” *IACR Cryptol. ePrint Arch.*, p. 1595.
- [67] A. R. Block, A. Garreta, J. Katz, J. Thaler, P. R. Tiwari, and M. Zajac, “Fiat-shamir security of FRI and related snarks,” in *ASIACRYPT*, 2023, pp. 3–40.
- [68] B. E. Diamond, “Zero-knowledge polynomial commitment in binary fields,” *IACR ePrint*, p. 1015, 2025.
- [69] J. Zhang, T. Xie, Y. Zhang, and D. Song, “Transparent polynomial delegation and its applications to zero knowledge proof,” in *IEEE SP*, 2020, pp. 859–876.
- [70] T. Liu, X. Xie, and Y. Zhang, “zkcn: Zero knowledge proofs for convolutional neural network predictions and accuracy,” in *CCS*, 2021, pp. 2968–2985.
- [71] T. Xie, J. Zhang, Y. Zhang, C. Papamanthou, and D. Song, “Libra: Succinct zero-knowledge proofs with optimal prover computation,” in *CRYPTO*, 2019, pp. 733–764.
- [72] T. Xie, J. Zhang, Z. Cheng, F. Zhang, Y. Zhang, Y. Jia, D. Boneh, and D. Song, “zkbridge: Trustless cross-chain bridges made practical,” in *CCS*, 2022, pp. 3003–3017.
- [73] “The Implementation of Blake3.” [Online]. Available: <https://github.com/BLAKE3-team/BLAKE3>
- [74] “Mpi,” <https://www.open-mpi.org/>.
- [75] M. Ito, A. Saito, and T. Nishizeki, “Secret sharing scheme realizing general access structure,” *Electronics and Communications in Japan*, vol. 72, no. 9, pp. 56–64, 1989.
- [76] A. Chiesa and E. Yogev, “Building cryptographic proofs from hash functions (2024),” *URL: <https://github.com/hash-based-snargs-book>*.

A Proof of Theorem 1

We first provide a performance analysis of **Construction 2**. Next we show that Π^{Enc} is complete and finally we show that our collaborative code satisfies (t, l) -zero knowledge, where $l = (m' - m)$.

Performance Analysis. Throughout our analysis we assume that $k' = \Theta(N)$, which should be the case for the application we focus. With that in mind, observe that from the analysis of the protocol of [62], the instantiation of $\mathcal{F}_{rand}$ requires one round of interaction, it has total communication complexity of $\mathcal{O}(Nl) = \mathcal{O}(k'l)$ and per-party time complexity of $\mathcal{O}(\frac{k'l}{N})$.

For the rest of this part we will focus on the online complexity of the protocol. For this, observe that at step 1, each party has to perform $\frac{m}{N}$ FFTs of size at most N requiring $\mathcal{O}(\frac{m}{N} \cdot (N \log N)) = \mathcal{O}(m' \log k')$ field operations. At step 3, each party has to perform one iFFT and an FFT of size at most $\mathcal{O}(M) = \mathcal{O}(m')$, which requires $\mathcal{O}(m' \log m')$ field operations. Hence, the total complexity of each party is $\mathcal{O}(m' \log(k'm')) = \mathcal{O}(\frac{m'k'}{N} \log(k'm'))$. Finally, observe that the protocol needs only one round of communication (step 2) in which the total communication complexity is $\mathcal{O}(N \cdot (N \frac{m'}{N})) = \mathcal{O}(Nm') = \mathcal{O}(k'm')$.

Remark 3. *In many applications, especially in collaborative zkSNARKs such as private delegation of proof generation, each party $\mathcal{P}_i$ will already be given the i -th row $\mathbf{X}'$ (e.g., provided by the delegator who must secret share its extended witness). In these cases, steps 1 and 2 of **Construction 2** are not needed, which leads to no communication overheads and a per-party time complexity of $\mathcal{O}(\frac{k'm'}{N} \log m')$.*

Completeness. Next, we show that for any $\mathbf{X}$ each party $\mathcal{P}_i$ will compute the i -th row of the $\text{RS}^{\otimes 2}$ codeword of $\mathbf{X}$. In particular, we will show that the row $\mathbf{C}[i, :]$ computed at step 3 of **Construction 2** is equal to $[p(\omega_1^i, \omega_2^1), \dots, p(\omega_1^i, \omega_2^M)]$, where $p : \mathbb{F}^2 \rightarrow \mathbb{F}$ the bivariate polynomial such that $p(z_1^i, z_2^j) = \tilde{\mathbf{X}}[i, j]$. First, due to the completeness of the $\mathcal{F}_{rand}$ functionality, the vector $\mathbf{R}'[i, :]$ that $\mathcal{P}_i$ holds at the end of the pre-processing phase must be equal to $[p_r^{(1)}(\omega_1^i), \dots, p_r^{(m'-m)}(\omega_1^i)]$. Next, by the definition of $\mathbf{X}$, it must hold that $c^{(i,j)}(X) = c^{((i-1)\frac{m}{N}+j)}(X)$, where $c^{(i,j)}(X)$ the polynomial that interpolates the j -th column of $\tilde{\mathbf{X}}_i$. Furthermore, at the end of step 2, each party $\mathcal{P}_i$ will hold the i -th row of $\mathbf{X}'_j$ of any other party $\mathcal{P}_j$. The concatenation of those rows equals to $[c^{(1,1)}(\omega_1^i), \dots, c^{(N,m)}(\omega_1^i)] = [c^{(1)}(\omega_1^i), \dots, c^{(m)}(\omega_1^i)]$. By adding $\mathbf{R}[i, :]$ we get $\mathbf{X}'[i, :]$ as defined in the main body. Since $\mathbf{X}'[i, :]$ is correctly computed, it follows that $\mathbf{C}[i, :]$ will also be correct due to the correctness of the iFFT and FFT algorithms.

(t, l) -zero knowledge. We first construct a simulator $\mathcal{S}$ that simulates the view of an $\mathcal{A}$ that controls up to t corrupted parties $\mathcal{P}^* = \{\mathcal{P}_{j_1}, \dots, \mathcal{P}_{j_t}\}$ and the verifier $\mathcal{V}$. The simulator is given access to the random tape of $\mathcal{A}$ (and consequently the index set $\mathcal{I}$), the index set of corrupted parties $\mathcal{J} = \{j_1, \dots, j_t\} \in [N]^t$, as well as their inputs $(\mathbf{X}_{j_1}, \dots, \mathbf{X}_{j_t})$ and works as follows:

1. Let $\mathcal{I}_c = \{(\omega_1^j, z_2^i)\}_{j \in \mathcal{J}, i \in [m'] \setminus \mathcal{J}_{col}} \cup \{\omega_1^j, \omega_2^i\}_{j \in \mathcal{J}, i \in [M]}$, be the set of points the evaluations of which are obtained by the adversary at the end of the Π^{Enc} protocol (excluding what $\mathcal{A}$ already knows at the beginning of the protocol). Note that $\mathcal{J}_{col}$ are the indexes of the columns of $\mathbf{X}$, the corrupted parties hold at the beginning of the protocol. Furthermore, let $\mathcal{Y}_{\mathcal{I}_c \cup \mathcal{I}}$ be the set of all “valid” values that $\mathbf{X}'$ and $\mathbf{C}$ can have at these points. In practice, $\mathcal{Y}_{\mathcal{I}_c \cup \mathcal{I}}$ is the set of evaluations of all k', m' -degree bivariate polynomials p at $\mathcal{I}_c \cup \mathcal{I}$ conditioned that they evaluate at $(\tilde{\mathbf{X}}_{j_1}, \dots, \tilde{\mathbf{X}}_{j_t})$ on the points $\{z_1^i, z_2^j\}_{i \in [k'], j \in \mathcal{J}_{col}}$. The simulator samples an element of $\mathbf{Y} \xleftarrow{\$} \mathcal{Y}_{\mathcal{I}_c \cup \mathcal{I}}$ uniformly at random in the following way:

- (a) $\mathcal{S}$ starts by computing the answers $\mathbf{c}$ to the queries $\mathcal{I}$. This is done as follows:
 - i. Parse $\mathcal{I}$ as a multi-set consisting of sets of queries that target the same column of $\mathbf{C}$, that is, $\mathcal{I}^{(j)} = \{(\omega_1^{i_1}, \omega_2^j), \dots, (\omega_1^{i_z}, \omega_2^j)\}$ for all $\{i_1, \dots, i_z\} \in \mathcal{I}_1^{(j)}$ and all possible columns $j \in \mathcal{I}_2$. Note that the number of different queried columns is at most l , namely, $|\mathcal{I}_2| \leq l$.
 - ii. Next, iterate through all $\mathcal{I}^{(j)}$ and sample a random “valid” column $\mathbf{C}[:, i]$ that it interpolates to a polynomial of degree k' . Store the points that correspond to the queries $\mathcal{I}^{(j)}$ and $\mathbf{C}[j, i]$, for all $j \in \mathcal{J}$ and discard all other points.

- (b) Sample uniformly at random the rows $\mathbf{X}'_i[j, :]$ sent by the honest parties to $\mathcal{J}$. Furthermore, if $|\mathcal{I}_2| < l$, sample $\mathbf{R}'[j, i]$, for all $j \in \mathcal{J}, i \in [l - |\mathcal{I}_2|]$ uniformly at random. Next, use: (1) $\mathbf{X}'_i[j, :]$, for all $i \in [N], j \in \mathcal{J}$, (2) the values $\{\mathbf{C}[j, i]\}_{j \in \mathcal{J}, i \in \mathcal{I}^{(2)}}$ computed at step *a.ii* and (3) the sampled points of $\mathbf{R}'$ (if any) to interpolate the m' -degree polynomials $r^{(j)}(Y)$, for all $j \in \mathcal{J}$. Finally use these polynomials to compute the rest of the points of $\mathbf{R}'[j, :]$.
2. Provided $\mathbf{c}, \mathbf{X}'_i[j, :]$, for all $i \in [N] \setminus \mathcal{J}$ and $j \in \mathcal{J}$ and $\mathbf{R}'[j, :]$, $j \in \mathcal{J}$, $\mathcal{S}$ simulates **Construction 2** as follows:
- (a) $\mathcal{S}$ simulates $\mathcal{F}_{rand}$ functionality by sending $\mathbf{R}[j_i, :]$ for all $j_i \in \mathcal{J}$ to the adversary. Finally, $\mathcal{S}$ sets $V_1 = \{\mathbf{R}'[j, :]\}_{j \in \mathcal{J}}$.
- (b) $\mathcal{S}$ simulates step 2 by sending $\mathbf{X}'_i[j, :]$, for all $i \in [N] \setminus \mathcal{J}$ to the adversary and setting $V_2 = \{\mathbf{X}'_i[j, :]\}_{i \in [N] \setminus \mathcal{J}, j \in \mathcal{J}}$.
- (c) Finally, from $\mathbf{Y}$ the simulator retrieves and sends to the adversary the reply queries $\mathbf{c}$. Finally it sets $V_3 = \{\mathbf{Y}\}$.

What remains is to show that $View_{\mathcal{J}}^{\Pi^{Enc}}(\mathbf{x})$, parsed as $\{V_1^*, V_2^*, V_3^*\}$ is indistinguishable to $S(\mathbf{x}_{j_1}, \dots, \mathbf{x}_{j_t}, \mathcal{I}) = \{V_1, V_2, V_3\}$, namely, show that $V_i \equiv V_i^*$, for $i = \{1, 2, 3\}$. The indistinguishability between the first two views comes directly from [15] as long as $t \leq k' - k + 1$. Consequently, we will focus on showing that $V_3 \equiv V_3^*$. As mentioned earlier, our analysis will focus on the case where $t = k' - k + 1$, but we can easily generalize our proof for $t < k' - k + 1$.

We start with the observation that the simulator samples $\mathbf{Y} = V_3$ uniformly at random from the set of valid answers $\mathcal{Y}_{\mathcal{I} \cup \mathcal{I}_c}$. Consequently, we need to show that V_3^* also follows a uniform distribution over $\mathcal{Y}_{\mathcal{I} \cup \mathcal{I}_c}$. In other words, we need to show that for any global input $\mathbf{X}$, and a k', m' -degree polynomial p that passes through $\{\mathbf{X}_j\}_{j \in [N] \setminus \mathcal{J}}$ and $\{\tilde{\mathbf{X}}_j\}_{j \in \mathcal{J}}$ it holds that:

$$Pr[p|_{\mathcal{I} \cup \mathcal{I}_c} = V_3] = \frac{1}{|\mathcal{Y}_{\mathcal{I} \cup \mathcal{I}_c}|}$$

To achieve this, we first prove the following lemma:

Lemma 2. *Let $\mathcal{I}'$ be a multiset, such that $\mathcal{I} \subset \mathcal{I}'$ which is defined as follows: If a set $\mathcal{I}^{(j)} \in \mathcal{I}$ has more than k elements, then $\mathcal{I}'^{(j)} = \mathcal{I}^{(j)}$ otherwise fill $\mathcal{I}'^{(j)}$ with indexes at the same column so that $|\mathcal{I}'^{(j)}| = k$. Finally, if $|\mathcal{I}_2| < l$ (where $\mathcal{I}_2$ the unique column indexes of the query set $\mathcal{I}$), fill $\mathcal{I}'$ with $l - |\mathcal{I}_2|$, k -sized sets of indexes, such that all sets $\mathcal{I}'^{(j)}$ contain indexes over a distinct column. For any $\mathbf{Y} \in \mathcal{Y}_{\mathcal{I}_c \cup \mathcal{I}'}$, there exists a unique k', m' -degree bivariate polynomial $p : \mathbb{F}^2 \rightarrow \mathbb{F}$ that passes through $\{\mathbf{X}_j\}_{j \in [N] \setminus \mathcal{J}}$ and $\{\tilde{\mathbf{X}}_j\}_{j \in \mathcal{J}}$ and $\mathbf{Y}$.*

Proof. Observe that given $\{\mathbf{X}_j\}_{j \in [N] \setminus \mathcal{J}}$ and $\{\tilde{\mathbf{X}}_j\}_{j \in \mathcal{J}}$ and $\mathbf{Y}$ we can interpolate the polynomial $p : \mathbb{F}^2 \rightarrow \mathbb{F}$ as follows: First, organize $\mathcal{I}' \cup \mathcal{I}_c$ into sets of indexes that belong to the same column. Then, filter from $\mathcal{I}' \cup \mathcal{I}_c$ all such sets with size smaller than k' . By definition of $\mathcal{I}_c$ and $\mathcal{I}'$ we remain with $m - t \frac{m}{N} + l = m' - t \frac{m}{N}$ such sets. Then, we use them to interpolate the k' -degree polynomials $c^{(j)}(X)$, where $j \in \mathcal{I}'^{(2)}$. Next, using $\{\tilde{\mathbf{X}}_j\}_{j \in \mathcal{J}}$ interpolate the remaining $t \frac{m}{N}$ polynomials $c^{(j)}(X)$, $j \in \mathcal{J}_{col}$. Finally, since the number of “partial” polynomials is m' , we can interpolate a m' -degree polynomial $q_i(Y)$ from the coefficients of each monomial of these polynomials thus constructing $p(X, Y) = \sum_{i \in [k']} X^i q_i(Y)$. $\square$

Provided Lemma 2, it is not hard to see that the probability, we obtain $\mathbf{Y} \in \mathcal{Y}_{\mathcal{I}_c \cup \mathcal{I}'}$ from a polynomial p that passes through $\{\mathbf{X}_j\}_{j \in [N] \setminus \mathcal{J}}$ and $\{\tilde{\mathbf{X}}_j\}_{j \in \mathcal{J}}$ is exactly:

$$\Pr[p|_{\mathcal{I}_c \cup \mathcal{I}'} = \mathbf{Y}] = \frac{1}{|\mathcal{Y}_{\mathcal{I}_c \cup \mathcal{I}'}|} = \frac{1}{|\mathbb{F}|^{m'k' - mk - \frac{tm}{N}(k' - k)}}$$

Furthermore, using the definition $\mathcal{I}'$ from Lemma 2, we can make the following observations: (1) $\mathcal{I} \cup \mathcal{I}_c \subset \mathcal{I}' \cup \mathcal{I}_c$ and (2) $\mathcal{Y}_{\mathcal{I}' \cup \mathcal{I}_c} = \mathcal{Y}_{\mathcal{I} \cup \mathcal{I}_c} \times \mathcal{Y}_{\mathcal{I}' \setminus \mathcal{I}}$ and that is because $\mathcal{I}' \cap \mathcal{I}_c = \emptyset$ and $\mathcal{Y}_{\mathcal{I}'} = \mathcal{Y}_{\mathcal{I}} \times \mathcal{Y}_{\mathcal{I}' \setminus \mathcal{I}}$. Based on these observations, we have that:

$$\begin{aligned} \Pr[p|_{\mathcal{I}' \cup \mathcal{I}_c} = \mathbf{Y}] &= \\ \Pr[(p|_{\mathcal{I} \cup \mathcal{I}_c} = \mathbf{Y}^{(1)}) \cup (p|_{(\mathcal{I}' \setminus \mathcal{I})} = \mathbf{Y}^{(2)})] &= \\ \Pr[(p|_{\mathcal{I} \cup \mathcal{I}_c} = \mathbf{Y}^{(1)})] \cdot \frac{1}{|\mathcal{Y}_{(\mathcal{I}' \setminus \mathcal{I})}|} &= \\ \Pr[(p|_{\mathcal{I} \cup \mathcal{I}_c} = \mathbf{Y}^{(1)})] \cdot \frac{1}{|\mathbb{F}|^{|\mathcal{I}' \setminus \mathcal{I}|}} \end{aligned}$$

Consequently:

$$\begin{aligned} \Pr[p|_{\mathcal{I} \cup \mathcal{I}_c} = \mathbf{Y}^{(1)}] &= \Pr[p|_{\mathcal{I}' \cup \mathcal{I}_c} = \mathbf{Y}] \cdot |\mathbb{F}|^{|\mathcal{I}' \setminus \mathcal{I}|} = \\ \frac{1}{|\mathbb{F}|^{m'k' - mk - \frac{tm}{N}(k' - k) - |\mathcal{I}' \setminus \mathcal{I}|}} \end{aligned}$$

B On the Foldability of $\text{RS}^{\otimes 2}$ Codes

Before showing the foldability of our tensorcode, we first introduce the following important lemma:

Lemma 3 (Ames et al. [30], Roth and Zemor). *Let $e < d/3$ where d the minimum distance of a code $\mathcal{C}$ over a finite field $\mathbb{F}$. If $\text{Dist}(\mathcal{U}, \mathcal{C}^m) > e$, then for a random $\mathbf{u}$ in the row-span of $\mathcal{U}$, it holds that $\Pr[\text{Dist}(\mathbf{u}, \mathcal{C}) < e] \leq (e + 1)/|\mathbb{F}|$.*

In this section, we show that $\text{RS}^{\otimes 2}$ are *foldable*. In more detail, and similarly to standard RS codes, we can define a folding function *Fold* that takes as input an element $a \in \mathbb{F}$ and a function $f : \mathcal{L}'_1 \times \mathcal{L}'_2^{(i)} \rightarrow \mathbb{F}$ and “hashes” it into a function of half its size. More precisely, $\text{Fold}(a, f) : \mathcal{L}'_1 \times \mathcal{L}'_2'^{(i+1)} \rightarrow \mathbb{F}$, where $|\mathcal{L}'_2'^{(i+1)}| = |\mathcal{L}'_2^{(i)}|/2$. For any $a \in \mathbb{F}$, and $y \in \mathcal{L}'_2'^{(i+1)}$ and $y', -y' \in \mathcal{L}'_2^{(i)}$ such that $y = y'^2 = (-y')^{2^4}$, we define $\text{Fold}(a, f)$ as follows:

$$\begin{aligned} \text{Fold}(a, f)(x, y) &= \frac{f(x, y') + f(x, -y')}{2} + \\ a \cdot \frac{f(x, y') - f(x, -y')}{2y'} \end{aligned}$$

Based on the above function, we make the following Lemma that will be necessary for the completeness and soundness of our collaborative IOPP:

Lemma 4. *For the Fold function of the above equation the following holds:*

1. *If $f \in \text{RS}^{\otimes 2}[\mathbb{F}, \mathcal{L}'_1 \times \mathcal{L}'_2^{(i)}, (k, k', N), (m, m', M)]$, then, for any $a \in \mathbb{F}$, $\text{Fold}(a, f) \in \text{RS}^{\otimes 2}[\mathbb{F}, \mathcal{L}_1 \times \mathcal{L}_2'^{(i+1)}, (k, k', N), (\frac{m}{2}, \frac{m'}{2}, \frac{M}{2})]$*

⁴In practice, $y' = y$ and $-y' = y + \frac{|\mathcal{L}_2^{(i)}|}{2}$

2. Let $\delta < \frac{\Delta}{3}$, where Δ the relative distance of the $RS^{\otimes 2}$ code. If $\Delta(f, \mathbf{C}_i) > \delta$ where $\mathbf{C}_i \in RS^{\otimes 2}[\mathbb{F}, \mathcal{L}_1 \times \mathcal{L}_2^{(i)}, (k, k', N), (m, m', M)]$, then for a random $a \in \mathbb{F}$ it holds that:

$$\Pr[\Delta(\text{Fold}(a, f), \mathbf{C}_{i+1}) \leq \delta] \leq \frac{n}{|\mathbb{F}|}$$

Where $n = |\mathcal{L}'_1 \times \mathcal{L}'_2^{(i+1)}|$ $\mathbf{C}_{i+1} \in RS^{\otimes 2}[\mathbb{F}, \mathcal{L}'_1 \times \mathcal{L}'_2^{(i+1)}, (k, k', N), (\frac{m}{2}, \frac{m'}{2}, \frac{M}{2})]$

Proof. To prove the first claim, let $p : \mathbb{F}^2 \rightarrow \mathbb{F}$ be the polynomial of individual degrees k' and m' respectively, that interpolates f . Furthermore, let $p_o, p_e : \mathbb{F}^2 \rightarrow \mathbb{F}$ be the polynomials from p that contain even and odd powers respectively on the Y variable. Similarly with the univariate case, we can define the two polynomials as $p_o(X, Y) = \frac{p(X, Y) + p(X, -Y)}{2}$ and $p_e(X, Y) = \frac{p(X, Y) - p(X, -Y)}{2Y}$. Next, we define the polynomial p^* of individual degrees k' and $\frac{m'}{2}$ respectively such that $p^*(X, Y') = p'_o(X, Y) + ap'_e(X, Y) = p_o(X, Y^2) + ap_e(X, Y^2)$. It is not hard to see that $p^*|_{\mathcal{L}'_1 \times \mathcal{L}'_2^{(i+1)}} = \text{Fold}(f, a)$, and hence $\text{Fold}(f, a) \in RS^{\otimes 2}[\mathbb{F}, \mathcal{L}'_1 \times \mathcal{L}'_2^{(i+1)}, (k, k', N), (\frac{m}{2}, \frac{m'}{2}, \frac{M}{2})]$. That is, because for each $(x, y) \in \mathcal{L}_1 \times \mathcal{L}_2^{(i+1)}$, $p^*(x, y) = p'_o(x, y) + ap'_e(x, y) = p_o(x, y') + ap_e(x, y') = \frac{f(x, y') - f(x, -y')}{2} + a \frac{f(x, y') - f(x, -y')}{2y'} = \text{Fold}(a, f)(x, y)$

To prove the second property, we use Lemma 3. First, let $\mathbf{u} = \text{Fold}(0, f)$, $\mathbf{u}^* = \text{Fold}(1, f) - \text{Fold}(0, f)$. Note that $\text{Fold}(a, f) = \mathbf{u} + a\mathbf{u}^*$. In addition, let $|\mathcal{L}'_1 \times \mathcal{L}'_2^{(i+1)}| = n$. Assume towards contradiction that $\Pr[\Delta(\text{Fold}(a, f), \mathbf{C}_{i+1}) \leq \delta] = \Pr[\Delta(\mathbf{u} + a\mathbf{u}^*, \mathbf{C}_{i+1}) \leq \delta] > \frac{n}{|\mathbb{F}|}$.

By Lemma 3, we have that $\Delta(\{\mathbf{u}, \mathbf{u}^*\}, \mathbf{C}_{i+1}) \leq n\delta$. This implies that there should exist $\mathbf{w}, \mathbf{w}^* \in RS^{\otimes 2}[\mathbb{F}, \mathcal{L}'_1 \times \mathcal{L}'_2^{(i+1)}, (k, k', N), (\frac{m}{2}, \frac{m'}{2}, \frac{M}{2})]$, and a set of indexes $\mathcal{T}$, such that $|\mathcal{T}| \geq n(1 - \delta)$ and $\mathbf{w}|_{\mathcal{T}} = \mathbf{u}|_{\mathcal{T}}$ and $\mathbf{w}^*|_{\mathcal{T}} = \mathbf{u}^*|_{\mathcal{T}}$. Let $p_w, p_{w^*} : \mathbb{F}^2 \rightarrow \mathbb{F}$ the bivariate polynomials obtained by interpolating $\mathbf{w}, \mathbf{w}^*$. Note that these polynomials have individual degree of at most k' and $m'/2$ respectively. Furthermore, we define the polynomials $\hat{p}(X, W, Y) = p_w(X, Y) + Wp_{w^*}(X, Y)$ and $p^*(X, Y) = \hat{p}(X, Y, Y^2)$. Note that the latter has individual degree of at most k' and m' .

Given those polynomials, we will show that f agrees with $p^*(X, Y)$ at all points (x, y') , $(x, -y')$, where $y = y'^2 = (-y')^2$, and $(x, y) \in \mathcal{T}$. For this, first observe that $\hat{p}(x, 0, y) = \text{Fold}(0, f)(x, y)$ and $\hat{p}(x, 1, y) = \text{Fold}(1, f)(x, y)$. Since both $\hat{p}(x, W, y)$ and $\text{Fold}(W, f)(x, y)$ are lines and agree at two points, it will hold that $\hat{p}(x, W, y) = \text{Fold}(W, f)(x, y)$. Consequently $p^*(x, y') = \hat{p}(x, y', y) = \text{Fold}(y', f) = f(x, y')$. Likewise, we argue about $(x, -y')$. Let $p^*|_{\mathcal{L}'_1 \times \mathcal{L}'_2^{(i+1)}} = \mathbf{C}_i$, then $\Delta(f, \mathbf{C}_i) \leq 1 - \frac{2|\mathcal{T}|}{2n} \leq \delta$ which contradicts our assumption. Consequently, $\Delta(f, \mathbf{C}_i) > \delta$. $\square$

C Proof of Theorem 2

Proof. Proving adaptive (knowledge) soundness against both standard and quantum adversaries is straightforward. In particular, assume a coIOP with round-by-round (knowledge) soundness errors $\epsilon_{rbr}, \kappa_{rbr}$. Using that coIOP, someone could build an IOP with exactly the same errors simply by emulating each individual prover $\mathcal{P}_i$ and their communication internally. Note that the non-interactive proof of that IOP (using the BCS transformation) would be identical to that of the coIOP using our transformation. Based on that, and from our initial hypothesis the non-interactive proof of the IOP would have (knowledge) soundness errors of $\epsilon_{fs} > Q\epsilon_{rbr} + 3(Q^2 + 1)/2^\lambda$ and $\kappa_{fs} > Q\kappa_{rbr} + 3(Q^2 + 1)/2^\lambda$ which contradicts the original BCS theorem [40]. We follow the same argument with quantum adversaries.

Having established that, we can now focus on t -zero knowledge. Initially, we consider the simplest case of honest verifier and prover t -zero knowledge and later show how to extend our proof when dealing with malicious provers.

As a first step, we construct a simulator $\mathcal{S}$ of the non-interactive protocol using the simulator $\mathcal{S}_{coIOP}$ of the coIOP. Our simulator works as follows:

1. In a pre-processing step, the simulator invokes the (honest-verifier) simulator $\mathcal{S}_{coIOP}$ (locally) to obtain a view $(\mathbf{st}_{j_1}, \dots, \mathbf{st}_{j_t}, \pi = \{(r_1, \dots, r_k), \mathbf{q}, \mathcal{I}\})$.
2. Then, it interacts with the adversary, to produce the simulated view. For this our simulator works in the same way as with $\mathcal{S}_{coIOP}$ but also simulates the additional rounds of the collaborative variant of BCS. In more detail, for each round $j \in [k]$, $\mathcal{S}$ works as follows:
 - (a) For each honest prover $\mathcal{P}_i$, run the simulator of the Merkle tree $rt_i^{(j)}, \mathbf{pr}_i^{(j)} \leftarrow \mathcal{S}^{MT}(\mathbf{q}_i^{(j)}, \mathcal{I}_i^{(j)})$, where $\mathbf{q}_i^{(j)}$ the replies provided by $\mathcal{P}_i$ on the j -th round.
 - (b) Receive $\{rt_i\}_{i \in \mathcal{J}}$ and $\{\tau_i^{(j)}\}_{i \in \mathcal{J}}$ from the adversary.
 - (c) Set the answer-query of the (programmable) oracle $\mathcal{H}'$ to $(r_{j+1}, (rt^{(j)} |_{\tau^{(j)}}))$, where $rt^{(j)} = MT^{Commit}(rt_1^{(j)}, \dots, rt_N^{(j)})$.
 - (d) Send $\{rt_i^{(j)}\}_{i \in [N] \setminus \mathcal{J}}$ and also the salt values $\{\tau_i\}_{i \in [N] \setminus \mathcal{J}}$ to the adversary.
3. Output $(\mathbf{st}_{j_1}, \dots, \mathbf{st}_{j_t}, (rt^{(0)}, \dots, rt^{(k)}), \mathbf{pr}, \mathbf{q}, \mathcal{I})$

Furthermore, we construct two hybrid simulators that work as follows:

- $\mathcal{S}^{H_1}$: This simulator takes as input the entire witness $\mathbf{w}$ and produces a real view $(\mathbf{st}_{j_1}^*, \dots, \mathbf{st}_{j_t}^*, \pi^* = \{(r_1^*, \dots, r_k^*), \mathbf{q}^*, \mathcal{I}^*\})$.
- $\mathcal{S}^{H_2}$: This simulator works exactly like $\mathcal{S}^{H_1}$ but it also replaces the simulator of the Merkle tree with the real algorithms.

Observe that the views $View_{\mathcal{A}}^{\mathcal{S}}$ and $View_{\mathcal{A}}^{H_1}$ differ only in the way the simulator samples the coIOP view, that is, in the first case that view is simulated while in the second case that view is real. By definition, the statistical error should be at most z_{coIOP} . Next, observe that the views $View_{\mathcal{A}}^{H_1}$ and $View_{\mathcal{A}}^{H_2}$ change only on that in the first one the simulator uses the simulator of a Merkle tree while in the second one the simulator uses the actual Merkle tree algorithms. By the binding properties of the Merkle tree, these distributions are bounded by z_{MT} . Finally, $View_{\mathcal{A}}^{H_2}$ and $View_{\mathcal{A}}^R$ differ only on that $\mathcal{S}^{H_2}$ programs the oracle while in the real protocol the random oracle is set fixed. While the distribution of the query-answer pairs is the same in both oracles, the prover can distinguish between both views if it queries the oracle of $\mathcal{S}^{H_2}$ on an index before it is programmed. Due to the salt values, this can happen only with probability $\frac{1}{2^{N\lambda/(N-t)}}$ [76].

Extending to Malicious-Prover Case. We can extend our proof for the case that our coIOP has maliciously secure t -zero knowledge as follows. $\mathcal{S}$ works similarly to the simulator of the honest-prover case but step 2 contains an additional step (e) in which the simulator sends $\{rt^{(j)}\}_{i \in [N] \setminus \mathcal{J}}$ and waits until it gets the random challenges from the adversary. If there is any rt' provided by the adversary such that $rt' \neq rt$ then the simulator aborts. Finally, note that the zero knowledge error from the salt values decreases from $\frac{1}{2^{N\lambda/(N-t)}}$ to $\frac{1}{2^\lambda}$ since only $(N - t)$ such values are guaranteed to be sampled uniformly at random. $\square$

D Proof of Theorem 3

D.1 Performance Analysis

We start by analyzing the performance of the online phase of our coIOPP. Let $L = km$ be the size of $\mathbf{X}$. It is not hard to see that the total number of communication rounds between the verifier is

$k_V(L) = \mathcal{O}(\log L)$ while the number of communication rounds only between the provers is $k_P(L) = 1$ which happens at step 1 of the commit protocol. Next, the total proof size $l(L) = |\mathbf{C}^{(\log M - c - 1)}| + \sum_{i \in [\log M - c - 1]} |\mathbf{C}^{(i)}| + |\mathbf{C}_r^{(i)}| \leq 4|\mathbf{C}^{(1)}| = \mathcal{O}(L)$, and $l_i(L) = \mathcal{O}(\frac{L}{N})$. The number of queries and the verification time are both $q(L, N) = vt(L, N) = \mathcal{O}(l \log L + N^{2^c})$. The $l \log L$ factor derives from the first step of the query algorithm while the N^{2^c} factor comes from the second one. Note that after applying the optimization, both complexities can be reduced to $q(L, N) = vt(L, N) = \mathcal{O}(l \log L + N)$.

Finally, the per-prover time complexity is $pt_i(L, N) = \mathcal{O}(\frac{L}{N} \log L)$ and the total communication complexity is $pc(L = km, N) = Nm \log |\mathbb{F}|$ bits. In practice, prover time is dominated by step 1 while in the rest of the protocol (and in particular the commitment phase) the per-prover time complexity is $\mathcal{O}(l_i(L)) = \mathcal{O}(\frac{L}{N})$. Finally, the total communication cost comes from the Π^{Enc} algorithm.

D.2 Completeness Proof

From the completeness property of Π^{Enc} , $\mathbf{C}^{(1)}$ is a valid codeword of the message $\mathbf{X}$. Likewise, all codewords $\mathbf{C}_r^{(1)}, \dots, \mathbf{C}_r^{(\log M - c - 1)}$ are valid by the definition of $\mathcal{F}_{rand}^{Enc}$. Based on that and from Lemma 4, we get that $\mathbf{C}^{(2)}, \dots, \mathbf{C}^{(\log M - c)}$ would be valid codewords, and in particular, $\mathbf{C}^{(i)} \in \text{RS}_i^{\otimes 2}$, for all $i \in [\log M - c]$. Consequently, step 2 of the query algorithm will pass with probability 1. Finally, by the definition of the fold function for $\text{RS}^{\otimes 2}$, it is not hard to see that all queries of step 1 will pass with probability 1. That is because

$$\begin{aligned} \mathbf{C}^{(j+1)}[(x, y)] &= \text{Fold}(r_j, \mathbf{C}^{(j)} + b_j \mathbf{C}_r^{(j)})(x, y) = \\ &= \frac{\mathbf{C}^{(j)}[(x, y')] + \mathbf{C}^{(j)}[(x, -y')] + b_j(\mathbf{C}_r^{(j)}[(x, y')] + \mathbf{C}_r^{(j)}[(x, -y')])}{2} + \\ &= r_j \frac{\mathbf{C}^{(j)}[(x, y')] - \mathbf{C}^{(j)}[(x, -y')] + b_j(\mathbf{C}_r^{(j)}[(x, y')] - \mathbf{C}_r^{(j)}[(x, -y')])}{2y'} = \\ &= \text{INTERPOLATE}(a_j, \mathbf{C}^{(j)}[(x, y)] + b_j \mathbf{C}_r^{(j)}[(x, y)], \mathbf{C}^{(j)}[(x, -y)] + \\ &\quad b_j \mathbf{C}_r^{(j)}[(x, -y)]) \end{aligned}$$

Remark 4. For the next sections, we simplify our analysis by assuming that provers send $\mathbf{C}^{(\log m' - c)}$ directly to the verifier instead of further reducing it to $\mathbf{C}^{\log m'}$. Note that our proofs can be naturally extended to cover our final version.

D.3 Proof of t -Zero Knowledge

We first construct a simulator $\mathcal{S}$ that simulates the view of an adversary $\mathcal{A}$ controlling up to t corrupted provers $\mathcal{P}^* = \{\mathcal{P}_{j_1}, \dots, \mathcal{P}_{j_t}\}$ and the verifier $\mathcal{V}$. The simulator is given the indexes of the corrupted provers $\mathcal{J}$ as well as access to the random tape of $\mathcal{A}$. In particular, it holds all random challenges $\mathbf{a} = (a_1, \dots, a_{\log m' - c - 1})$, $\mathbf{b} = (b_1, \dots, b_{\log m' - c - 1})$ of $\mathcal{V}$ during the *Commit* phase and also all query indexes $\mathcal{I} = \mathcal{I}^{(1)} \cup \mathcal{I}_r^{(1)} \cup \dots \cup \mathcal{I}^{(\log m' - c - 1)} \cup \mathcal{I}_r^{(\log m' - c - 1)}$, of $\mathcal{V}$ during the *Query* phase (where $\mathcal{I}^{(i)}, \mathcal{I}_r^{(i)}$ the queries done over $\mathbf{C}^{(i)}$ and $\mathbf{C}_r^{(i)}$ resp.). The simulator works as follows:

1. *Initialization Step.* First, it generates the set of replies $\mathbf{q}^{(1)}, \dots, \mathbf{q}^{(\log m' - c - 1)}, \mathbf{q}_r^{(1)}, \dots, \mathbf{q}_r^{(\log m' - c - 1)} \in \mathbb{F}^l$, where $\mathbf{q}^{(i)}, \mathbf{q}_r^{(i)}$, the query replies from $\mathbf{C}^{(i)}$ and $\mathbf{C}_r^{(i)}$ respectively. This is recursively done as follows: first it samples uniformly at random the replies $\mathbf{q}^{(1)}, \mathbf{q}_r^{(1)}$ from the “valid” reply sets $\mathcal{Y}_{\mathcal{I}^{(1)}}$ and $\mathcal{Y}_{\mathcal{I}_r^{(1)}}$ respectively (as done by the (t, l) -zero knowledge simulator of the collaborative code). Then, it uses $\mathbf{q}_r^{(1)}, \mathbf{q}^{(1)}$ to compute the elements of $\mathbf{q}^{(2)}$ on the indexes queried by the

verifier at step $b.i$ of the *Query* algorithm (when $j = 1$) and obtains the rest of the elements of $\mathbf{q}^{(2)}$ (e.g., the elements queried at step $b.i$ when $j = 2$) by sampling them uniformly at random such that $\mathbf{q}^{(2)} \in \mathcal{Y}_{\mathcal{I}^{(2)}}$. Finally, it samples $\mathbf{q}_r^{(2)}$ uniformly at random from $\mathcal{Y}_{\mathcal{I}_r^{(2)}}$. Likewise, use $\mathbf{q}^{(2)}, \mathbf{q}_r^{(2)}$ to construct $\mathbf{q}^{(3)}$, and recursively repeat the same process until reaching $\mathbf{q}^{(\log m' - c)}$ that is equal to $\mathbf{C}^{(\log m' - c)}$. The simulator computes $\mathbf{q}^{(\log m' - c)}$ as follows. First, it computes the elements of $\mathbf{q}^{(\log m' - c)}$ that correspond to the queries of step $b.i$ and samples the rest of $\mathbf{q}^{(\log m' - c)}$ uniformly at random such that $\mathbf{q}^{(\log m' - c)} = \mathbf{C}^{(\log m' - c)} \in \text{RS}_{\log m' - c}^{\otimes 2}$. Let $p' : \mathbb{F}^2 \rightarrow \mathbb{F}$ be the bivariate polynomial of individual degree k' and 2^c , that interpolates $\mathbf{C}^{(\log m' - c)}$.

2. *Simulate Π^{Enc} .* $\mathcal{S}$ invokes the simulator of the Π^{Enc} protocol for $\mathbf{C}^{(1)}$ but also all codewords $\mathbf{C}_r^{(1)}, \dots, \mathbf{C}_r^{(\log m' - c - 1)}$ with the minor difference that in the first step the simulator sets the query answers to be $\mathbf{q}^{(1)}$ and $\mathbf{q}_r^{(i)}$ respectively and for the first $\log m' - c - 2$ masked codewords it uses a zero matrix. In addition, for $\mathbf{C}^{(\log m' - c - 1)}$, it initializes the view of the corrupted parties so that it agrees with $\mathbf{C}^{\log m' - c}$. Let V_1 be the view of the adversary (excluding the replies), that is, the view of each corrupted party after step 1 of the Commit phase.
3. *Simulate the rest of Commit phase.* The simulator follows all the steps performed by the corrupted provers. Furthermore, it setups the oracles by filling all parts of honest provers with zeros except from the positions in $\mathcal{I}^{(j)}, \mathcal{I}_r^{(j)}$ where it fills them with elements in $\mathbf{q}^{(j)}$ and $\mathbf{q}_r^{(j)}$ respectively. Let V_2 be the set containing all oracles.
4. Finally, the simulator sets the view to be $\{V_1, V_2, V_3 = \{\{\mathbf{q}^{(i)}, \mathbf{q}_r^{(i)}\}_{i \in [\log m' - c - 1]}, \mathbf{q}^{(\log m' - c)}\}$, where V_3 is the view of the verifier at the Query phase.

Let $\text{View}_{\mathcal{J}}(\mathbf{X}) = \{V_1^*, V_2^*, V_3^* = \{\{\mathbf{q}^{*(i)}, \mathbf{q}_r^{*(i)}\}_{i \in [\log m' - c - 1]}, \mathbf{q}^{*(\log m' - c)}\}$ be the real view of the adversary. We will show that $\{V_1^*, V_2^*, V_3^*\} \equiv \{V_1, V_2, V_3\}$. First, we omit V_2 and V_2^* as they are inherently indistinguishable. So, we focus on proving that the distributions of (V_1, V_3) and (V_1^*, V_3^*) are identical. Observe that (V_1, V_3) has been sampled uniformly at random (from the set of accepting views). Consequently, we will need to show that (V_1^*, V_3^*) also follows a uniform distribution.

To this end, we need to show that for any joint input $\mathbf{X}$, the probability **Construction 3** generates the accepting view (V_1, V_3) is $\frac{1}{|\mathcal{V}_t|}$ where $\mathcal{V}_t$ the set of all accepting views (for the specific verifier's challenges). To achieve this, we need to show that (1) for any accepting view (V_1, V_3) , we can obtain a set of bivariate polynomials $p, p_r^{(1)}, \dots, p_r^{(\log m' - c)} : \mathbb{F}^2 \rightarrow \mathbb{F}$, of the desirable individual degrees, that interpolate the codewords $\mathbf{C}^{(1)}, \mathbf{C}_r^{(1)}, \dots, \mathbf{C}^{(\log m' - c)}$ and that (2) any accepting view has the same number of such polynomials.

To prove (1), we follow a recursive strategy in which we first find $p^{(1)} := p$ and $p_r^{(1)}$ and then use them to compute $p^{(2)}$. With that in mind, observe that since the verifier performs only l queries over $\mathbf{C}^{(1)}$ and due to its (t, l) -zero knowledge property, by Lemma 2, we know that there will always be a polynomial p that interpolates $\mathbf{X}$ and evaluates to $\mathbf{q}_1^{(1)}$ at $\mathcal{I}^{(1)}$. What remains, is to interpolate $\mathbf{C}_r^{(1)}$. For this observe that $\mathbf{C}_r^{(1)}$ is directly queried by the verifier at step $b.i$ on the first round and indirectly at step $b.i$ on the second round of **Construction 3** when $\mathcal{V}$ queries $\mathbf{C}^{(2)}$. Let $\mathcal{I}'^{(2)}$ be the index set of those last queries, and $\mathbf{q}'^{(2)}$ the replies. We add to $\mathbf{q}_r^{(1)}$ at most l pairs $(\mathbf{C}_r^{(1)}[x_i, y_i], \mathbf{C}_r^{(1)}[x_i, -y_i])$ sampled uniformly at random such that:

$$\begin{aligned} & b_1^{-1}(\mathbf{q}'^{(2)}[i] - \text{INTERPOLATE}(\mathbf{C}^{(1)}[(x_i, y_i)], \mathbf{C}^{(1)}[(x_i, -y_i)])) \\ &= \text{INTERPOLATE}(\mathbf{C}_r^{(1)}[(x_i, y_i)], \mathbf{C}_r^{(1)}[(x_i, -y_i)]) \end{aligned}$$

This is done by sampling all points of the form $\mathbf{C}_r^{(1)}[x_i, y_i]$ from a valid reply set $\mathcal{Y}_{\{(x_1, y_1), \dots, (x_l, y_l)\}}$ and then use them to compute $\mathbf{C}_r^{(1)}[x_i, -y_i]$. Because the left-hand side of the above equation and $\mathbf{C}_r^{(1)}[x_i, y_i]$ are both valid it follows that $\mathbf{C}_r^{(1)}[x_i, -y_i]$ will also be valid⁵

At this point, note that the updated vector $\mathbf{q}_r^{(1)}$ will have $3l$ points. Since $\mathbf{C}_r^{(1)}$ is $(t, 3l)$ -zero knowledge, it is guaranteed by Lemma 2 that there exists a polynomial $p_r^{(1)}$ with the desired degree that interpolates $\mathbf{C}_r^{(1)}$. Lastly, using p and $p_r^{(1)}$ evaluate $\mathbf{C}^{(2)}$ and $p^{(2)}$. Following recursively the above process, we compute $p_r^{(i)}$ for all $i \in \{2, \dots, \log m' - c - 2\}$. Note that in order to compute $p_r^{(\log m' - c - 1)}$ we directly use all elements of $\mathbf{q}^{(\log m' - c)}$.

It remains to show that given (V_1, V_3) and an input $\mathbf{X}$, the number of polynomials $\{p^{(1)}, p_r^{(1)}, \dots, p_r^{(\log m' - c - 1)}\}$ we can find is fixed. First, let $\mathcal{U}$ be the vector space that contains all accepting views (V_1, V_3) . Furthermore, let $\mathbf{A} \cdot \mathbf{c} = \mathbf{v}$ be the linear equation where $\mathbf{v} \in \mathcal{V}_t$, $\mathbf{c}$ consists of the coefficients of all polynomials $p^{(1)}, p_r^{(1)}, \dots, p_r^{(\log m' - c)}$ and $\mathbf{A}$ the constraint system that evaluates to $\mathbf{v}$. Let $r_1 > \dim(\mathcal{V}_t)$ be the rank of $\mathbf{A}$. From (1) we know that $\mathbf{A}$ is solvable for any $\mathbf{v} \in \mathcal{V}_t$ and the number of possible solutions is constant and equal to $\mathbb{F}^{|\mathbf{c}| - r_1}$.

D.4 Proof of Soundness

We will first focus on proving the standard soundness of our IOPP protocol and postpone the proof for round-by-round (knowledge) soundness for the next section. Initially, we consider the case that $\Delta(\pi^{(1)}, \mathbf{C}^{(1)}) = \max(\Delta(\pi^{(1)}, \mathbf{C}^{(1)}), \Delta(\pi_r^{(1)}, \mathbf{C}^{(1)}), \dots, \Delta(\pi_r^{(\log m' - c - 1)}, \mathbf{C}^{(\log m' - c - 1)}))$. In the end, we will also discuss how to extend our proof for the general case. In particular, in the commitment phase, and for each round i , we define two bad events E_i^* and E_i as follows:

1. E_i^* : $\delta_i^* = \min(\Delta(\pi^{(i)} + \beta_i \pi_r^{(i)}, \frac{\Delta}{3}), \Delta(\text{Fold}(a_i, \pi^{(i)} + \beta_i \pi_r^{(i)}), \mathbf{C}^{(i+1)})) < \delta_i^*$.
2. E_i : $\delta_i = \min((\Delta(\pi^{(i)}, \mathbf{C}^{(i)}), \frac{\Delta}{3}))$, but $\Delta(\pi^{(i)} + b_i \pi_r^{(i)}, \mathbf{C}^{(i)}) < \delta_i$

From Claim 4 we know that E_i^* happens only with probability of at most $\frac{|\pi^{(1)}|}{|\mathbb{F}|}$. The same holds for E_i too, from Lemma 3. By taking the union bound of these probabilities, we get $\Pr[E = \bigcup_{i \in [\log m' - c]} (E_i \cup E_i^*)] = \frac{2(\log m' - c)|\pi^{(1)}|}{|\mathbb{F}|}$.

Next, assuming that conditions E do not hold, we calculate the rejection probability of the verifier at the query phase. We start with the modification done over the oracles as presented in the previous proofs. Here we note that we also modify the oracles $\pi_r^{(1)}, \dots, \pi_r^{(\log m' - c - 1)}$ so that they produce accepting paths to $\pi^{(\log m' - c)}$. Finally, following a similar argument with [35] we end up with a rejection probability of $\sum_{i \in [\log m' - c - 1]} \beta_i$, where $\beta_i = \Delta(\pi^{(i+1)}, \text{Fold}(a_i, \pi^{(i)} + b_i \pi_r^{(i)}))$. We next, find a lower bound for $\sum_{i \in [\log m' - c - 1]} \beta_i$.

To achieve this, we first show that $\beta^{(i+1)} \geq \delta_i - \delta_{i+1}$. From the triangular inequality, we have that:

$$\begin{aligned} \Delta(\text{Fold}(a_i, \pi^{(i)} + b_i \pi_r^{(i)}), \mathbf{C}^{(i+1)}) &\leq \\ \Delta(\pi^{(i+1)}, \text{Fold}(a_i, \pi^{(i)} + b_i \pi_r^{(i)})) + \Delta(\pi^{(i+1)}, \mathbf{C}^{(i+1)}) &\equiv \\ \delta_i^* &\leq \beta_{i+1} + \Delta(\pi^{(i+1)}, \mathbf{C}^{(i+1)}) = \beta_{i+1} + \delta_{i+1} \end{aligned}$$

⁵Note that by the definition of the query algorithm the indexes of the sets $\mathcal{I}_r^{(1)} = \mathcal{I}_r^{(1)}$, $\{(x_1, y_1), \dots, (x_l, y_l)\}$ and $\mathcal{I}_r'^{(1)} = \{(x_1, -y_1), \dots, (x_l, -y_l)\}$ do not lie on the same columns of $\mathbf{C}_r^{(1)}$. Consequently, we can sample from $\mathcal{Y}_{\mathcal{I}_r^{(1)}}$, $\mathcal{Y}_{\mathcal{I}_r'^{(1)}}$ and $\mathcal{Y}_{\mathcal{I}_r^{(1)}}$ independently.

Note that $\delta_{i+1} = \Delta(\pi^{(i+1)}, \mathbf{C}^{(i+1)})$, since otherwise the claim would hold trivially (see [35, 23] for more details). It remains to “link” δ_i^* and δ_i with an inequality. For this we observe that $\delta_i \leq \delta_i^*$. To prove this, we consider two cases. In the first case, we assume that $\delta_i^* = \frac{\Delta}{3} \geq \min(\Delta(\pi^{(i)}, \mathbf{C}^{(i)}), \frac{\Delta}{3}) = \delta_i$. In the second case, we assume that $\delta_i^* = \Delta(\pi^{(i)} + b_i \pi_r^{(i)}, \mathbf{C}^{(i)})$. Then, because E does not happen, it holds that $\delta_i^* = \Delta(\pi^{(i)} + b_i \pi_r^{(i)}, \mathbf{C}^{(i)}) \geq \delta_i$. Hence, we conclude that:

$$\beta_{i+1} \geq \delta_i^* - \delta_{i+1} \geq \delta_i - \delta_{i+1}$$

Last but not least, observe that we have $\sum_{i \in [\log m' - c - 1]} \epsilon_i \geq \delta_1 - \delta_{\log m' - c}$. Note that $\delta_{\log m' - c} = 0$ since otherwise the verifier would reject, so $\sum_{i \in [\log m' - c - 1]} \epsilon_i \geq \delta_1 = \delta$. Hence, for l queries, the acceptance probability would be $(1 - \sum_{i \in [\log m' - c - 1]} \epsilon_i)^l \leq (1 - \delta)^l$

Finally, to prove the soundness of our coIOPP when there is a masked oracle $\pi_r^{(i)}$ with larger hamming distance, we follow the same procedure but change the way we modify the oracles in the last step of the proof. In particular we allow modification on $\pi^{(1)}$ but not on $\pi_r^{(i)}$.

E Proof of Theorem 4

Lemma 5 (Completeness). *The coIOPP of **Construction 4** has perfect completeness.*

Proof. We show that if the provers follow honestly the protocol and if $f(\mathbf{t}') = y$, then the verifier will always accept. From the completeness of our coIOPP we know that check (1) will pass with probability 1. Furthermore, because of the completeness of the sumcheck protocol, checks of (2) will also pass with probability 1. Finally, for check (3), due to the coIOPP completeness we have that:

$$\begin{aligned} g'(\mathbf{x}, \mathbf{y}) &= g^{(\log m' - c - 1)}(r_{\log m' - c - 1}, \mathbf{x}, \mathbf{y}) + \\ &\quad b_{r_{\log m' - c - 1}} g_r^{(\log m' - c - 1)}(r_{\log m' - c - 1}, \mathbf{x}, \mathbf{y}) \end{aligned}$$

and because of the completeness of the sumcheck protocol it holds that:

$$\begin{aligned} h^{(f)}(r_f) &= \sum_{\mathbf{x}, \mathbf{y}} f_{\mathbf{t}'_1}(\mathbf{x}) f_{\mathbf{t}'_2}(\mathbf{r}, \mathbf{y}) \left(g^{(f)}(r_f, \mathbf{x}, \mathbf{y}) + b_{r_f} g_r^{(f)}(r_f, \mathbf{x}, \mathbf{y}) \right) = \\ &\quad \sum_{\mathbf{x}, \mathbf{y}} f_{\mathbf{t}'_1}(\mathbf{x}) f_{\mathbf{t}'_2}(\mathbf{r}, \mathbf{y}) g'(\mathbf{x}, \mathbf{y}) \end{aligned}$$

where $f = \log m' - c - 1$. □

Lemma 6 (t -zero knowledge). ***Construction 4** is a perfect honest-verifier and prover t -zero knowledge coIOPP in the $\mathcal{F}_{rand}^{Enc}, \mathcal{F}_{ip}$ -Hybrid model with a corrupted prover threshold of $t \leq N - k + 1$.*

Proof. We first construct a simulator $\mathcal{S}$ that has access to the random tape of the adversary that corrupts at most t provers $\{\mathcal{P}_{j_1}, \dots, \mathcal{P}_{j_t}\}$ and the verifier $\mathcal{V}$ as well as the inputs of the corrupted provers. Our simulator works as follows:

1. First, it invokes step 1 of $\mathcal{S}^{coIOPP}$ —the simulator of the coIOPP protocol—to obtain all replies from the coIOPP query phase, as well as the polynomial g' . Then, it samples the bivariate polynomials $\hat{h}^{(1)}(Y, X), \dots, \hat{h}^{(\log m' - c - 1)}(Y, X)$ (computed at step 1.d) uniformly at random with the only restriction that $\hat{h}^{(i)}(\omega_1^j, X)$, for all $j \in \mathcal{J}, i \in [\log m' - c - 1]$ are consistent with what the corrupted provers compute. Next, it instantiates the simulators $\mathcal{S}_1$ and $\mathcal{S}_2$ of the commitment and evaluation phase.

2. Simulator $\mathcal{S}_1$. The simulator runs step 2 and the beginning of step 3 of $\mathcal{S}^{coIOPP}$ (that is, up to sending the oracle of $\mathbf{C}^{(1)}$).
3. Simulator $\mathcal{S}_2$. Next, the simulator runs the rest of $\mathcal{S}^{coIOPP}$ for the evaluation phase. In addition to that, however, for each round i , it also simulates steps 1.a – 1.c as follows. In the first iteration, it sets $y_r^{(1)} = b_1^{-1}(h^{(1)}(0) + h^{(1)}(1) - y)$ while in the rest of the rounds it sets $y_r^{(j)} = b_j^{-1}(h^{(j)}(0) + h^{(j)}(1) - h^{(j-1)}(r_{j-1}))$ and simulates the functionality $\mathcal{F}_{ip}$ by returning the point $y_r^{(i)}$. Then, it locally computes the polynomials $\hat{h}^{(i)}(\omega^{j_1}, X), \dots, \hat{h}^{(i)}(\omega^{j_t}, X)$ and samples the rest of them uniformly at random such that they all evaluate at $\hat{h}^{(i)}(Y, X)$. These polynomials and also $h^{(i)}$ are sent to the adversary.
4. Let $V = (V_1, V_2, V_3)$ be the simulated view such that (1) V_1 contains the oracles, (2) V_2 the polynomials $\hat{h}^{(1)}(Y, X), \dots, \hat{h}^{(\log m' - c - 1)}(Y, X)$ (we omit the sumcheck polynomials, the univariate polynomials of the form $\hat{h}^{(i)}(\omega_1^j, X)$ and the evaluations $y_r^{(1)}, \dots, y_r^{(\log m' - c - 1)}$ since they can be derived by those polynomials), and (3) V_3 the data returned by the simulators during the Π^{Enc} protocol and the replies to the queries of the *Query* phase of the coIOPP protocol, that is, the view of the coIOPP protocol.

Let V^* be the view of the adversary when running the PCS on the real world. We will show that the distributions of (V_1, V_2, V_3) and (V_1^*, V_2^*, V_3^*) are identical. Indistinguishability between V_1 and V_1^* stems directly from the indistinguishability of the oracles. Hence, we will focus on proving $(V_2, V_3) \equiv (V_2^*, V_3^*)$. For this, we use an almost identical process with the one of Lemma 3. Namely, we will show that given any such view, and any original input $\mathbf{X}$, we can generate the polynomials $p, p_r^{(1)}, \dots, p_r^{(\log m' - c)} : \mathbb{F}^2 \rightarrow \mathbb{F}$ that produce $\mathbf{C}^{(1)}, \mathbf{C}_r^{(1)}, \dots, \mathbf{C}_r^{(\log m' - c)}$, and that the number of these polynomials is constant for any different accepting view V_2 .

With that in mind, we begin by first showing how to find the polynomials p and $p_r^{(1)}$. Note that we can not interpolate $p_r^{(1)}$ by directly following the proof of Lemma 3. This is because $\mathbf{R}_1$ is not zero and, in fact, need to evaluate it in such a way that is consistent with the sumcheck reply $h^{(1)}(X)$. To achieve this, we first compute the polynomial $h_r^{(1)}(Y, X) = b_1^{-1}(\hat{h}^{(1)}(Y, X) - h_x(Y, X))$, where $h_x(Y, X)$ and $h_r^{(1)}(Y, X)$ are bivariate polynomials such that $h_x(\omega^i, X) = \sum_{\mathbf{x} \in \{0,1\}^{\log m' - 1}} f_{t'_2}(X, \mathbf{x}) g_i(X, \mathbf{x})$ and $h_r(\omega^i, X) = \sum_{\mathbf{x} \in \{0,1\}^{\log m' - 1}} f_{t'_2}(X, \mathbf{x}) g_{i,r}^{(1)}(X, \mathbf{x})$ respectively. At this point, recall that only the first two columns of $\mathbf{R}_1$ are non-zero, which implies that only the first two coefficients $g_{i,r}^{(1)}$ are non-zero (that is, $g_{i,r}^{(1)}(0, \mathbf{0})$ and $g_{i,r}^{(1)}(1, \mathbf{0})$). Thus, we compute $g_{i,r}^{(1)}$ by setting $g_{i,r}(0) = h_r(\omega_1^i, 0)$ and $g_{i,r}(1) = h_r(\omega_1^i, 1)$ and use them to derive the columns of $\mathbf{R}_1$. Given $\mathbf{R}_1$, we can now follow the same process as with Lemma 3 to compute $p_r^{(1)}$. Finally, we recursively continue following the same process to compute all the other polynomials $p_r^{(2)}, \dots, p_r^{(\log m' - c - 1)}$.

Up until now, we have shown that for any $\mathbf{X}$ and any accepting view V_2 , we can find a set of polynomials $p, p_r^{(1)}, \dots, p_r^{(\log m' - c - 1)}$ that produce V_2 . It remains to show that the number of these polynomials is constant for any V_2 . For this we follow the same argument as in Lemma 3. $\square$

Lemma 7 ((Round-By-Round) Soundness). *Let $\pi^{(1)}, \pi_r^{(1)}, \pi_r^{(2)}, \dots, \pi_r^{(\log m' - c)}$ be the messages of the oracles sent to the verifier in the first step of the commitment algorithm. In addition, let $\delta = \min(\max(\Delta(\pi^{(1)}, \mathbf{C}^{(1)}), \{\Delta(\pi_r^{(i)}, \mathbf{C}^{(i)})\}_{i \in [\log m' - c - 1]}), \frac{\Delta}{3})$ where $\mathbf{C}^{(i)} \in RS_i^{\otimes 2}$. Assuming that $(1 - \delta)^l < \text{negl}(\lambda)$ and that $\frac{|\pi^{(1)}|}{|\mathbb{F}|} < \text{negl}(\lambda)$, then, if the verifier of **Construction 4** passes the checks of step 3, with probability $1 - \text{negl}(\lambda)$, it holds that $\Delta(\pi_r^{(i)}, \mathbf{C}^{(i)}) \leq \delta$, for all $i \in [\log m' - c - 1]$ and there exists a unique polynomial $f : \mathbb{F}^{\log k' m'} \rightarrow \mathbb{F}$ such that $\Delta(\pi_r^{(1)}, \text{Enc}(f)) \leq \delta$ and $f(\mathbf{t}') = y$.*

Lemma 8 ((Round-By-Round) Knowledge Soundness). *Let $\delta < \frac{\Delta}{3}$ and l that satisfy conditions of Lemma 7. Then, for any instance $(\pi^{(1)}, \pi_r^{(1)}, \dots, \pi_r^{(\log m' - c - 1)}, y, \mathbf{t})$ and any PPT prover that generates an accepting proof, there exists a PPT extractor that runs in (expected) polynomial time and outputs a multi-linear polynomial $f : \mathbb{F}^{\log k' m'} \rightarrow \mathbb{F}$ such that $f(\mathbf{t}) = y$ and $\Delta(\pi^{(1)}, \text{Enc}(f)) < \delta$ with probability of $1 - \kappa > 1 - \text{negl}(\lambda)$. Last but not least, our scheme has round-by-round knowledge error $\kappa_{rbr} = \max(\frac{|\mathbf{C}^{(1)}|}{|\mathbb{F}|}, (1 - \delta)^l)$.*

E.1 Proof of Lemma 7

Proof. Let $\delta < \frac{\Delta}{3}$ $\mathbf{t}_0 = (\pi^{(1)}, \{\pi_r^{(i)}\}_{i \in [\log m' - c - 1]}, y, \mathbf{r}')$ be the initial state. We say that the initial state is doomed if and only if one of the following conditions hold:

1. $\Delta(\pi^{(1)}, \mathbf{C}^{(1)}) > \delta$ or $\exists i \in [\log m' - c - 1]$ such that $\Delta(\pi_r^{(i)}, \mathbf{C}^{(i)}) > \delta$.
2. $\Delta(\pi^{(1)}, \mathbf{C}^{(1)}) \leq \delta$ but $y \neq f(\mathbf{t}') = \sum_{\mathbf{x}, \mathbf{y}} g(\mathbf{y}, \mathbf{x}) f_{\mathbf{t}'_1}(\mathbf{y}) f_{\mathbf{t}'_2}(\mathbf{x})$, where $f : \mathbb{F}^{\log k' m'} \rightarrow \mathbb{F}$ the multi-linear extension of the unique (since $\delta < \frac{\Delta}{3}$) message obtained by $\pi^{(1)}$.

Next, for each of the $j \in [\log m' - c - 1]$ rounds of the evaluation protocol, we define the following two states and describe the conditions on which they belong to the doomed set $\mathcal{D}$:

- The state $\mathbf{t}_{2i} = (\mathbf{t}_{2i-1}, \pi^{(j)}, y_r^{(j)}, b_j)$. We say that this state is doomed if for a randomly chosen b_j one of the following hold:
 1. There exists $i > j$, such that $\pi_r^{(i)}$, where $\Delta(\pi_r^{(i)}, \mathbf{C}^{(i)}) > \delta$.
 2. There exists $i \in \{1, \dots, j\}$ such that $\pi^{(i)} \neq \text{Fold}(r_j, \pi^{(i-1)} + b_{i-1} \pi_r^{(i-1)})$.
 3. $\Delta(\pi^{(j)} + b_j \pi_r^{(j)}, \mathbf{C}^{(j)}) > \delta$.
 4. $\Delta(\pi^{(j)} + b_j \pi_r^{(j)}, \mathbf{C}^{(j)}) \leq \delta$, but $y_i + a y_r^{(j)} \neq \sum_{\mathbf{x}} (g^{(j)}(\mathbf{x}) + b g_r^{(j)}(\mathbf{x})) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(\mathbf{r}, \mathbf{x})$, where $g^{(j)}, g_r^{(j)}$ the unique polynomials obtained by $\pi^{(j)}$ and $\pi_r^{(j)}$.
- The state $\mathbf{t}_{2i+1} = (\mathbf{t}_{2i}, h^j(X), r_j)$. This state is doomed if for a randomly chosen r_j the following holds:
 1. There exists $i > j$, such that $\pi_r^{(i)}$, where $\Delta(\pi_r^{(i)}, \mathbf{C}^{(i)}) > \delta$.
 2. There exists $i \in \{1, \dots, j\}$ such that $\pi^{(i)} \neq \text{Fold}(r_j, \pi^{(i-1)} + b_{i-1} \pi_r^{(i-1)})$.
 3. $\pi^{(j+1)} = \Delta(\text{Fold}(r_j, \pi^{(j)} + b_j \pi_r^{(j)}), \mathbf{C}^{(j+1)}) > \delta$
 4. $\Delta(\text{Fold}(r_j, \pi^{(j)} + b_j \pi_r^{(j)}), \mathbf{C}^{(j+1)}) \leq \delta$ but $y^{(j)} = h^{(j)}(r_j) \neq \sum_{\mathbf{x}} g^{(j+1)}(\mathbf{x}) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}((\mathbf{r}, r_j), \mathbf{x})$, where $g^{(j+1)}(\mathbf{x}) \leftarrow g^{(j)}(r_j, \mathbf{x}) + b_j g_r^{(j)}(r_j, \mathbf{x})$.

Provided these states, we will first find the probabilities $\epsilon_{2j}, \epsilon_{2j+1}$ where $\mathbf{t}_{2j}$ and $\mathbf{t}_{2j+1}$ transition from a doomed to a non-doomed state. For this, we will show that:

$$\epsilon_{2j} = \Pr_{b_j \xleftarrow{\$} \mathbb{F}} [\mathbf{t}_{2j} \notin \mathcal{D}] \leq \frac{|\pi^{(j)}|}{|\mathbb{F}|} + \frac{1}{|\mathbb{F}|} \text{ and } \epsilon_{2j+1} = \Pr_{r_j \xleftarrow{\$} \mathbb{F}} [\mathbf{t}_{2j+1} \notin \mathcal{D}] \leq \frac{|\pi^{(j)}|}{|\mathbb{F}|} + \frac{2}{|\mathbb{F}|}$$

We will prove the above inductively starting from $j = 1$ and assuming that $\mathbf{t}_0 \in \mathcal{D}$. First, we find ϵ_{2j} and show that for any randomly sampled $b_j \in \mathbb{F}$, the probability $(\mathbf{t}_0, \pi^{(1)}, y_r^{(1)}) \notin \mathcal{D}$ is negligible. We consider three cases:

1. E_1 : In case $\Delta(\pi^{(1)}, \mathbf{C}^{(1)}) > \delta$, then due to the Lemma 3, it will hold that $\Delta(\pi^{(1)} + b_1 \pi_r^{(1)}) \leq \delta$ only with probability $\frac{|\pi^{(1)}|}{|\mathbb{F}|}$.

2. E_2 : In case $\Delta(\pi^{(1)}, \mathbf{C}^{(1)}) \leq \delta$ and $y \neq \sum_{\mathbf{x}} g(\mathbf{x}) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(\mathbf{x})$, if $\Delta(\pi^{(1)} + b_1 \pi_r^{(1)}) > \delta$, then the prover stays in a doomed state with probability 1. Hence, we assume that $\Delta(\pi^{(1)} + b_1 \pi_r^{(1)}) < \delta$. Since $y = y^{(1)} \neq f(\mathbf{t}')$ then by the Schwartz-Zippel lemma, the prover leaves the doomed state with probability of only $\frac{1}{|\mathbb{F}|}$.
3. E_3 : There is $\Delta(\pi_r^{(i)}, \mathbf{C}^{(i)}) > \delta$ for $i > 1$. In this case, the prover moves to a doomed state with probability 1.

By taking the union bound of both cases, we get that $\Pr[(\mathbf{t}_0, \pi^{(1)}, y_r^{(1)}, b_1) \notin \mathcal{D}] \leq \frac{|\pi^{(1)}|}{|\mathbb{F}|} + \frac{1}{|\mathbb{F}|}$. Next, assuming that $(\mathbf{t}_0, \mathbf{t}_1)$ are in a doomed state, we will show that also $(\mathbf{t}_0, \mathbf{t}_1, h^{(1)}(X))$ will remain in a doomed state with overwhelming probability. Once again, we consider three cases:

1. E'_1 : From the previous doomed state it holds that $\Delta(\pi^{(1)} + b_1 \pi_r^{(1)}) > \delta$. Then, by Lemma 4, it holds that $\pi^{(2)} = \Delta(\text{Fold}(r_1, \Delta(\pi^{(1)} + b_1 \pi_r^{(1)})), \mathbf{C}^{(2)}) < \epsilon$ only with probability $\frac{|\pi^{(1)}|}{|\mathbb{F}|}$.
2. E'_2 : From the previous doomed state it holds that $\Delta(\pi^{(1)} + b_1 \pi_r^{(1)}, \mathbf{C}^{(2)}) \leq \delta$ but $y^{(1)} + b_1 y_r^{(1)} \neq \sum_{\mathbf{x}} (g^{(1)}(\mathbf{x}) + b_1 g_r^{(1)}(\mathbf{x})) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(\mathbf{x})$. In this case, $\text{Fold}(r_1, \pi^{(1)} + b_1 \pi_r^{(1)}) \leq \delta$ and $g^{(2)}(\mathbf{x}) = g^{(1)}(r_1, \mathbf{x}) + b_1 g_r^{(1)}(r_1, \mathbf{x})$, where $g^{(2)}$ the unique polynomial obtained by the folded code-word. Due to the Schwartz-Zippel lemma, $h^{(1)}(r_1) = y^{(2)}$ equals to $\sum_{\mathbf{x}} g^{(2)}(\mathbf{x}) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(r_1, \mathbf{x}) = \sum_{\mathbf{x}} (g^{(1)}(r_1, \mathbf{x}) + b_1 g_r^{(1)}(r_1, \mathbf{x})) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(r_1, \mathbf{x})$ with probability of only $\frac{2}{|\mathbb{F}|}$.
3. E_3 : There is $\Delta(\pi_r^{(i)}, \mathbf{C}^{(i)}) > \delta$ for $i > 1$. In this case, the prover moves to a doomed state with probability 1.

We follow an identical argument for any $j \in \{2, \dots, \log m' - c\}$, but we also consider one additional case, each time, where there exists $i \in \{1, \dots, j\}$ such that $\pi^{(i)} \neq \text{Fold}(r_j, \pi^{(i-1)} + b_{i-1} \pi_r^{(i-1)})$ and either $\Delta(\pi^{(i-1)} + b_{i-1} \pi_r^{(i-1)}, \mathbf{C}^{(i-1)}) > \delta$ or $g^{(i)} \neq g^{(i)}$ such that $\Delta(\pi^{(i)}, \text{Enc}(g^{(i)})) \leq \delta$. If this case is true, then the prover stays to a doomed state with probability 1.

Finally, we define the transcript $\mathbf{t}_f = (\mathbf{t}_{2(\log m' - c)}, \mathbf{C}^{(\log m' - c)}, \mathcal{I} = \{(x_1, y_1), \dots, (x_l, y_l)\})$, where $\mathbf{C}^{(\log m' - c)}$ the last codeword sent by the prover, and $\mathcal{I}$ the coIOPP queries of the verifier. We say that $\mathbf{t}_f$ is doomed if the following hold:

1. $\mathbf{C}^{(\log m' - c)}$ is not a valid codeword.
2. $h^{(\log m' - c - 1)}(r_{\log m' - c - 1}) \neq \sum_{\mathbf{x}} g^{(\log m' - c)}(\mathbf{x}) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(\mathbf{r}, \mathbf{x})$, where $g^{(\log m' - c)}$ the polynomial obtained by $\mathbf{C}^{(\log m' - c)}$.
3. There is an inconsistent query in the *coIOPP.Query* algorithm.

Assume that $\mathbf{t}_{2(\log m' - c) - 1} \in \mathcal{D}$, then, we will show that the probability $\mathbf{t}_f \notin \mathcal{D}$ is at most $(1 - \delta)^l$. Observe that if $\mathbf{C}^{(\log m' - c)}$ is not a valid codeword, the verifier rejects with probability 1. Next, assume that $\Delta(\pi^{(1)}, \mathbf{C}^{(1)}) > \delta$, or $\exists \pi^{(i)}$ such that $\Delta(\pi^{(i)}, \mathbf{C}^{(i)}) > \delta$ or $\Delta(\pi^{(i)} + b_i \pi_r^{(i)}, \mathbf{C}^{(i)}) > \delta$. By Theorem 3, we get that the probability the verifier passes the *coIOPP.Query* is $(1 - \delta)^l$. Finally, assume that $\Delta(\pi^{(1)}, \mathbf{C}^{(1)}) < \delta$ but $f(\mathbf{t}') \neq y$. We furthermore, assume that $\forall \pi^{(i)}$, it holds that $\Delta(\pi^{(i)}, \mathbf{C}^{(i)}) \leq \delta$ and $\Delta(\pi^{(i)} + b_i \pi_r^{(i)}, \mathbf{C}^{(i)}) \leq \delta$ since otherwise the *coIOPP.Query* algorithm would fail with overwhelming probability. Then, the only way the prover can move to a non-doomed would be to send an oracle π_i that encodes a polynomial $g^{(i)}$ different than that of $\text{Fold}(r_{i-1}, \pi^{(i-1)} + b_{i-1} \pi_r^{(i-1)})$, and also

$\Delta(\pi_i, \text{Enc}(g^{(i)})) \leq \delta$. Following the same argument as in [23], we can show that if the prover passes the *coIOPP*, then with overwhelming probability it holds that $\Delta(\pi^{(i)}, \text{Enc}(g^{(i)})) \leq 2\delta$, and thus, we end up with $\Delta \leq \Delta(\text{Enc}(g^{(i)}, g^{(i)})) \leq 3\delta < \Delta$ which is a contradiction. $\square$

E.2 Proof of Lemma 8

We prove round-by-round knowledge soundness using the same process with the one shown in [67]. We focus on the first $2(\log m' - c - 1) + 1$ states as defined in our proof for round-by-round soundness. Assume that:

$$\Pr_{r_j \xleftarrow{\$} \mathbb{F}} [(\mathbf{t}_{2j}, h^{(j)}(X)) \notin \mathcal{D}] > \epsilon_{rbr}$$

Then, we will show that $\mathbf{t}_{2j} \notin \mathcal{D}$. Since $(\mathbf{t}_{2j}, h^{(j)}(X)) \notin \mathcal{D}$, it implies that $\pi^{(i)} = \text{Fold}(r_i, \pi^{(i-1)} + b_{i-1}\pi_r^{(i-1)})$. Consequently, if $\mathbf{t}_{2j} \in \mathcal{D}$, one of the following should hold:

1. $\Delta(\pi^{(j)} + b_j\pi_r^{(j)}, \mathbf{C}^{(j)}) > \delta$ but $\Delta(\text{Fold}(r_j, \pi^{(j)} + b_j\pi_r^{(j)}), \mathbf{C}^{(j+1)}) \leq \delta$
2. $\Delta(\text{Fold}(r_j, \pi^{(j)} + b_j\pi_r^{(j)}), \mathbf{C}^{(j+1)}) \leq \delta$ and $y^{(j)} + ay_r^{(j)} \neq \sum_{\mathbf{x}} (g^{(j)}(\mathbf{x}) + b_j g_r^{(j)}(\mathbf{x})) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(\mathbf{r}, \mathbf{x})$ but $y^{(j)} = \sum_{\mathbf{x}} g^{(j+1)}(\mathbf{x}) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}((\mathbf{r}, r_j), \mathbf{x})$

Observe that the first case can happen with probability larger than ϵ_{rbr} if and only if $\Delta(\pi^{(j)} + b_j\pi_r^{(j)}, \mathbf{C}^{(j)}) \leq \delta$, since otherwise Lemma 4 would be invalid, leading to a contradiction. Furthermore, due to the Schwartz-Zippel Lemma, the second case can happen with probability larger than ϵ_{rbr} only if $y^{(j)} + ay_r^{(j)} = \sum_{\mathbf{x}} (g^{(j)}(\mathbf{x}) + b_j g_r^{(j)}(\mathbf{x})) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(\mathbf{r}, \mathbf{x})$. Consequently, $\mathbf{t}_{2j} \notin \mathcal{D}$.

Next, assume that:

$$\Pr_{b_j \xleftarrow{\$} \mathbb{F}} [(\mathbf{t}_{2j-1}, \pi^{(j)}, y_r^{(j)}) \notin \mathcal{D}] > \epsilon_{rbr}$$

We will show that $\mathbf{t}_{2j-1} \notin \mathcal{D}$ must hold. As above, $(\mathbf{t}_{2j-1}, \pi^{(j)}, y_r^{(j)}) \notin \mathcal{D}$ implies that $\pi^{(i)} = \text{Fold}(r_{i-1}, \pi^{(i-1)} + b_{i-1}\pi_r^{(i-1)})$ for all $i \in [j]$. In addition, if $\mathbf{t}_{2j-1} \in \mathcal{D}$, the following must hold:

1. $\Delta(\pi^{(j)}, \mathbf{C}^{(j)}) > \delta$ or $\Delta(\pi_r^{(j)}, \mathbf{C}^{(j)}) > \delta$ and $\Delta(\pi^{(j)} + b_j\pi_r^{(j)}, \mathbf{C}^{(j)}) \leq \delta$.
2. $\Delta(\pi^{(j)}, \mathbf{C}^{(j)}) \leq \delta$, $\Delta(\pi_r^{(j)}, \mathbf{C}^{(j)}) \leq \delta$, and $y^{(j)} \neq \sum_{\mathbf{x}} g^{(j)}(\mathbf{x}) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(\mathbf{r}, \mathbf{x})$ or $y_r^{(j)} \neq \sum_{\mathbf{x}} g_r^{(j)}(\mathbf{x}) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(\mathbf{r}, \mathbf{x})$ but $y^{(j)} + ay_r^{(j)} = \sum_{\mathbf{x}} (g^{(j)}(\mathbf{x}) + b_j g_r^{(j)}(\mathbf{x})) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(\mathbf{r}, \mathbf{x})$.

It is not hard to see that if the first case is true for probability larger than ϵ_{rbr} , then it contradicts Lemma A.1 of [30]. Likewise, due to the Schwartz-Zippel Lemma, the second case happens with probability larger than ϵ_{rbr} if and only if y_r and $y^{(j)}$ are correct. Hence, we conclude that $\mathbf{t}_{2j-1} \notin \mathcal{D}$.

We follow the same process inductively to show that $\mathbf{t}_0 \notin \mathcal{D}$. What remains is to show that if $\mathbf{t}_f = (\mathbf{t}_{2(\log m' - c - 1)}, \pi^{(\log m' - c)}, \mathcal{I}) \notin \mathcal{D}$ with probability larger than ϵ_{rbr} , then it must hold that $\mathbf{t}_{2(\log m' - c - 1)} \notin \mathcal{D}$. Since $\mathbf{t}_{2(\log m' - c)} \in \mathcal{D}$, we consider three cases:

1. $\Delta(\pi^{(\log m' - c)}, \mathbf{C}^{(\log m' - c)}) > \delta$
2. $\Delta(\pi^{(\log m' - c)}, \mathbf{C}^{(\log m' - c)}) = 0$ but $h^{(\log m' - c - 1)}(r_{\log m' - c - 1}) \neq \sum_{\mathbf{x}} g^{(\log m' - c)}(\mathbf{x}) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(\mathbf{r}, \mathbf{x})$.
3. There exists $i \in [\log m' - c - 1]$ such that $\pi^{(i)} \neq \text{Fold}(r_i, \pi^{(i-1)} + b_i\pi_r^{(i-1)})$.

Since the verification algorithm is accepting, the first and second case must not hold. Furthermore, by the soundness analysis of the query phase, and by the assumption that $\mathbf{t}_0 \in \mathcal{D}$, the third case only holds with probability of at most $(1 - \delta)^l$. Consequently, $\mathbf{t}_{2(\log m' - c - 1)} \notin \mathcal{D}$.

F Proof of Theorem 5

F.1 Completeness and Performance Analysis

Completeness comes from the completeness of $\mathcal{F}_{rand}^{Enc}$, the collaborative sumcheck protocol of [12, 13] as well as the homomorphic properties of our collaborative code. Performance-wise, our coSumcheck protocol has the following properties. First, $k_V(n, N)$ and $k_P(n, N)$ are $\mathcal{O}(\log n)$. The first one, is due to step 3, the sumcheck rounds and steps 3, 5. The complexity of $k_P(n, N)$ derives from steps 2, 4 and 5.a. In addition, verifier has to make a single query over the polynomial oracle of f and $\mathcal{O}(l \log(l + \log n) + N)$ queries over $\mathbf{C}^{(1)}$ and $\mathbf{C}^{(2)}$ (when optimization 3 is applied). Likewise, we argue about verification time which is $vt(n, N) \in \mathcal{O}((l + \log n)N)$.

It is not hard to see that $pt_i(n, N) \in \mathcal{O}(\frac{n}{N})$ since, the prover time is dominated by step 4 which requires $\mathcal{O}(\frac{n}{N})$ field operations [13]. Last but not least, note that $cm(n, N) \in \mathcal{O}(\log n)$ because of the steps 2, 4 and 5.a.

F.2 Proof of t -zero knowledge

We begin by constructing a simulator $\mathcal{S}$ that has access to an adversary that corrupts t provers with indexes $\mathcal{J} = \{j_1, \dots, j_t\}$ and the verifier $\mathcal{V}$. The simulator also has access to the sum K , oracle access to the polynomial f and works as follows:

1. The simulator samples the sumcheck polynomials $\{h_1(X), \dots, h_{\log n}(X)\}$ such that they produce an accepting sumcheck proof and sets $K' = h_1(0) + h_1(1)$. Then, it picks a matrix $\mathbf{R}_1$ uniformly at random such that $K' = \sum_{\mathbf{x}} f_r(\mathbf{x})$ and $f_r(\mathbf{r}) = y_r = b^{-1}(h_{\log n}(\mathbf{r}_{\log n}) - y)$. Finally, it samples $\mathbf{R}_2$ uniformly at random. Given these data, $\mathcal{S}$ simulates **Construction 5** as follows:
2. It first simulates the $\mathcal{F}_{rand}^{Enc}$ functionality, by sending $\mathbf{C}_j^{(1)}$ and $\mathbf{C}_j^{(2)}$ for all $j \in \mathcal{J}$ to the adversary. Let V_1 be the view of the adversary at that phase.
3. It simulates $\mathcal{F}_{ip}$ by sending K' to the adversary.
4. It simulates $\mathcal{F}_{sumcheck}$ by sending $h_1(X), \dots, h_{\log n}(X), y, y_r$ to the adversary.
5. It simulates step 5 by first sending y'_r , then $\mathbf{C}'$ and, finally, the replies $\mathbf{q}^{(1)}, \mathbf{q}^{(2)} \in \mathbb{F}^l$ of $\mathbf{C}^{(1)}$ and $\mathbf{C}^{(2)}$ at the indexes $\mathcal{I}$.

It is not hard to see that the view generated by the simulator is an accepting zero-knowledge proof for the sumcheck instance $K = \sum_{\mathbf{x}} f(\mathbf{x})$ sampled uniformly at random. Hence, we will focus on showing that the view obtained by running **Construction 5** in the real world follows a uniform distribution over all accepting proofs.

As a first step, we will show that for any such view V , and polynomial f there exist bivariate polynomials $p_r, p'_r : \mathbb{F}^2 \rightarrow \mathbb{F}$ that produce the codewords $\mathbf{C}^{(1)}, \mathbf{C}^{(2)}$ and also the sumcheck polynomials. To achieve this, we first compute $h_r^{(i)}(X) \leftarrow b^{-1}(h^{(i)}(X) - h_f^{(i)}(X))$, where $h_f^{(i)}(X)$ the i -th sumcheck message of the instance $K = \sum_{\mathbf{x}} f(\mathbf{x})$. Then, we use them to compute the matrix $\mathbf{R}_1$ by solving a system of linear equations obtained by these conditions (similarly to what is done in [71]). Given that $\mathbf{C}^{(1)}$ is (t, l) -zero knowledge, from lemma 2 we know that there will always exist a polynomial p_r of correct degree that passes through $\mathbf{R}_1, \{\mathbf{C}_j^{(1)}\}_{j \in \mathcal{J}}$ and $\mathbf{q}^{(1)}$. We thus select a p_r , compute its codeword $\mathbf{C}^{(1)}$, and finally find $\mathbf{C}^{(2)} = \mathbf{C}' - \mathbf{C}^{(1)}$. From $\mathbf{C}^{(2)}$, we find p'_r .

What remains, is to show that the number of polynomials p_r, p'_r for any view is the same. Note that the number of different matrixes $\mathbf{R}_1$ we can find for a single view is $|\mathbb{F}|^{k \log n - \log n}$ since the

system of linear equations described previously has full rank. Furthermore, for any such $\mathbf{R}_1$ we can interpolate the same number of polynomials p_r and subsequently p'_r . Consequently, the number of polynomials p_r, p'_r that can produce a single view is constant and the same with any other view. Hence, for any given p_r, p'_r and f , the probability we obtain a view V is constant. $\square$

F.3 Proof of Soundness

First, we show that if the evaluation claim of the masked matrix is incorrect, that is, $y_r \neq f_r(\mathbf{r})$, then the verifier accepts step 4 only with negligible probability. Initially, assume the event that $y'_r \neq f'_r(\mathbf{r})$ but $f_r(\mathbf{r}) + cy'_r = y_r + cy'_r$. Due to the Schwartz-Zippel lemma, this can happen only with probability $\frac{1}{|\mathbb{F}|}$. Hence, $f_r(\mathbf{r}) + cy'_r \neq y_r + cy'_r$. Next, assume that $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) > \frac{|\pi^{(1)}|\Delta}{3}$, where $\pi^{(1)}$ and $\pi^{(2)}$ the oracles of $\mathbf{R}_1, \mathbf{R}_2$ sent to the verifier. Then, by [30] we have that $\Delta(\pi^{(1)} + c\pi^{(2)}, \mathbf{C}) > \delta$ with all but $\frac{|\pi^{(1)}|}{|\mathbb{F}|}$ probability. So, the verifier would reject with probability $(1 - \delta)^n < \text{negl}(\lambda)$. Otherwise, assume that $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) < \delta$, and let $\mathbf{C}^{(1)}, \mathbf{C}^{(2)}$ be the codewords closest to $\pi^{(1)}, \pi^{(2)}$. Observe that $\pi^{(1)}, \pi^{(2)}$ uniquely decode to $\mathbf{R}_1, \mathbf{R}_2$. Based on this, we consider two cases: (1) $\mathbf{C}^{(1)} + c\mathbf{C}^{(2)} = \mathbf{C}$ and (2) $\mathbf{C}^{(1)} + c\mathbf{C}^{(2)} \neq \mathbf{C}$. For (1), observe that it must hold that $\mathbf{R}' = \mathbf{R}_1 + b\mathbf{R}_2$ and thus, by our initial assumption, hence the verifier would reject with probability 1 since $f''(\mathbf{r}) \neq y_r + cy'_r$. For (2) we follow the same argument as in [22] (Section 4.1) to show that the chance of accepting all queries is at most $(1 - \frac{2}{3}\Delta)^n < \text{negl}(\lambda)$.

Having established that, we can now consider two cases. The first is that $K + bK' = \sum_{\mathbf{x}}(f(\mathbf{x}) + bf_r(\mathbf{x}))$. This however can happen only with probability $\frac{1}{|\mathbb{F}|}$. Consequently, we assume that $K + bK' \neq \sum_{\mathbf{x}}(f(\mathbf{x}) + bf_r(\mathbf{x}))$. Due to the soundness of the sumcheck protocol, the probability the verifier obtains an accepting proof is only $\frac{\log n}{|\mathbb{F}|}$. Consequently, with overwhelming probability it holds that $y_r \neq f_r(\mathbf{r})$. As discussed above, the verifier accepts it only with probability $(1 - \Delta/3)^l = \text{negl}(\lambda)$. By taking the union bound over all cases we end up with a soundness error of $\frac{\log n + 1}{|\mathbb{F}|} + (1 - \delta)^l$. $\square$

F.4 Proof of Round-by-Round (Knowledge) Soundness

We first prove round-by-round soundness and then show how to modify the original proof in order to prove round-by-round (knowledge) soundness. Let $\mathbf{t}_0 = (K, \hat{f}, \pi^{(1)}, \pi^{(2)})$ be the initial state (where $\hat{f}$ is the oracle of f). We say that it is doomed if (1) $K \neq \sum_{\mathbf{x}} f(\mathbf{x})$ and $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) \leq \delta$ or (2) $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) > \delta$.

Next, let $\mathbf{t}_1 = (\mathbf{t}_0, K', b)$. This state is doomed if (1) $K + bK' \neq \sum_{\mathbf{x}}(f(\mathbf{x}) + bf_r(\mathbf{x}))$ and $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) \leq \delta$ or (2) $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) > \delta$. Observe that the probability ϵ_1 that $\mathbf{t}_0 \in \mathcal{D}$, where $\mathcal{D}$ the doomed set, but $\mathbf{t}_1 \notin \mathcal{D}$. We will show that this happens only with probability $\frac{1}{|\mathbb{F}|}$. Assume that $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) > \delta$. Then the prover moves to a doomed state with probability 1. So we consider the case $K \neq \sum_{\mathbf{x}} f(\mathbf{x})$ and $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) \leq \delta$. Let f_r be the polynomial obtained by $\pi^{(1)}$. The prover can move to a non-doomed state if $K + bK' = \sum_{\mathbf{x}}(f(\mathbf{x}) + bf_r(\mathbf{x}))$. This however happens only with $\frac{1}{|\mathbb{F}|}$ probability.

Let $\mathbf{t}_j$ for $j \in \{2, 2 + \log n\}$ be the states obtained by the sumcheck protocol. We note that if $\mathbf{t}_j$ is doomed then $\mathbf{t}_{j+1}$ will be non-doomed with probability $\frac{1}{|\mathbb{F}|}$. To prove this we consider two cases. The first one is to start with a wrong sumcheck instance which implies a round-by-round soundness of $\frac{1}{|\mathbb{F}|}$ [39]. The other case is that $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) > \frac{\Delta}{3}$ where the round-by-round soundness is 0.

Let $\mathbf{t}_{3+\log n} = (\mathbf{t}_{2+\log n}, y, y_r, y'_r, c)$ be the state after the last sumcheck round. This state is doomed if (1) $h^{(\log n)}(r_{\log n}) \neq y + cy_r$, (2) $y \neq f(\mathbf{r})$ or $y_r \neq f_r(\mathbf{r})$ or (3) $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) > \delta$. We will show that the probability that $\mathbf{t}_{2+\log n} \in \mathcal{D}$ but $\mathbf{t}_{3+\log n} \notin \mathcal{D}$ is $\frac{2\log n}{|\mathbb{F}|}$. First, if $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) > \delta$,

then the prover remains in a doomed state with probability 1. To this point, the only way a prover can leave a doomed state is if $h^{(\log n)}(r_{\log n}) = y + cy_r$ and $y = f(\mathbf{r})$, $y_r = f_r(\mathbf{r})$. That can happen only with probability $\frac{2 \log n}{|\mathbb{F}|}$. Note that if $y \neq f(\mathbf{r})$, the prover can not leave the doomed state for the rest of the protocol hence verification will fail. So we assume that $y = f(\mathbf{r})$ and thus $y_r \neq f_r(\mathbf{r})$.

Let $\mathbf{t}_f = (\mathbf{t}_{3+\log n}, \mathbf{C}, \mathcal{I})$, where $\mathcal{I}$ the queries of the verifier over the oracles $\pi^{(1)}, \pi^{(2)}$. We want to show that if $\mathbf{t}_{3+\log n}$ is in a doomed state, that is, $y_r \neq f_r(\mathbf{r})$ or $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) > \delta$ then $\mathbf{t}_f$ goes to a non-doomed state only with probability $(1 - \delta)^l$. This comes directly by the standard soundness proof of our scheme.

In summary, the round-by-round complexity of **Construction 5** is $\epsilon_{rbr} = \max(\epsilon_1, \dots, \epsilon_{3+\log n}, \epsilon_f) = \max(\frac{2 \log n}{|\mathbb{F}|}, (1 - \frac{\Delta}{3})^l)$.

Proving Round-by-Round Knowledge Soundness. Our proof is similar to that of proving round-by-round knowledge soundness of our coIOPP protocol. Here, we will provide a proof sketch on why $\kappa_{rbr} = \epsilon_{rbr}$. First, assume that $\mathbf{t}_f$ is not doomed but $\mathbf{t}_{3+\log n} \in \mathcal{D}$ with probability larger than ϵ_{rbr} . This means that (1) $y_r = f_r(\mathbf{r})$ (2) $y = f(\mathbf{r})$ and that the verifier passes all checks. Since $\mathbf{t}_{3+\log n} \in \mathcal{D}$, it must hold that $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) > \delta$. However the verifier passes the checks with probability larger than ϵ_{rbr} which indicates a contradiction. Consequently, $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) < \delta$ and $\mathbf{t}_{3+\log n} \notin \mathcal{D}$.

Next, assume that $\mathbf{t}_{3+\log n} \notin \mathcal{D}$ but $\mathbf{t}_{2+\log n} \in \mathcal{D}$ with probability larger than ϵ_{rbr} . Because $\mathbf{t}_{3+\log n} \notin \mathcal{D}$, it must hold that: (1) $h^{(\log n)}(r_{\log n}) = y + cy_r$, (2) $y = f(\mathbf{r})$ and $y_r = f_r(\mathbf{r})$ and (3) $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) \leq \delta$. Since $\mathbf{t}_{2+\log n} \in \mathcal{D}$, it must also hold that $h^{(\log n)}(X) \neq f'(\mathbf{r}, X) + cf'_r(\mathbf{r}, X)$ or $f'(\mathbf{x}) \neq f(\mathbf{x})$ or $f'_r(\mathbf{x}) \neq f_r(\mathbf{x})$. Based on this, and from the Schwartz-Zippel lemma, for any $r_{\log n}$ we move from $\mathbf{t}_{1+\log n} \in \mathcal{D}$ to $\mathbf{t}_{2+\log n} \notin \mathcal{D}$ only with probability of at most $\frac{\log n}{|\mathbb{F}|}$ which is smaller than ϵ_{rbr} , leading to a contradiction. We follow a similar argument for the rest of the sumcheck rounds.

Finally, assume that $\mathbf{t}_0 \in \mathcal{D}$ but $\mathbf{t}_1 \notin \mathcal{D}$ with probability larger than ϵ_{rbr} . By definition, it must hold that $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) \leq \delta$ and $K + bK' = \sum_{\mathbf{x}} (f'(\mathbf{x}) + bf_r(\mathbf{x}))$. Since, by assumption, $\mathbf{t}_0$ it must hold that $K \neq \sum_{\mathbf{x}} f(\mathbf{x})$ or $K' \neq \sum_{\mathbf{x}} f_r(\mathbf{x})$. By the Schwartz-Zippel lemma, the probability that $\mathbf{t}_1 \notin \mathcal{D}$ is at most $\frac{1}{|\mathbb{F}|} < \epsilon_{rbr}$ which leads to contradiction. So, $\mathbf{t}_0 \notin \mathcal{D}$ which concludes the proof. $\square$

G Proof of Theorem 6

Lemma 9. *The coPIOP protocol of **Construction 5** is t -zero knowledge with $t \leq \frac{N}{2} - k + 1 = (\frac{1}{2} - \epsilon)N + 1$, where $\epsilon \in (0, \frac{1}{2})$.*

Proof. We first construct a simulator $\mathcal{S}$ that has access to an adversary that controls t corrupted provers as well as the verifier. The simulator also has oracle access to the multi-linear extension of $\mathbf{w}$ and works as follows:

1. Sets $\mathbf{w} = \mathbf{0}$, samples uniformly at random the values $K', y_L, y_R, y_O, y_w, y_1, y_2, y_3$. Sets $rnd \leftarrow y_w \cdot \beta(\mathbf{n}, \mathbf{r}'')^{-1}$ and finds $\mathbf{r}_1, \mathbf{r}_2$ and $\mathbf{r}_3$ such that they satisfy the following constraints:

- $K'(r')^{-1} = \beta(\mathbf{0}, \mathbf{r})(r_{31} - r_{11}r_{21}) + \beta(\mathbf{1}, \mathbf{r})(r_{32} - r_{12}r_{22}) + \beta(\mathbf{2}, \mathbf{r})r_{33}$.
- $y_L \cdot \mathbf{r}'[\log m + 1]^{-1} = \beta(\mathbf{0}, \mathbf{r}'_l)r_{11} + \beta(\mathbf{1}, \mathbf{r}'_l)r_{12}$, where $\mathbf{r}'_l$ has the $\log m$ least significant values.
- $y_R \cdot \mathbf{r}'[\log m + 1]^{-1} = \beta(\mathbf{0}, \mathbf{r}'_l)r_{21} + \beta(\mathbf{1}, \mathbf{r}'_l)r_{22}$.
- $y_O \cdot \mathbf{r}'[\log m + 1]^{-1} = \beta(\mathbf{0}, \mathbf{r}'_l)r_{31} + \beta(\mathbf{1}, \mathbf{r}'_l)r_{32} + \beta(\mathbf{2}, \mathbf{r}'_l)r_{33}$.
- $y_M = \beta(\mathbf{0}, \mathbf{r}'')r_{M1} + \beta(\mathbf{1}, \mathbf{r}'')r_{M2} + \beta(\mathbf{2}, \mathbf{r}'')r_{M3}$, for $M \in [3]$. Note that for $M \in [2]$, $r_{M3} = 0$.

2. Note that if this system is solvable, then it has a unique solution. In addition, for any randomly selected $\mathbf{r}, \mathbf{r}', \mathbf{r}''$, the probability that the above system does not has a solution is negligible (inversely proportional to $|\mathbb{F}|$) [23, 22]. If the system is not solvable, the simulator aborts.
3. Sends the oracles of the MLEs of $(\mathbf{0}, \mathbf{w}_r)$ and $\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3$ to the adversary.
4. Simulates the $\mathcal{F}_{ip}$ functionality by sending K' to the adversary.
5. Calls the simulator of our sumcheck protocol for the first sumcheck instance and obtains the view $V_{sumcheck}^{(1)}$.
6. Emulate the honest verifiers at step 5.
7. Calls the simulator of our sumcheck protocol for the second sumcheck instance and obtains the view $V_{sumcheck}^{(2)}$.

Let $V = \{V_1 = (K', y_L, y_R, y_O, y_w, y_1, y_2, y_3), V_{sumcheck}^{(1)}, V_{sumcheck}^{(2)}\}$ be the view obtained by simulator. Here we exclude the oracles since by default they are indistinguishable. Furthermore, we exclude y_A, y_B, y_C since they are publicly known. With that in mind, we want to show that V has the same distribution with the one obtained in the real world, namely, $V^* = \{V_1^* = (K'^*, y_L^*, y_R^*, y_O^*, y_w^*, y_1^*, y_2^*, y_3^*), V_{sumcheck}^{*(1)}, V_{sumcheck}^{*(2)}\}$. By the t -zero knowledge property **Construction 5**, we get that $V_{sumcheck}^{*(1)} \equiv V_{sumcheck}^{(1)}$ and $V_{sumcheck}^{*(2)} \equiv V_{sumcheck}^{(2)}$. We thus focus on showing that the rest of the elements are indistinguishable. Note that V_1 follows a uniform distribution over $\mathbb{F}^8$. So, we need to show that V_1^* also follows a uniform distribution. We will show that for any fixed $\mathbf{w}$, there exist $\mathbf{w}_r, \mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3$ that generate an accepting view V_1^* . Let E_1 be the event that the constraint system of V_1^* as described in the the simulator algorithm is solvable. Then there exist unique $rnd, \mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3$ that produce V_1^* . Consequently, conditioned E_1 , V_1^* is uniformly distributed. In other words:

$$\Pr[V \leftarrow View_{\mathcal{A}}^S(\mathbf{x}, \mathbf{w}_A) | E_1] \equiv \Pr[V^* \leftarrow View_{\mathcal{A}}^R(\mathbf{x}, \mathbf{w}) | E_1]$$

In addition, observe that from [23] the probability $\neg E$ happens is only when the expansions $\exp(\mathbf{r}), \exp(\mathbf{r}'), \exp(\mathbf{r}'')$ (with, $\exp(\mathbf{r}) = [\beta(\mathbf{0}, \mathbf{r}), \dots, \beta(\mathbf{n}, \mathbf{r})]$) do not lead to a rank three system which happens only with probability $\frac{3}{|\mathbb{F}|}$. Consequently, $\Pr[V^* \leftarrow View_{\mathcal{A}}^R(\mathbf{x}, \mathbf{w}) | \neg E_1] = \frac{3}{|\mathbb{F}|}$ [23]. Thus, the statistical distance between the distributions $\Pr[V \leftarrow View_{\mathcal{A}}^S(\mathbf{x}, \mathbf{w}_A)]$ and $\Pr[V^* \leftarrow View_{\mathcal{A}}^R(\mathbf{x}, \mathbf{w})]$ is $\frac{3}{|\mathbb{F}|}$.

Security Threshold. All sumchecks in our protocol involve two secret shared polynomials but one of them also involves a public polynomial that is multi-linear extension of a vector with a tensor-like structure. We know that the security threshold of these sumchecks is $t \leq \frac{N}{2} - k + 1$ [13]. Finally, using the protocols of [12, 13] the security threshold of $\mathcal{F}_{ip}$ for two secret matrixes is also $t \leq \frac{N}{2} - k + 1$. Hence, the overall security threshold of our construction is $t \leq \frac{N}{2} - k + 1$. $\square$

Lemma 10. *Construction 6 is a coIOPP with soundness error $\epsilon = \frac{3 \log m + 2 \log n + 10}{|\mathbb{F}|} + 4(1 - \delta)^l$.*

Proof. Let E_1 be the event that $K' \neq \sum_i \beta(\mathbf{i}, \mathbf{r})(\mathbf{r}_1[i] \mathbf{r}_2[i] - \mathbf{r}_3[i])$ or there exists $\mathbf{u}_L[i] \mathbf{u}_R[i] \neq \mathbf{u}_O[i]$ for $i \in [m]$. By the Schwartz-Zippel Lemma, for any $\mathbf{r}$, the probability that the first sumcheck instance is $\frac{1 + \log m}{|\mathbb{F}|}$. Next, let E_2 be the even that the instance does not hold, but at the end of the protocol the verifier ends up with an accepting proof and correct evaluation claims. Due to the soundness of **Construction 5**, this can happen only with probability $\frac{2 \log m + 3}{|\mathbb{F}|} + 2(1 - \delta)^l$. Now, assume E_3 be the event that there exists an invalid y_L, y_R, y_O but the sumcheck instance is valid. This can happen only with probability $\frac{3}{|\mathbb{F}|}$ due to the Schwartz-Zippel Lemma. Finally, let E_4 be the event that the

sumcheck instance is invalid but the provers generate an accepting proof and that the evaluation claims are valid. Due to the soundness of **Construction 5** this can happen only with probability $\frac{2\log n+3}{|\mathbb{F}|} + 2(1-\delta)^l$.

By taking the union bound over these events, we get a soundness error of $\epsilon = \frac{3\log m+2\log n+10}{|\mathbb{F}|} + 4(1-\delta)^l$. $\square$

Lemma 11. *Construction 6 has round-by-round (knowledge) soundness errors $\kappa_{rbr} = \epsilon_{rbr} = \max(\frac{2\log n}{|\mathbb{F}|}, \frac{2\log m}{|\mathbb{F}|}, (1-\delta)^l)$.*

Proof. Let $\mathbf{t}_0 = (\pi_{pr}, f_{\mathbf{w}^*})$ be the initial state, where π_{pr} contains the oracles of the mask codewords computed in the pre-processing phase of the sumcheck protocol. We say that $\mathbf{t}_0$ is doomed if (1) $\exists i \in [m]$, such that $\mathbf{u}_L[i]\mathbf{u}_R[i] \neq \mathbf{u}_O[i]$, or (2) if there exists a masking codeword such that $\Delta(\pi, \mathbf{C}) > \delta$. Let $\mathbf{t}_1 = (\mathbf{t}_0, f_1, f_2, f_3, \mathbf{r})$. If $\mathbf{t}_0 \in \mathcal{D}$, then, the probability ϵ_1 $\mathbf{t}_1$ leaves a doomed state is at most $\frac{\log m}{|\mathbb{F}|}$. To prove this we consider two cases: (a) $\mathbf{u}_L \circ \mathbf{u}_R \neq \mathbf{u}_O$, so the probability to leave a doomed state is if and only if $0 = \sum_{\mathbf{x}} \beta(\mathbf{r}, \mathbf{x})(f_L(\mathbf{x})f_R(\mathbf{x}) - f_O(\mathbf{x}))$, which due to the Schwartz-Zippel Lemma happens only with probability $\frac{\log m}{|\mathbb{F}|}$. (b) $\Delta(\pi, \mathbf{C}) > \delta$ holds, so the prover stays in a doomed state with probability 1.

Next, assume that $\mathbf{t}_1$ is in a doomed state. Then, with probability $\epsilon_2 = \frac{1}{|\mathbb{F}|}$, $\mathbf{t}_2 = (\mathbf{t}_1, K', r')$ will be in a doomed state. We consider two cases: (a) $0 \neq \sum_{\mathbf{x}} \beta(\mathbf{r}, \mathbf{x})(f_L(\mathbf{x})f_R(\mathbf{x}) - f_O(\mathbf{x}))$, so due to the Schwartz-Zippel lemma, the sumcheck instance becomes accepting only with probability $\frac{1}{|\mathbb{F}|}$. (b) $\Delta(\pi, \mathbf{C}) > \delta$ holds so the prover stays in a doomed state with probability 1.

Let $\mathbf{t}_3 = (\mathbf{t}_2, \mathbf{t}_{smc1})$ be the state after the sumcheck protocol of step 4. If $\mathbf{t}_2$ is in a doomed state, then, from Theorem 5, with probability $\epsilon_3 = \max(\frac{2\log m}{|\mathbb{F}|}, (1-\delta)^l)$, $\mathbf{t}_3$ will remain in a doomed state. For the rest of this proof we omit the case where the sumcheck verifier fails since the prover will permanently stay in doomed state.

Assume that $\mathbf{t}_3$, then by the Schwartz-Zippel lemma, $\mathbf{t}_4 = (\mathbf{t}_3, a, b, c)$ will stay in a doomed state with probability $\epsilon_4 = \frac{3}{|\mathbb{F}|}$. Finally, let $\mathbf{t}_f = (\mathbf{t}_4, \mathbf{t}_{smc2})$ be the final transcript. By the round-by-round soundness of our coSumcheck, the probability that $\mathbf{t}_4$ is doomed and $\mathbf{t}_f$ is not, is $\epsilon_5 = \max(\frac{2\log n}{|\mathbb{F}|}, (1-\delta)^l)$.

To sum up, the round-by-round soundness error of **Construction 6** is $\epsilon_{rbr} = \max(\epsilon_1, \epsilon_2, \epsilon_3, \epsilon_4, \epsilon_5) = \max(\frac{2\log n}{|\mathbb{F}|}, \frac{2\log m}{|\mathbb{F}|}, (1-\delta)^l)$. Similarly to the proof of Theorems 5- 4 we prove round-by-round knowledge soundness. $\square$